# 5장. 참조 타입

이것이 자바다



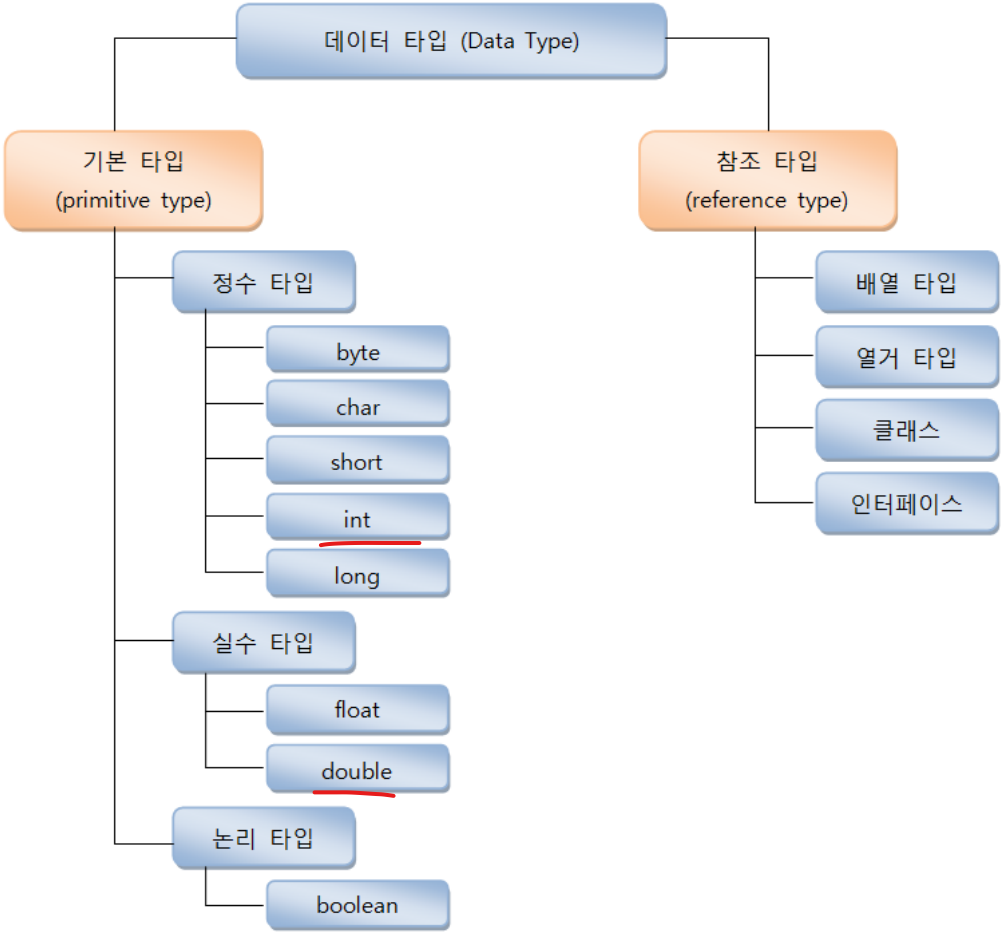
# Contents


- 1절. 데이터 타입 분류
- 2절. 메모리 사용 영역
- 3절. 참조 변수의 ==, != 연산
- 4절. null과 NullPointerException
- 5절. String 타입
- 6절. 배열 타입
- 7절. 열거 타입




# 1절. 데이터 타입 분류

## ❖ 데이터 타입 분류



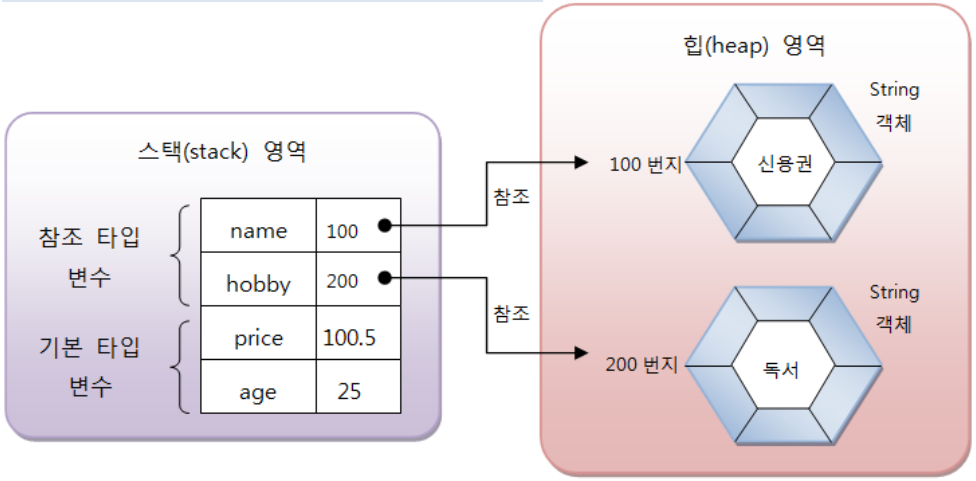
# 1절. 데이터 타입 분류

## ❖ 변수의 메모리 사용

- 기본 타입 변수 – 실제 값을 변수 안에 저장
- 참조 타입 변수 – 주소를 통해 객체 참조

```
[기본 타입 변수]
int age = 25;
double price = 100.5;

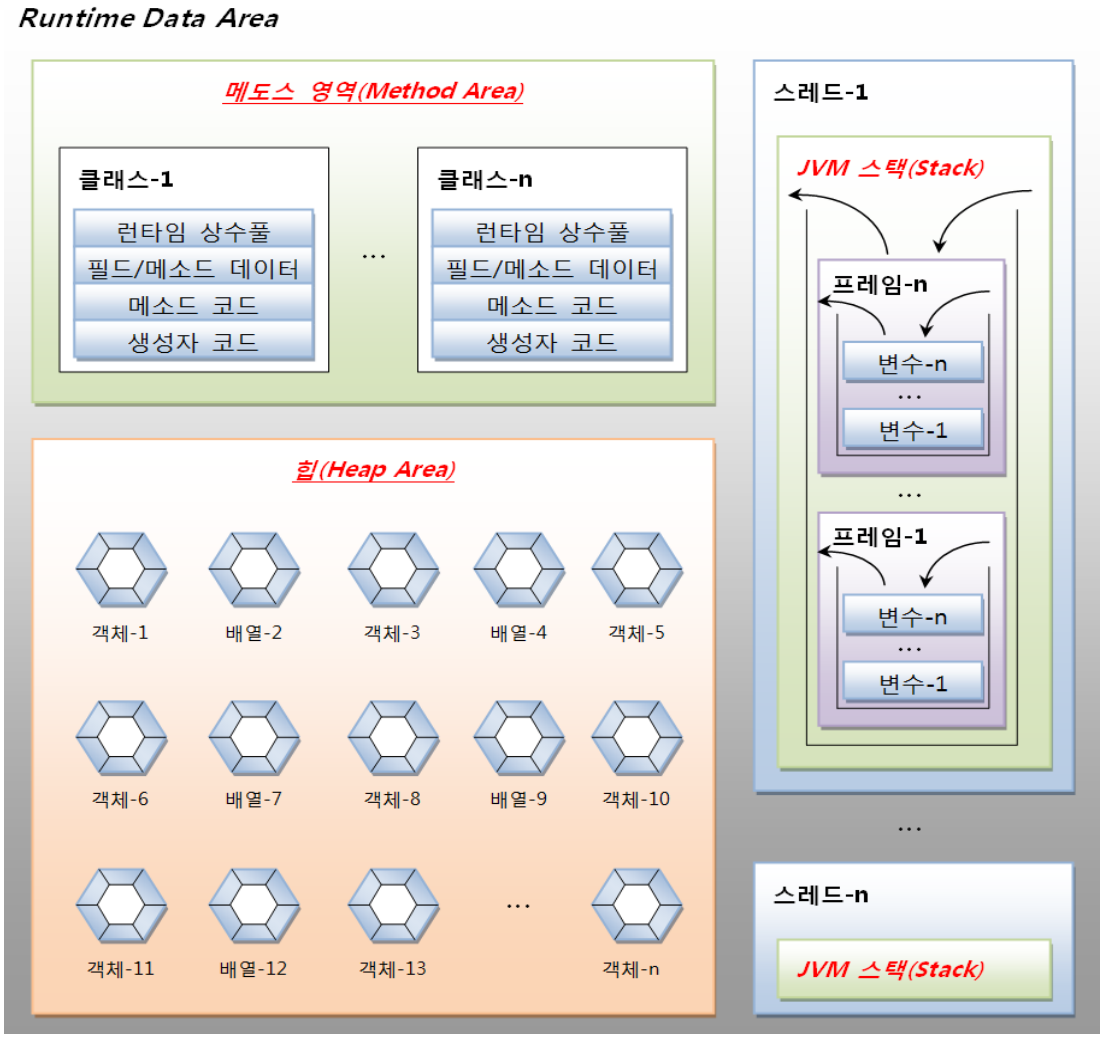
[참조 타입 변수]
String name = "신용권"
String hobby = "독서";
```



# 2절. 메모리 사용 영역

## ❖ JVM이 사용하는 메모리 영역

- OS에서 할당 받은 메모리 영역(Runtime Data Area)을 세 영역으로 구분



# 2절. 메모리 사용 영역

## ❖ JVM이 사용하는 메모리 영역 (p.140~142)

- 메소드 영역
  - JVM 시작할 때 생성
  - 로딩된 클래스 바이트 코드 내용을 분석 후 저장
  - 모든 스레드가 공유

- 힙 영역
  - JVM 시작할 때 생성
  - 객체/배열 저장
  - 사용되지 않는 객체는 Garbage Collector 가 자동 제거

- JVM 스택
  - 스레드 별 생성
  - 메소드 호출할 때마다 Frame을 스택에 추가(push)
  - 메소드 종료하면 Frame 제거(pop)

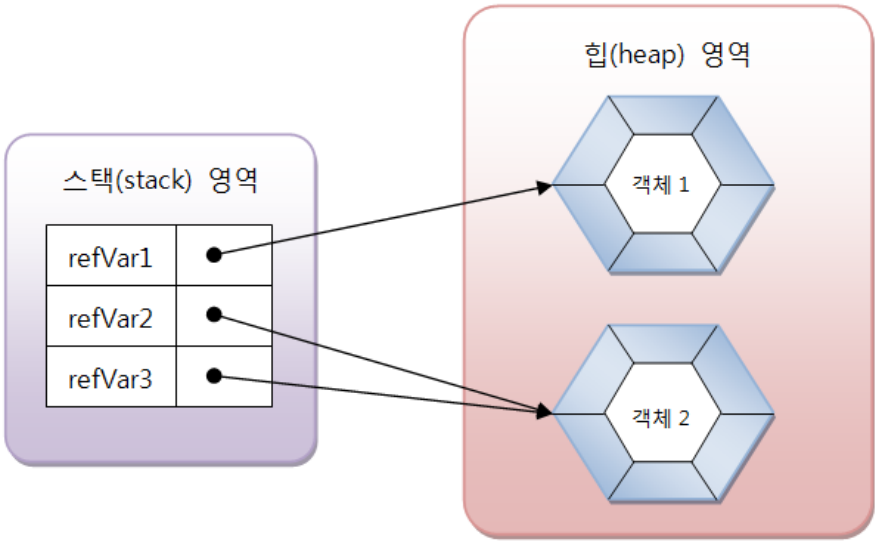


# 3절. 참조 변수의 ==, != 연산

## ❖ 변수의 값이 같은지 다른지 비교

- 기본 타입: byte, char, short, int, long, float, double, boolean
  - 의미 : 변수의 값이 같은지 다른지 조사

- 참조 타입: 배열, 열거, 클래스, 인터페이스
  - 의미 : 동일한 객체를 참조하는지 다른 객체를 참조하는지 조사




```
refVar1 == refVar2      결과:   false
refVar1 != refVar2      결과:   true
```

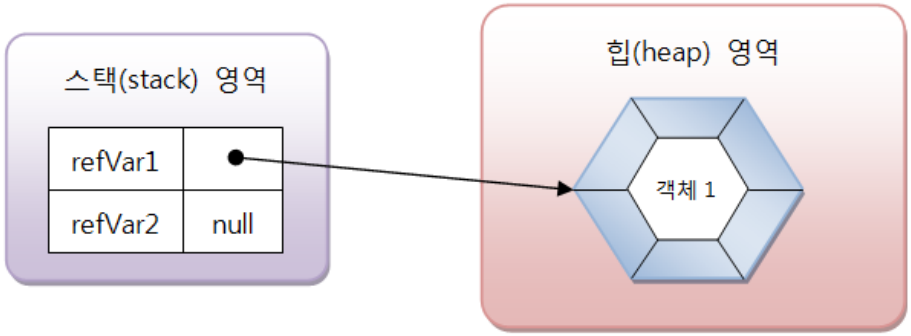
```
refVar2 == refVar3      결과:   true
refVar2 != refVar3      결과:   false
```

```
if( refVar2 == refVar3 ) { ... }
```

# 4절. null과 NullPointerException

## ❖ null(널)

- 변수가 참조하는 객체가 없을 경우 초기값으로 사용 가능
- 참조 타입의 변수에만 저장가능
- null로 초기화된 참조 변수는 스택 영역 생성




- ==, != 연산 가능

그림에서 refVar1 은 힙 영역의 객체를 참조하므로 연산의 결과는 다음과 같다.

```
refVar1 == null        결과:   false
refVar1 != null        결과:   true
```

refVar2 는 null 값을 가지므로 연산의 결과는 다음과 같다.

```
refVar2 == null        결과:   true
refVar2 != null        결과:   false
```

# 4절. null과 NullPointerException

## ❖ NullPointerException의 의미

- 예외(Exception)
  - 사용자의 잘못된 조작 이나 잘못된 코딩으로 인해 발생하는 프로그램 오류

- NullPointerException
  - 참조 변수가 null 값을 가지고 있을 때
    - 객체의 필드나 메소드를 사용하려고 했을 때 발생

```
int[] intArray = null;
intArray[0] = 10;          //NullPointerException
```

```
String str = null;
System.out.println("총 문자수: " + str.length());    //NullPointerException
```

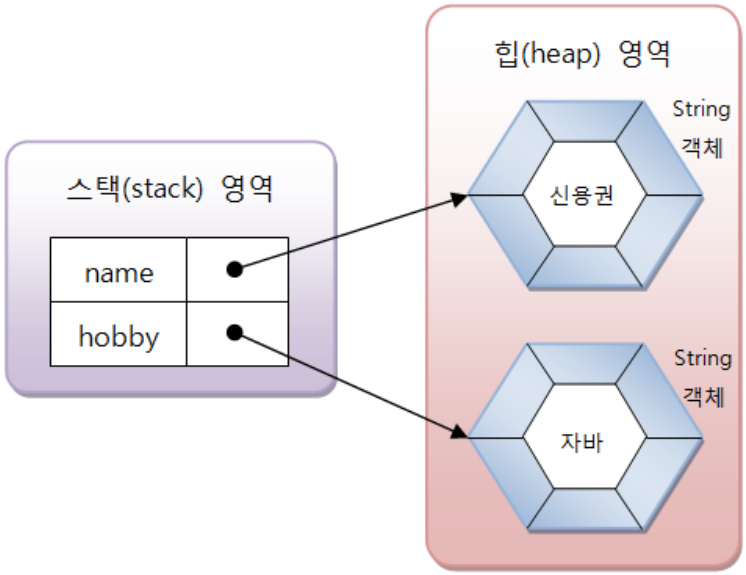


# 5절. String 타입

## ❖ String 타입 (p.145~148)

- 문자열을 저장하는 클래스 타입

```
String name;
name = "신용권";
String hobby = "자바";
```

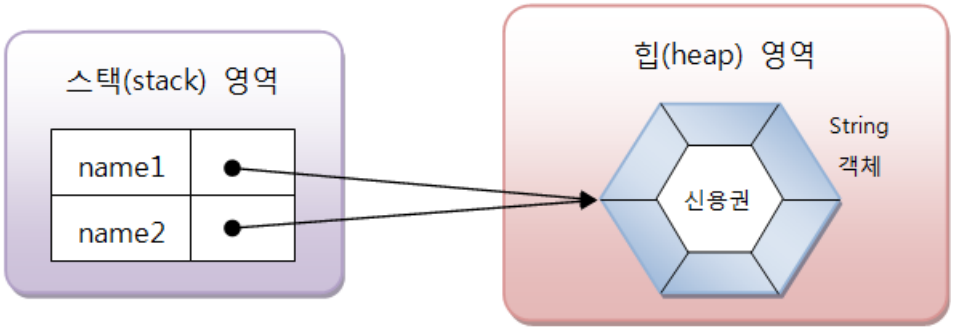


# 5절. String 타입

## ❖ String 타입 (p.145~148)

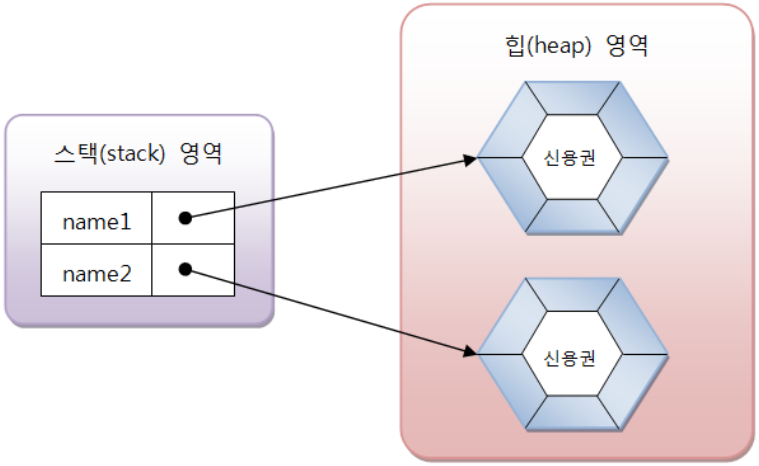
- 문자열 리터럴 동일하다면 String 객체 공유

```
String name1 = "신용권";
String name2 = "신용권";
```




- new 연산자를 이용한 String 객체 생성
  - 힙 영역에 새로운 String 객체 생성
  - String 객체를 생성한 후 번지 리턴

```
String name1 = new String("신용권");
String name2 = new String("신용권");
```



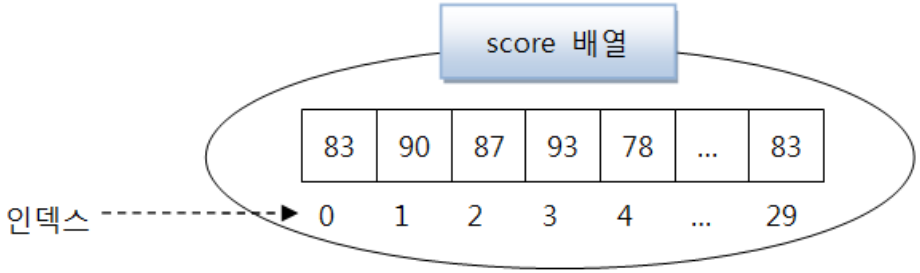
# 6절. 배열 타입

## ❖ 배열이란?

- 같은 타입의 데이터를 연속된 공간에 저장하는 자료구조

- 각 데이터 저장 위치는 인덱스 부여해 접근

```
int score1= 83;
int score2 = 90;
int score3 = 87;
 :
int score30= 75;
```




항목 접근: 배열이름[인덱스] ex) score[0], score[3]

# StringEqualsExample.java_문자열 비교

```
1 2public class StringEqualsExample {
2     public static void main(String[] args) {
3         String strVar1 = "신민철";
4         String strVar2 = "신민철";
5
6         if(strVar1 == strVar2) {
7             System.out.println("strVar1과 strVar2는 참조가 같음");
8         } else {
9             System.out.println("strVar1과 strVar2는 참조가 다름");
10        }
11
12        if(strVar1.equals(strVar2)) {
13            System.out.println("strVar1과 strVar2는 문자열이 같음");
14        }
15
16        String strVar3 = new String("신민철");
17        String strVar4 = new String("신민철");
18
19        if(strVar3 == strVar4) {
20            System.out.println("strVar3과 strVar4는 참조가 같음");
21        } else {
22            System.out.println("strVar3과 strVar4는 참조가 다름");
23        }
24
25        if(strVar3.equals(strVar4)) {
26            System.out.println("strVar3과 strVar4는 문자열이 같음");
27        }
28    }
29 }
```

```
C:₩Windows₩system32₩cmd.exe
strVar1과 strVar2는 참조가 같음
strVar1과 strVar2는 문자열이 같음
strVar3과 strVar4는 참조가 다름
strVar3과 strVar4는 문자열이 같음
계속하려면 아무 키나 누르십시오 . . .
```

equals() 메서드 활용하기



# 6절. 배열 타입

## ❖ 배열의 장점

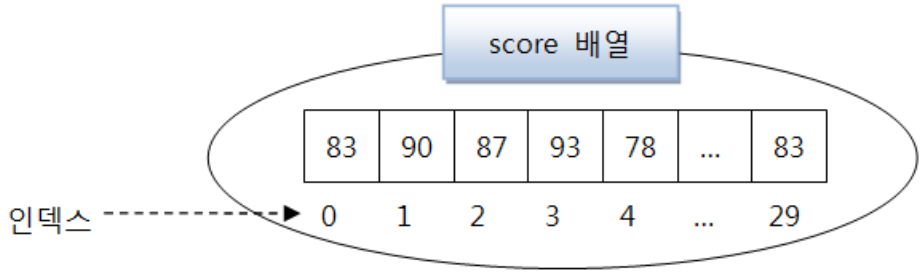
- 중복된 변수 선언 줄이기 위해 사용

- 반복문 이용해 요소들을 쉽게 처리

```
int sum = score1;
sum += score2;
sum += score3;
  :
sum += score30;
int avg = sum / 30;
```

```
int sum = 0;
for(int i=0; i<30; i++) {
   sum += score[i];
}
int avg = sum / 30;
```

```
int score1= 83;
int score2 = 90;
int score3 = 87;
 :
int score30= 75;
```




항목 접근: 배열이름[인덱스] ex) score[0], score[3]

# 6절. 배열 타입

## ❖ 배열 선언

- 배열을 사용하기 위해 우선 배열 변수 선언

```
타입[ ] 변수;
```

```
타입 변수[ ];
```

```
int[] intArray;
double[] doubleArray;
String[] strArray;
```

```
int intArray[];
double doubleArray[];
String strArray[];
```

- 배열 변수는 참조 변수 – 배열 생성되기 전 null로 초기화 가능

```
타입[] 변수 = null;
```

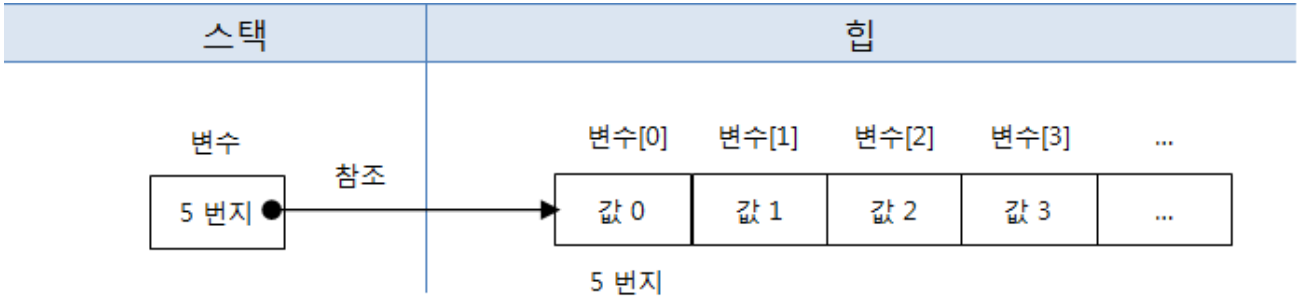
  - 배열 변수가 null 값을 가진 상태에서 항목에 접근 불가
    - 변수[인덱스]” 못함
      - » NullPointerException 발생

# 6절. 배열 타입

## ❖ 값 목록으로 배열 생성하는 방법

### ▪ 변수 선언과 동시에 값 목록 대입

```
데이터타입[] 변수 = { 값 0, 값 1, 값 2, 값 3, ... };
```

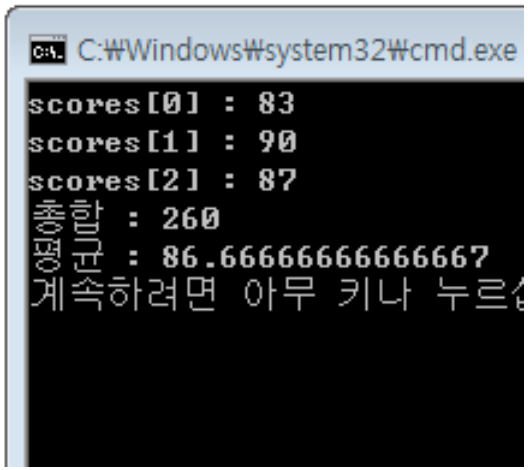



### ▪ 변수 선언 후 값 목록 대입

```
데이터타입[] 변수;
변수 = new 타입[] { 값 0, 값 1, 값 2, 값 3, ... };
```

# ArrayCreateByValueListExample1.java_값 목록으로 배열 생성

```
1  public class ArrayCreateByValueListExample1 {
2      public static void main(String[] args) {
3          int[] scores = { 83, 90, 87 };
4
5          System.out.println("scores[0] : " + scores[0]);
6          System.out.println("scores[1] : " + scores[1]);
7          System.out.println("scores[2] : " + scores[2]);
8
9          int sum = 0;
10         for(int i=0; i<3; i++) {
11             sum += scores[i];
12         }
13         System.out.println("총합 : " + sum);
14         double avg = (double) sum / 3;
15         System.out.println("평균 : " + avg);
16     }
17 }
```



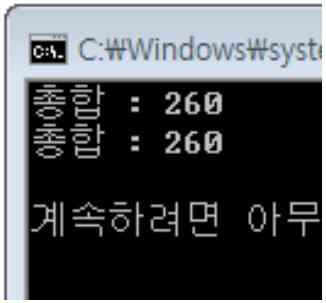

scores.lenght

# ArrayCreateByValueListExample2.java_값의 리스트로 배열 생성

```
1  public class ArrayCreateByValueListExample2 {
2      public static void main(String[] args) {
3          int[] scores;
4          scores = new int[] { 83, 90, 87 };
5          int sum1 = 0;
6          for(int i=0; i<3; i++) {
7              sum1 += scores[i];
8          }
9          System.out.println("총합 : " + sum1);
10
11         int sum2 = add( new int[] { 83, 90, 87 } );
12         System.out.println("총합 : " + sum2);
13         System.out.println();
14     }
15
16     public static int add(int[] scores) {
17         int sum = 0;
18         for(int i=0; i<3; i++) {
19             sum += scores[i];
20         }
21         return sum;
22     }
```

새로운 배열임

인자의 형태를



# 6절. 배열 타입

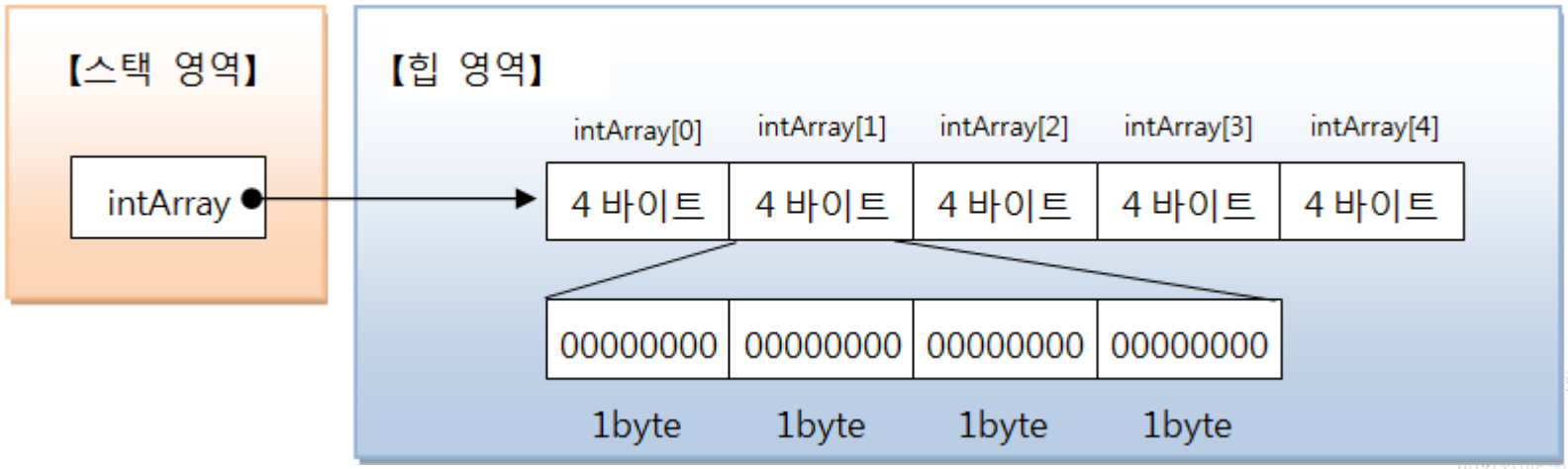
## ❖ new 연산자로 배열 생성

- 배열 생성시 값 목록을 가지고 있지 않음
- 향후 값들을 저장할 배열을 미리 생성하고 싶을 경우

```
타입[] 변수 = new 타입[길이];
```

```
타입[] 변수 = null;
변수 = new 타입[길이];
```

```
int[] intArray = new int[5];
```



# 6절. 배열 타입

## ❖ 타입 별 항목의 기본값

| 분류 | 데이터 타입 | 초기값 |
|---|---|---|
| 기본 타입 (정수) | byte[] | 0 |
| | char[] | '₩u0000' |
| | short[] | 0 |
| | int[] | 0 |
| | long[] | 0L |
| 기본 타입 (실수) | float[] | 0.0F |
| | double[] | 0.0 |
| 기본 타입 (논리) | boolean[] | false |
| 참조 타입 | 클래스[] | null |
| | 인터페이스[] | null |



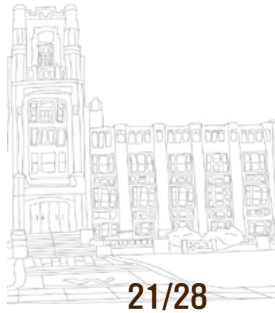
# ArrayCreateByNewExample.java_new 연산자로 배열 생성

```java
package sec06.exam02_array_bynew;
public class ArrayCreateByNewExample {
    public static void main(String[] args) {
        int[] arr1 = new int[3];
        for(int i=0; i<3; i++) {
            System.out.println("arr1[" + i + "] : " + arr1[i]);
        }
        arr1[0] = 10;
        arr1[1] = 20;
        arr1[2] = 30;
        for(int i=0; i<3; i++) {
            System.out.println("arr1[" + i + "] : " + arr1[i]);
        }

        double[] arr2 = new double[3];
        for(int i=0; i<3; i++) {
            System.out.println("arr2[" + i + "] : " + arr2[i]);
        }
        arr2[0] = 0.1;
        arr2[1] = 0.2;
        arr2[2] = 0.3;
        for(int i=0; i<3; i++) {
            System.out.println("arr2[" + i + "] : " + arr2[i]);
        }
        String[] arr3 = new String[3];
        for(int i=0; i<3; i++) {
            System.out.println("arr3[" + i + "] : " + arr3[i]);
        }
        arr3[0] = "1월";
        arr3[1] = "2월";
        arr3[2] = "3월";
        for(int i=0; i<3; i++) {
            System.out.println("arr3[" + i + "] : " + arr3[i]);
        }
    }

}
```

```
arr1[0] : 0
arr1[1] : 0
arr1[2] : 0
arr1[0] : 10
arr1[1] : 20
arr1[2] : 30
arr2[0] : 0.0
arr2[1] : 0.0
arr2[2] : 0.0
arr2[0] : 0.1
arr2[1] : 0.2
arr2[2] : 0.3
arr3[0] : null
arr3[1] : null
arr3[2] : null
arr3[0] : 1월
arr3[1] : 2월
arr3[2] : 3월
```



# 6절. 배열 타입

## ❖ 배열의 길이

- 배열에 저장할 수 있는 전체 항목 수
- 코드에서 배열의 길이 얻는 방법

```
배열변수.length;
```

```
int[] intArray = { 10, 20, 30 };
int num = intArray.length;
```

- 배열의 길이는 읽기 전용

```
intArray.length = 10;   //잘못된 코드
```

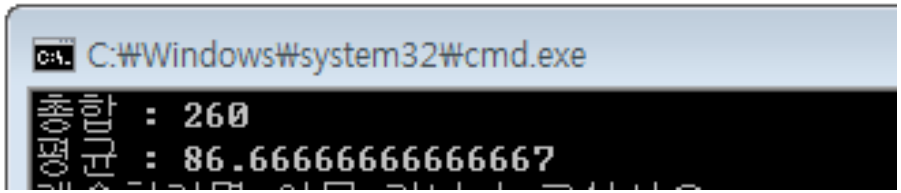
- 배열의 길이는 for문의 조건식에서 주로 사용

```
int[] scores = { 83, 90, 87 };

int sum = 0;                                    3
for(int i=0; i<scores.length; i++) {
   sum += scores[i];
}
System.out.println("총합 : " + sum);
```

# ArrayLengthExample.java_ 배열의 Length 필드

```
1  public class ArrayLengthExample {
2      public static void main(String[] args) {
3          int[] scores = { 83, 90, 87 };
4
5          int sum = 0;
6          for(int i=0; i<scores.length; i++) {
7              sum += scores[i];
8          }
9          System.out.println("총합 : " + sum);
10
11         double avg = (double) sum / scores.length;
12         System.out.println("평균 : " + avg);
13     }
14 }
15
```



# 6절. 배열 타입

## ❖ 다차원 배열

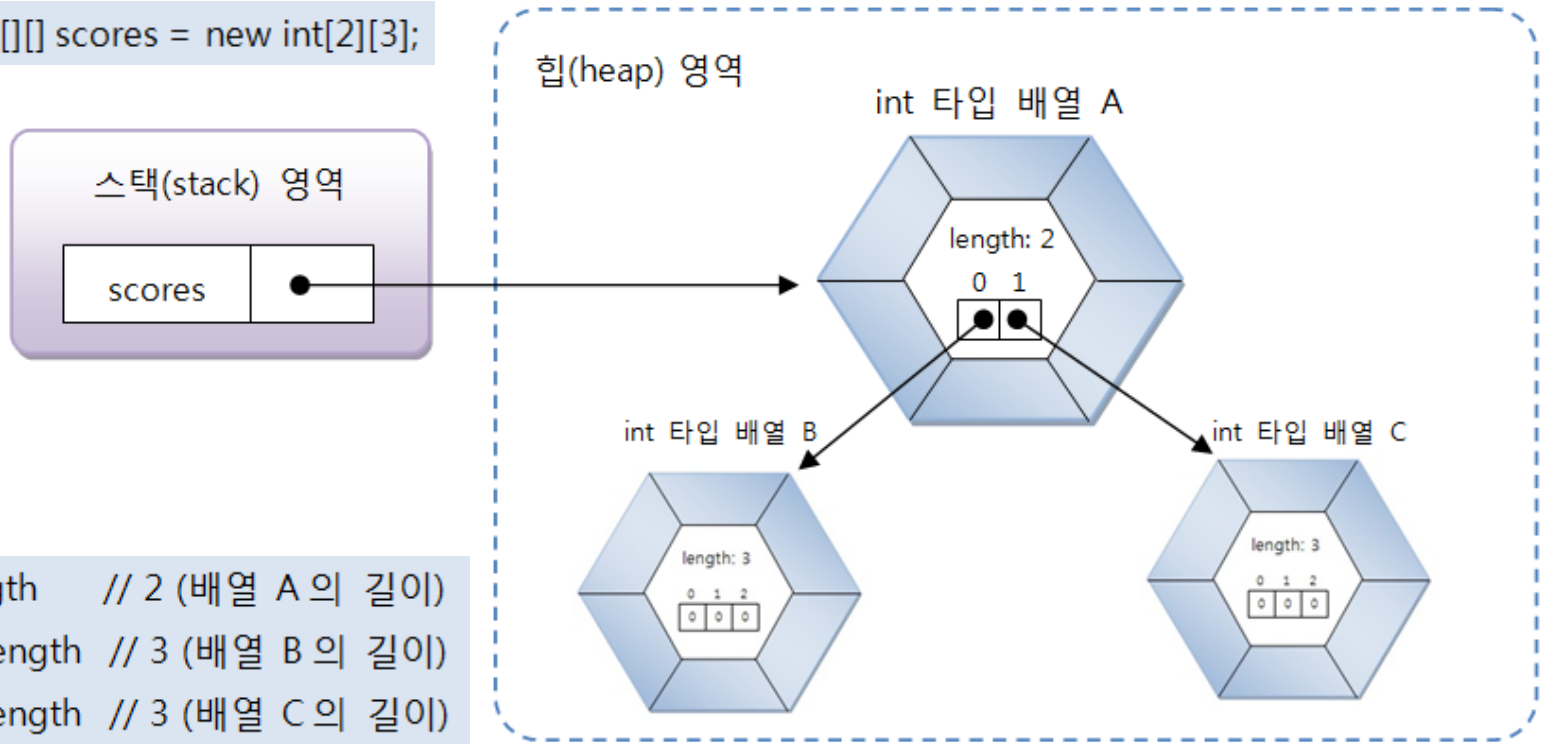
- 2차원 배열 이상의 배열
  - 수학의 행렬과 같은 자료 구조

【2 x 3 행렬의 구조】

| 행 \ 열 | 0 | 1 | 2 |
|---|---|---|---|
| 0 | (0,0) | (0,1) | (0,2) |
| 1 | (1,0) | (1,1) | (1,2) |

- 자바는 1차원 배열을 이용해 2차원 배열 구현

```
int[][] scores = new int[2][3];
```




```
scores.length     // 2 (배열 A의 길이)
scores[0].length  // 3 (배열 B의 길이)
scores[1].length  // 3 (배열 C의 길이)
```

# ArrayInArrayExample.java_ 배열 속의 배열

```java
package sec06.exam05_array_in_array;

public class ArrayInArrayExample {
    public static void main(String[] args) {

        int[][] mathScores = new int[2][3];
        for(int i=0; i<mathScores.length; i++) {
            for(int k=0; k<mathScores[i].length; k++) {
                System.out.println("mathScores["+i+"]["+k+"]="
                        +mathScores[i][k]);
            }
        }
        System.out.println();

        int[][] englishScores = new int[2][];
        englishScores[0] = new int[2];
        englishScores[1] = new int[3];
        for(int i=0; i<englishScores.length; i++) {
            for(int k=0; k<englishScores[i].length; k++) {
                System.out.println("englishScores["+i+"]["+k+"]=" + englishScores[i][k]);
            }
        }

        System.out.println();

        int[][] javaScores = { {95, 80}, {92, 96, 80} };
        for(int i=0; i<javaScores.length; i++) {
            for(int k=0; k<javaScores[i].length; k++) {
                System.out.println("javaScores["+i+"]["+k+"]="
                        +javaScores[i][k]);
            }
        }
    }
}
```

행

열

new int [][3] X 안 됨
배열 객체 주소 참조가 불가능




```
mathScores[0][0]=0
mathScores[0][1]=0
mathScores[0][2]=0
mathScores[1][0]=0
mathScores[1][1]=0
mathScores[1][2]=0

englishScores[0][0]=0
englishScores[0][1]=0
englishScores[1][0]=0
englishScores[1][1]=0
englishScores[1][2]=0

javaScores[0][0]=95
javaScores[0][1]=80
javaScores[1][0]=92
javaScores[1][1]=96
javaScores[1][2]=80
계속하려면 아무 키나 누
```

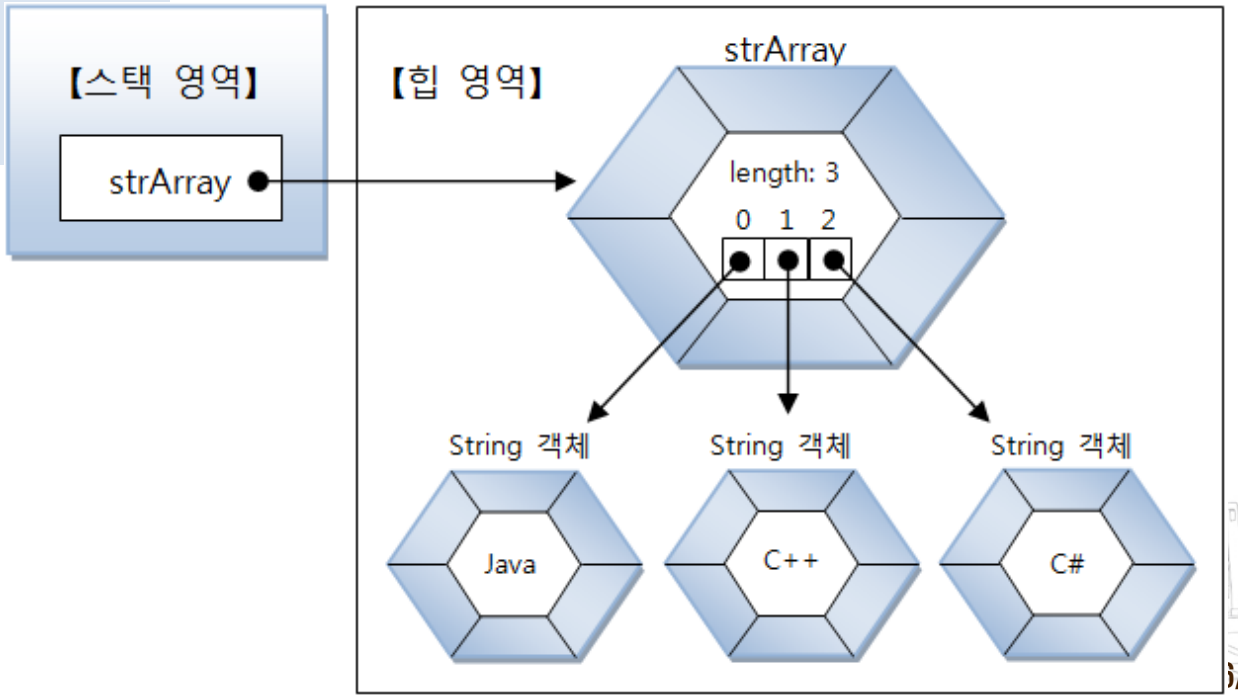
# 6절. 배열 타입

## ❖ 객체를 참조하는 배열

- 기본 타입(byte, char, short, int, long, float, double, boolean) 배열
  - 각 항목에 직접 값을 가지고 있음

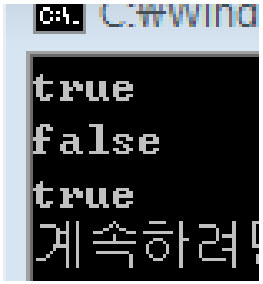
- 참조 타입(클래스, 인터페이스) 배열 – 각 항목에 객체의 번지 가짐

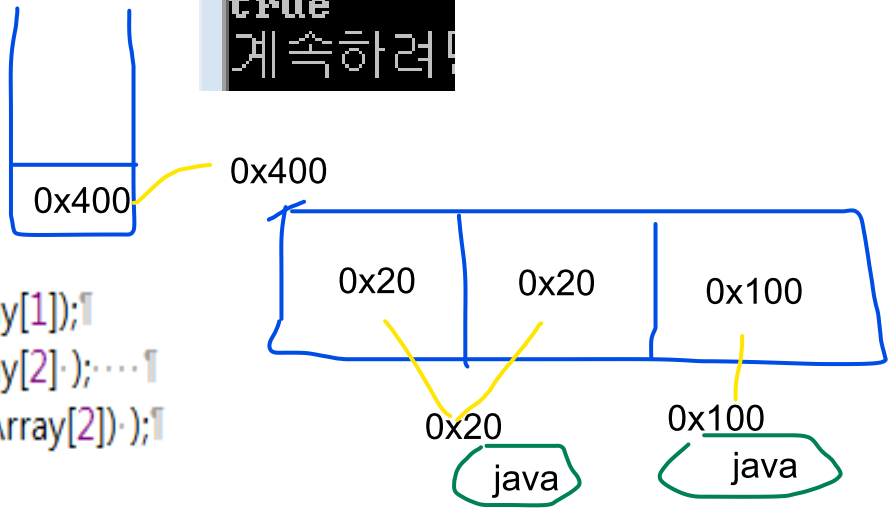
```
String[] strArray = new String[3];
strArray[0] = "Java";
strArray[1] = "C++";
strArray[2] = "C#";
```



# ArrayReferenceObjectExample.java_ 객체를 참조하는 배열

```
1  package sec06.exam06_array_reference_object;
2  public class ArrayReferenceObjectExample {
3      public static void main(String[] args) {
4          String[] strArray = new String[3];
5          strArray[0] = "Java";
6          strArray[1] = "Java";
7          strArray[2] = new String("Java");
8
9          System.out.println( strArray[0] == strArray[1]);
10         System.out.println( strArray[0] == strArray[2] );
11         System.out.println( strArray[0].equals(strArray[2]) );
12     }
13 }
14
15
```




"java" 리터럴은 클래스가 로딩될 때
힙영역 String Constant Pool(문자열 상수풀)에 String 객체로 생성

# 6절. 배열 타입

## ❖ 배열 복사

- 배열은 한 번 생성하면 크기 변경 불가
- 더 많은 저장 공간이 필요하다면 보다 큰 배열을 새로 만들고 이전 배열로부터 항목 값들을 복사

## ❖ 배열 복사 방법

- for문 이용
- System.arrayCopy() 메소드 이용
- Arrays 클래스 이용

가변적임

```
ArrayList<Integer> calculator = new ArrayList<>();
```

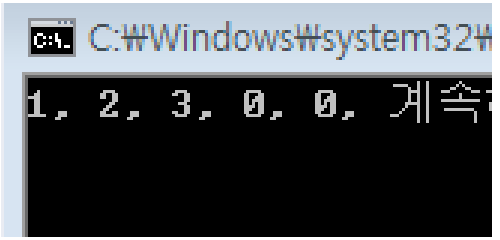
get(i)  size()

```
calculator.remove(Integer.valueOf(5));
```



# ArrayCopyByforExample.java_ for문으로 배열 복사

```
1  package sec06.exam07_array_copy;
2
3  public class ArrayCopyByForExample {
4      public static void main(String[] args) {
5          int[] oldIntArray = { 1, 2, 3 };
6          int[] newIntArray = new int[5];
7
8          for(int i=0; i<oldIntArray.length; i++) {
9              newIntArray[i] = oldIntArray[i];
10         }
11
12         for(int i=0; i<newIntArray.length; i++) {
13             System.out.print(newIntArray[i] + ", ");
14         }
15     }
16 }
```

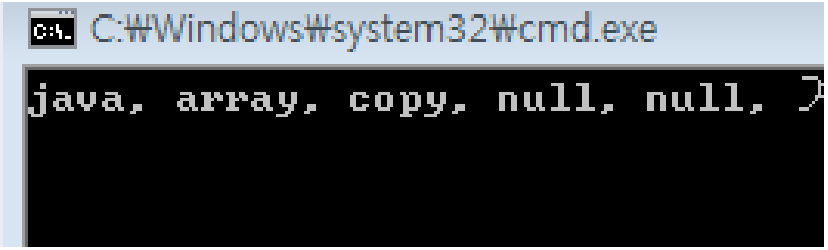



스택에 배열 객체 주소가지고 있는 변수명

# ArrayCopyExample.java_ System.arraycopy()로 배열 복사

```
1  package sec06.exam07_array_copy;
2
3
4  public class ArrayCopyExample {
5      public static void main(String[] args) {
6          String[] oldStrArray = {"java", "array", "copy"};
7          String[] newStrArray = new String[5];
8
                                          n          m
9          System.arraycopy( oldStrArray, 0, newStrArray, 0, oldStrArray.length);
10                          원본 시작위치, 새 시작위치
11         for(int i=0; i<newStrArray.length; i++) {
12             System.out.print(newStrArray[i] + ", ");
13         }
14     }
15 }
```

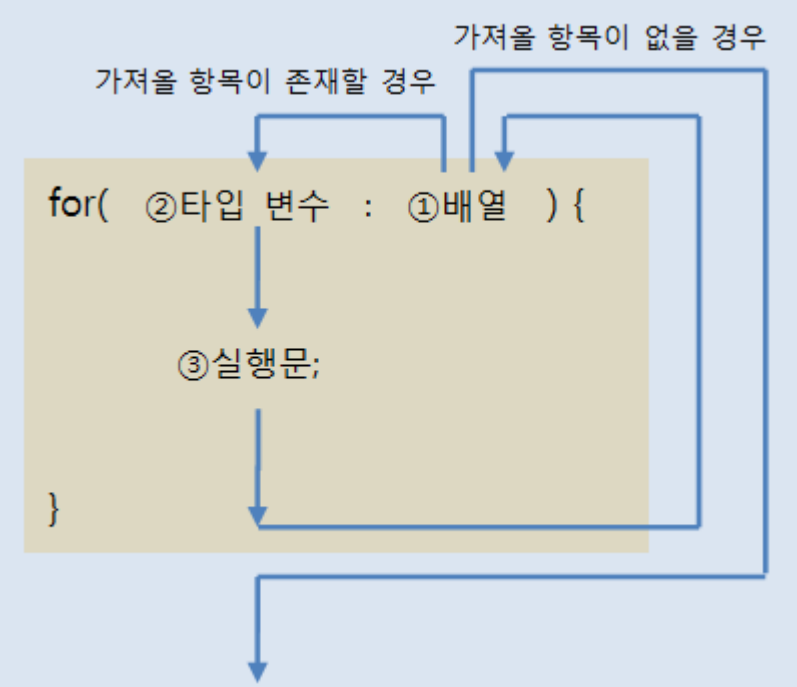
주의: oldStrArray.length-n



# 6절. 배열 타입

## ❖ 향상된 for 문

- 배열 및 컬렉션(15장에서 다룸)의 항목 요소를 순차적으로 처리
- 인덱스 이용하지 않고 바로 항목 요소 반복




```
int[] scores = { 95, 71, 84, 93, 87 };

int sum = 0;
for (int score : scores) {
    sum = sum + score;
}
```

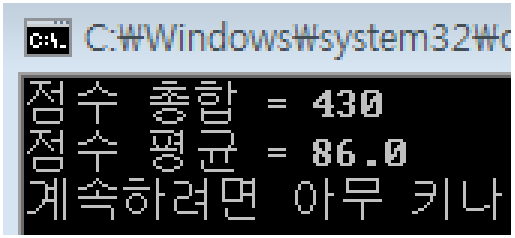
# AdvancedForExample.java_ 향상된 for문

```java
package sec06.exam08_advanced_for;
public class AdvancedForExample {
    public static void main(String[] args) {
        int[] scores = { 95, 71, 84, 93, 87 };

        int sum = 0;
        for (int score : scores) {
            sum = sum + score;
        }
        System.out.println("점수 총합 = " + sum);

        double avg = (double) sum / scores.length;
        System.out.println("점수 평균 = " + avg);

    }
}
```



# 7절. 열거 타입

## ❖ 열거 타입(Enumeration Type)

- 한정된 값만을 갖는 데이터 타입

- 한정된 값은 열거 상수(Enumeration Constant)로 정의



# 7절. 열거 타입

## ❖ 열거 타입 선언

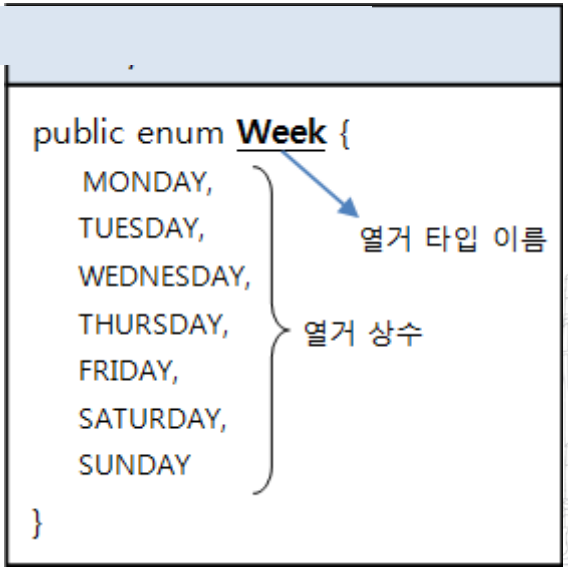
- 파일 이름과 동일한 이름으로 다음과 같이 선언 (첫 글자 대문자)

```
public enum 열거타입이름 { ... }
```

- 한정된 값인 열거 상수 정의
  - 열거 상수 이름은 관례적으로 모두 대문자로 작성
  - 다른 단어가 결합된 이름일 경우 관례적으로 밑줄( _ )로 연결

```
public enum Week { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, ... }
```

```
public enum LoginResult { LOGIN_SUCCESS, LOGIN_FAILED }
```



# Week.java_ 열거 타입 선언

```
1  public enum Week {
2      MONDAY,
3      TUESDAY,
4      WEDNESDAY,
5      THURSDAY,
6      FRIDAY,
7      SATURDAY,
8      SUNDAY
9  }
10
```



# 7절. 열거 타입

## ❖ 열거 타입 변수

- 열거 타입 변수 선언

```
열거타입 변수;
```

```
Week today;
Week reservationDay;
```

- 열거 상수 값 저장 – 열거 타입 변수값은 열거 상수 중 하나

```
열거타입 변수 = 열거타입.열거상수;
```

```
Week today = Week.SUNDAY;
```

- 열거 타입 변수는 참조 타입
  - 열거 타입 변수는 참조 타입이므로 null 값 저장 가능

```
Week birthday = null;
```



# EnumWeek.java_ 열거 타입과 열거 상수

```java
package sec07.exam01_enum;

import java.util.Calendar;

public class EnumWeekExample {
    public static void main(String[] args) {
        Week today = null;

        Calendar cal = Calendar.getInstance();
        int week = cal.get(Calendar.DAY_OF_WEEK);

        switch(week) {
            case 1:
                today = Week.SUNDAY; break;
            case 2:
                today = Week.MONDAY; break;
            case 3:
                today = Week.TUESDAY; break;
            case 4:
                today = Week.WEDNESDAY; break;
            case 5:
                today = Week.THURSDAY; break;
            case 6:
                today = Week.FRIDAY; break;
            case 7:
                today = Week.SATURDAY; break;
        }

        System.out.println("오늘 요일: "+ today);

        if(today == Week.SUNDAY) {
            System.out.println("일요일에는 축구를 합니다.");
        } else {
            System.out.println("열심히 자바 공부합니다.");
        }
    }
}
```

싱글톤 객체로 생성해서 그 객체의 주소를 반환해줌

열거 객체형

```
C:\Windows\System32\cmd.exe
오늘 요일: FRIDAY
열심히 자바 공부합니다.
계속하려면 아무 키나 누르십시오 . . .
```

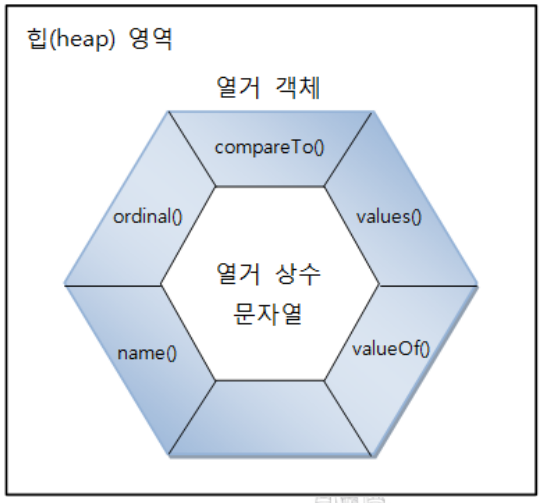
# 7절. 열거 타입

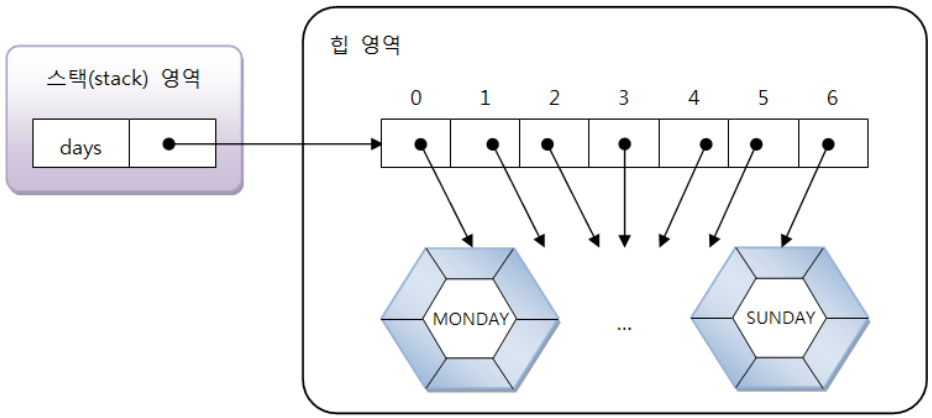
## ❖ 열거 객체의 메소드 (p.176~180)

- 열거 객체는 열거 상수의 문자열을 내부 데이터로 가지고 있음

- 열거 타입은 컴파일 시 java.lang.Enum 클래스를 자동 상속
  - 열거 객체는 java.lang.Enum 클래스의 메소드 사용 가능

| 리턴타입 | 메소드(매개변수) | 설명 |
|---|---|---|
| String | name() | 열거 객체의 문자열을 리턴 |
| int | ordinal() | 열거 객체의 순번(0 부터 시작)를 리턴 |
| int | compareTo() | 열거 객체를 비교해서 순번 차이를 리턴 |
| 열거타입 | valueOf(String name) | 주어진 문자열의 열거 객체를 리턴 |
| 열거배열 | values() | 모든 열거 객체들을 배열로 리턴 |



# EnumMethodExample.java_ 열거 객체의 메소드

```java
package sec07.exam01_enum;

public class EnumMethodExample {
    public static void main(String[] args) {
        //name() 메소드
        Week today = Week.SUNDAY;
        String name = today.name();
        System.out.println(name);

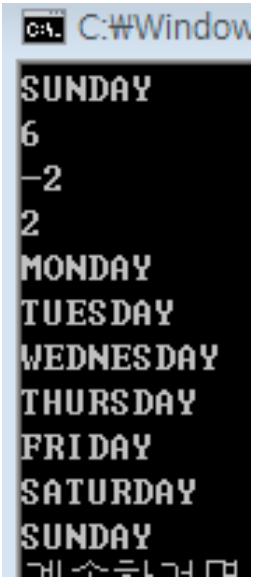
        //ordinal() 메소드
        int ordinal = today.ordinal();
        System.out.println(ordinal);

        //compareTo() 메소드
        Week day1 = Week.MONDAY;
        Week day2 = Week.WEDNESDAY;
        int result1 = day1.compareTo(day2);
        int result2 = day2.compareTo(day1);
        System.out.println(result1);
        System.out.println(result2);

        //valueOf() 메소드
        /*Week weekDay = Week.valueOf("SUNDAY");
        if(weekDay == Week.SATURDAY || weekDay == Week.SUNDAY) {
            System.out.println("주말 이군요");
        } else {
            System.out.println("평일 이군요");
        }*/

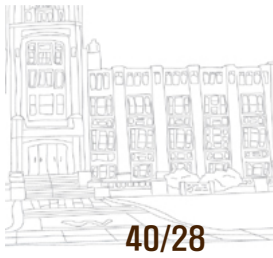
        if(args.length == 1) {
            String strDay = args[0];
            Week weekDay = Week.valueOf(strDay);
            if(weekDay == Week.SATURDAY || weekDay == Week.SUNDAY) {
                System.out.println("주말 이군요");
            } else {
                System.out.println("평일 이군요");
            }
        }

        //values() 메소드
        Week[] days = Week.values();
        for(Week day : days) {
            System.out.println(day);
        }
    }
}
```



# 6장. 클래스

이것이 자바다



# Contents


- 1절. 객체 지향 프로그래밍
- 2절. 객체(Object)와 클래스(Class)
- 3절. 클래스 선언
- 4절. 객체 생성과 클래스 변수
- 5절. 클래스의 구성 멤버
- 6절. 필드(Field)
- 7절. 생성자(Constructor)
- 8절. 메소드(Method)
- 9절. 인스턴스 멤버와 this
- 10절. 정적 멤버와 static
- 11절. final 필드와 상수(static final)
- 12절. 패키지(package)
- 13절. 접근 제한자
- 14절. Getter와 Setter
- 15절. 어노테이션(Annotation)



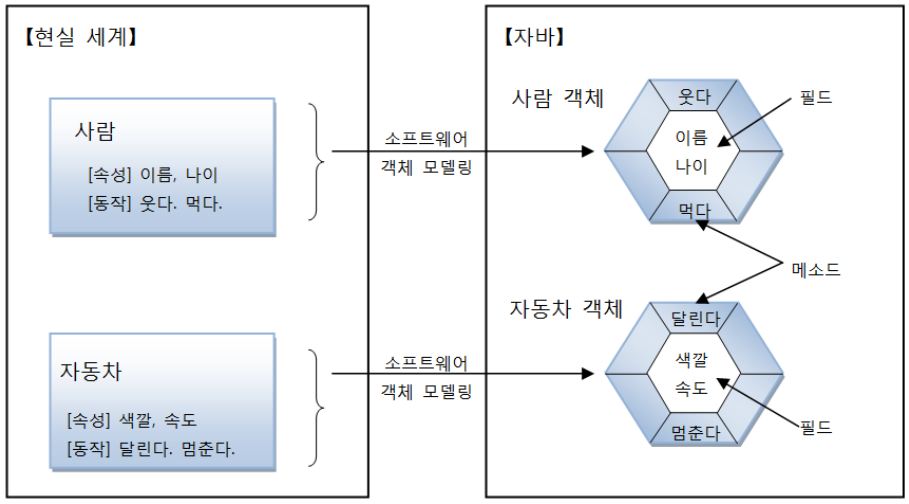

# 1절. 객체 지향 프로그래밍

## ❖ 객체 지향 프로그래밍

- OOP: Object Oriented Programming
- 부품 객체를 먼저 만들고 이것들을 하나씩 조립해 완성된 프로그램을 만드는 기법

## ❖ 객체(Object)란?

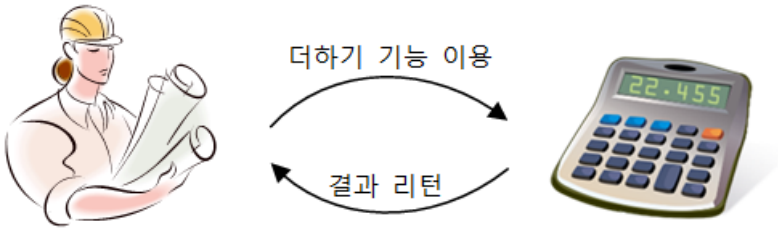
- 물리적으로 존재하는 것 (자동차, 책, 사람)
- 추상적인 것(회사, 날짜) 중에서 자신의 속성과 동작을 가지는 모든 것
- 객체는 필드(속성) 과 메소드(동작)로 구성된 자바 객체로 모델링 가능

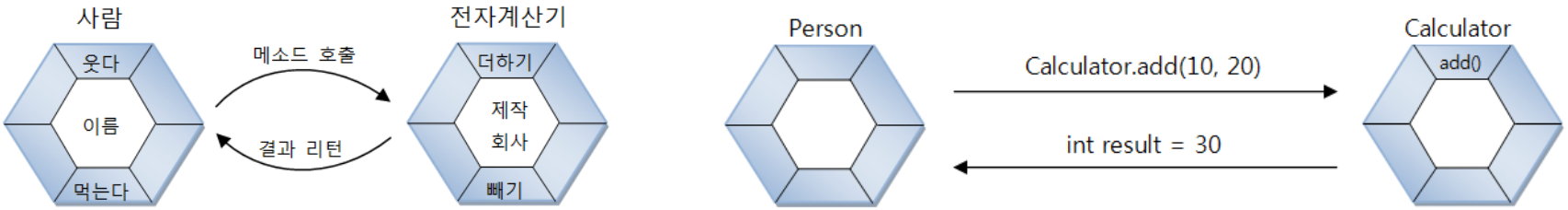


# 1절. 객체 지향 프로그래밍

## ❖ 객체의 상호 작용

- 객체들은 서로 간에 기능(동작)을 이용하고 데이터를 주고 받음

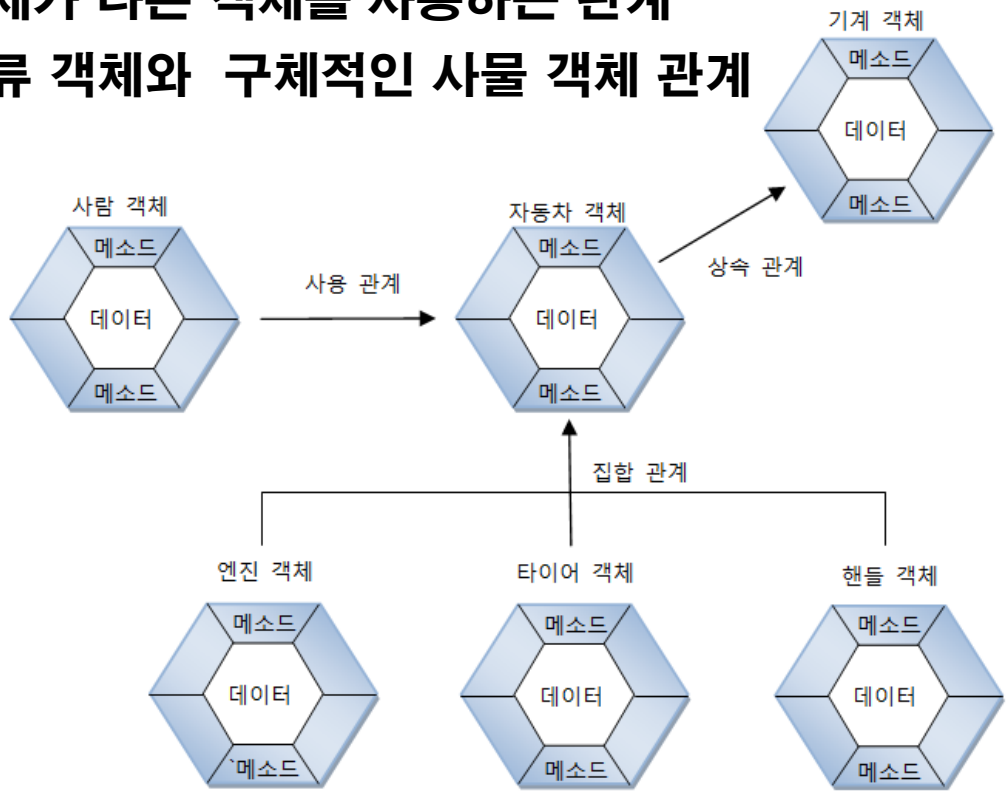


# 1절. 객체 지향 프로그래밍

## ❖ 객체간의 관계

- 객체 지향 프로그램에서는 객체는 다른 객체와 관계를 맺음
- 관계의 종류
  - 집합 관계: 완성품과 부품의 관계
  - 사용 관계: 객체가 다른 객체를 사용하는 관계
  - 상속 관계: 종류 객체와  구체적인 사물 객체 관계



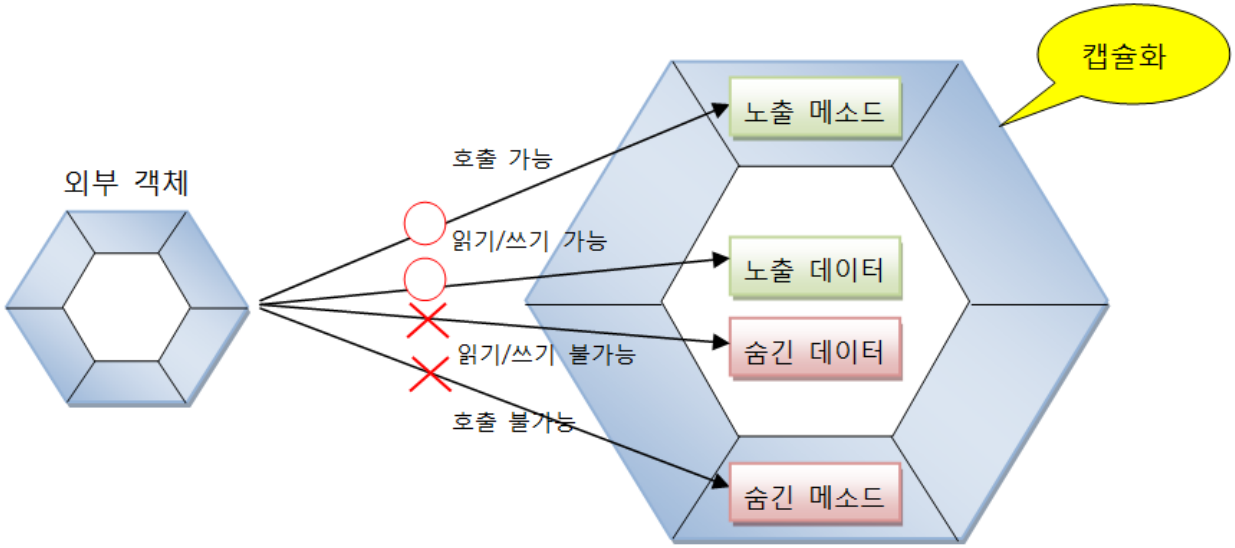
# 1절. 객체 지향 프로그래밍

## ❖ 객체 지향 프로그래밍의 특징

getinstanceof 도 일종의 캡슐화

### ▪ 캡슐화

- 객체의 필드, 메소드를 하나로 묶고, 실제 구현 내용을 감추는 것
- 외부 객체는 객체 내부 구조를 알지 못하며 객체가 노출해 제공하는 필드와 메소드만 이용 가능
- 필드와 메소드를 캡슐화하여 보호하는 이유는 외부의 잘못된 사용으로 인해 객체가 손상되지 않도록
- 자바 언어는 캡슐화된 멤버를 노출시킬 것인지 숨길 것인지 결정하기 위해 접근 제한자(Access Modifier) 사용

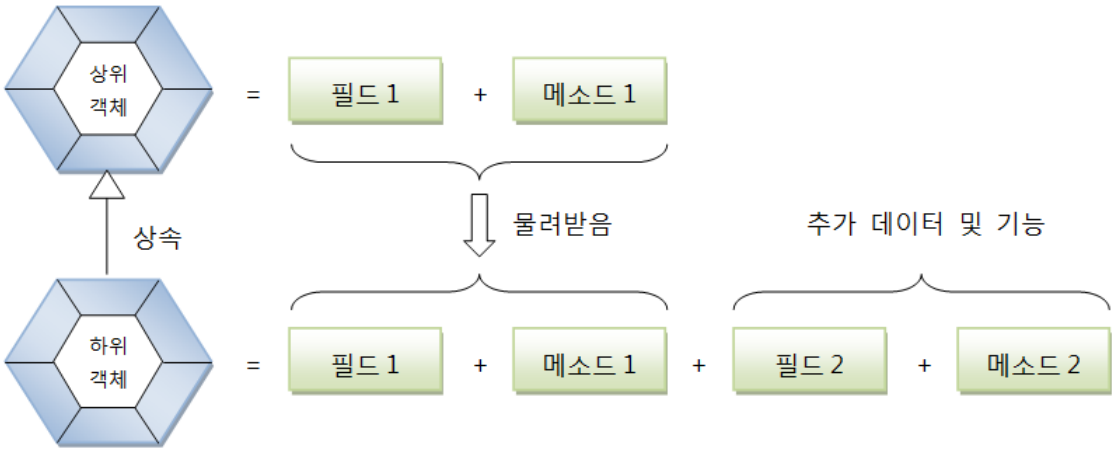


# 1절. 객체 지향 프로그래밍

## ❖ 객체 지향 프로그래밍의 특징

### ▪ 상속

- 상위(부모) 객체의 필드와 메소드를 하위(자식) 객체에게 물려주는 행위
- 하위 객체는 상위 객체를 확장해서 추가적인 필드와 메소드를 가질 수 있음
- 상속 대상: 필드와 메소드
- 상속의 효과
  - 상위 객체를 재사용해서 하위 객체를 빨리 개발 가능
  - 반복된 코드의 중복을 줄임
  - 유지 보수의 편리성 제공
  - 객체의 다형성 구현



# 1절. 객체 지향 프로그래밍

## ❖ 객체 지향 프로그래밍의 특징

- 다형성 (Polymorphism)
  - 같은 타입이지만 실행 결과가 다양한 객체를 대입할 수 있는 성질
    - 부모 타입에는 모든 자식 객체가 대입
    - 인터페이스 타입에는 모든 구현 객체가 대입
  - 효과
    - 객체를 부품화시키는 것 가능
    - 유지보수 용이



# 2절. 객체와 클래스

## ❖ 객체(Object)와 클래스(Class)

- 현실세계: 설계도 → 객체
- 자바: 클래스 → 객체
- 클래스에는 객체를 생성하기 위한 필드와 메소드가 정의
- 클래스로부터 만들어진 객체를 해당 클래스의 인스턴스(instance)
- 하나의 클래스로부터 여러 개의 인스턴스를 만들 수 있음

[객체를 생성하는 순서]



# 3절. 클래스 선언

## ❖ 클래스의 이름

- 자바 식별자 작성 규칙에 따라야

| 번호 | 작성 규칙 | 예 |
|---|---|---|
| 1 | 하나 이상의 문자로 이루어져야 한다. | Car, SportsCar |
| 2 | 첫 번째 글자는 숫자가 올 수 없다. | Car, 3Car(x) |
| 3 | '$', '_' 외의 특수 문자는 사용할 수 없다. | $Car, _Car, @Car(x), #Car(x) |
| 4 | 자바 키워드는 사용할 수 없다. | int(x), for(x) |

- 한글 이름도 가능하나, 영어 이름으로 작성
- 알파벳 대소문자는 서로 다른 문자로 인식
- 첫 글자와 연결된 다른 단어의 첫 글자는 대문자로 작성하는 것이 관례

Calculator, Car, Member, ChatClient, ChatServer, Web_Browser



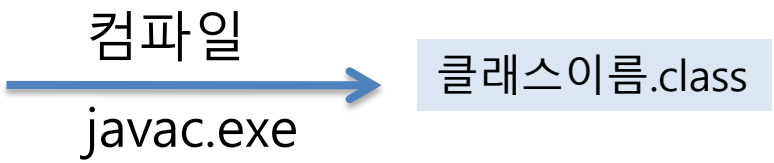
# 3절. 클래스 선언

## ❖ 클래스 선언과 컴파일

- 소스 파일 생성: 클래스이름.java (대소문자 주의)
- 소스 작성

```
public class 클래스이름 {


}
```




- 소스 파일당 하나의 클래스를 선언하는 것이 관례
  - 두 개 이상의 클래스도 선언 가능
  - 소스 파일 이름과 동일한 클래스만 public으로 선언 가능
  - 선언한 개수만큼 바이트 코드 파일이 생성

Car.java

```
public class Car {

}

class Tire {

}
```



# 4절. 객체 생성과 클래스 변수

## ❖ new 연산자

- 객체 생성 역할

```
new 클래스();
```






  - 클래스()는 생성자를 호출하는 코드
  - 생성된 객체는 힙 메모리 영역에 생성

- new 연산자는 객체를 생성 후, 객체 생성 번지 리턴

# 4절. 객체 생성과 클래스 변수

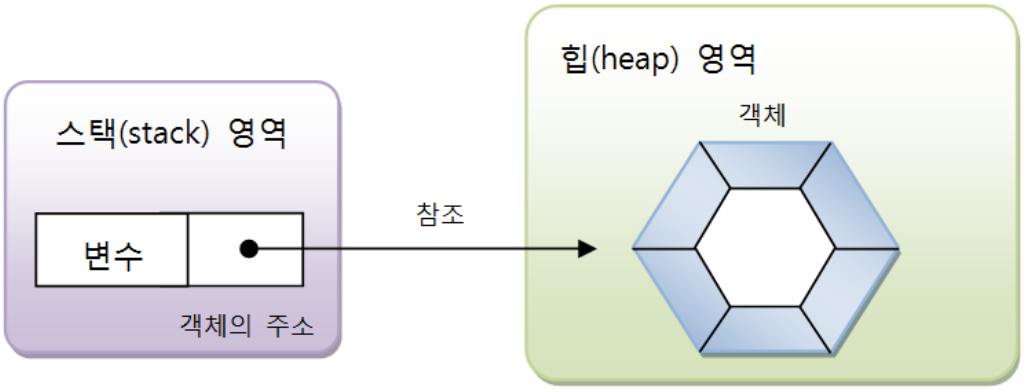
## ❖ 클래스 변수

- new 연산자에 의해 리턴 된 객체의 번지 저장 (참조 타입 변수)

- 힙 영역의 객체를 사용하기 위해 사용

```
클래스 변수;
변수 = new 클래스();

클래스 변수 = new 클래스();
```



# 4절. 객체 생성과 클래스 변수

## ❖ 클래스의 용도

- 라이브러리(API: Application Program Interface) 용
  - 자체적으로 실행되지 않음
  - 다른 클래스에서 이용할 목적으로 만든 클래스

- 실행용
  - main() 메소드를 가지고 있는 클래스로 실행할 목적으로 만든 클래스

1개의 애플리케이션 = (1개의 실행클래스) + (n개의 라이브러리 클래스)



# Student.java(클래스 선언) & StudentExample.java(클래스로부터 객체 생성)

```
1  public class Studer
2
3  }
4
```

```
1  public class StudentExample {
2      public static void main(String[] args) {
3          Student s1 = new Student();
4          System.out.println("s1 변수가 Student 객체를 참조합니다.");
5
6          Stud
7          Syste                                          조합니다.");
8      }
9  }
10
```

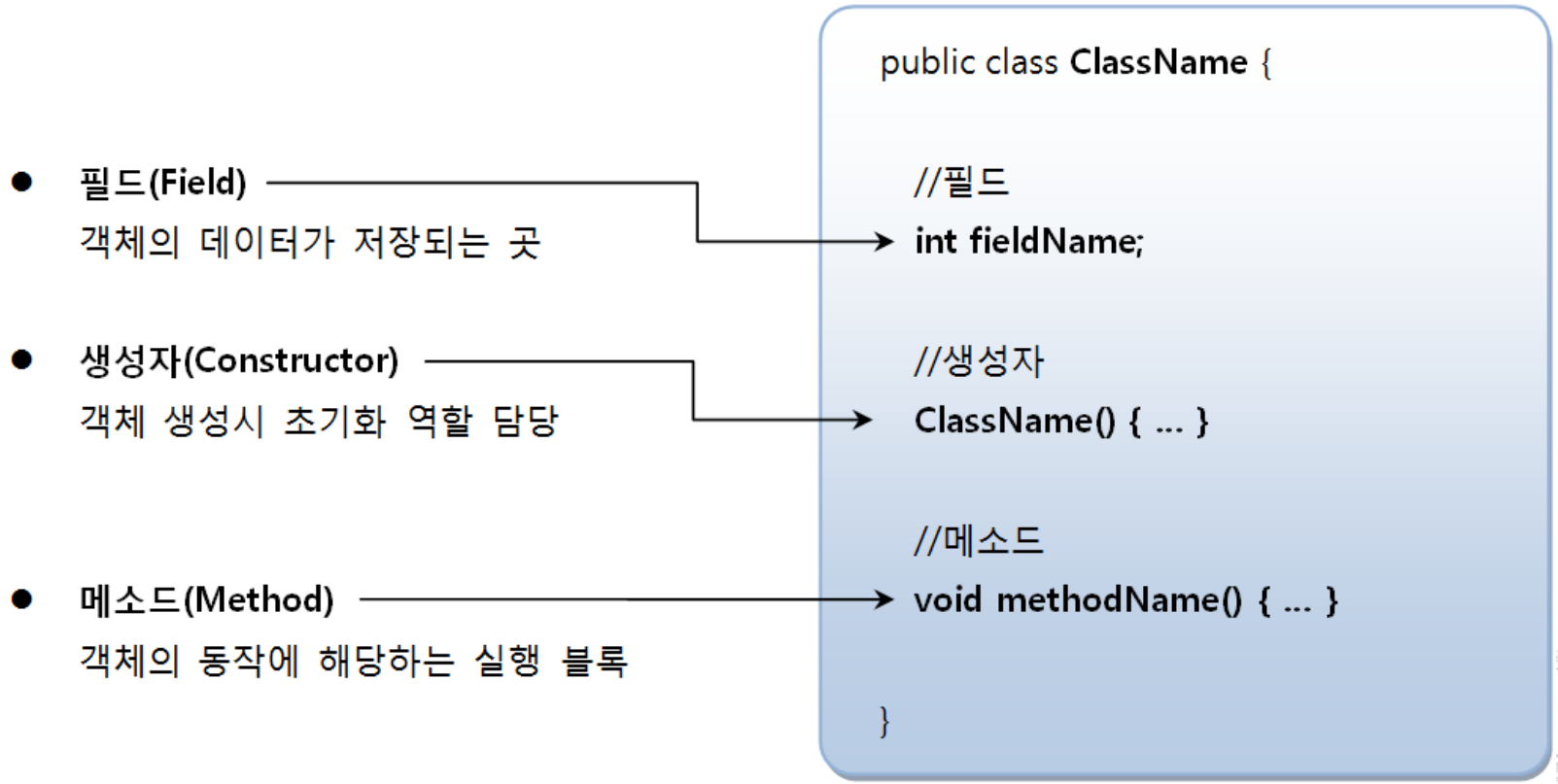
```
s1 변수가 Student 객체를 참조합니다.
s2 변수가 또 다른 Student 객체를 참조합니다.
계속하려면 아무 키나 누르십시오 . . .
```

활용클래스 : StudentExample.java_클래스로부터 객체 생성

# 5절. 클래스의 구성 멤버

## ❖ 클래스의 구성 멤버

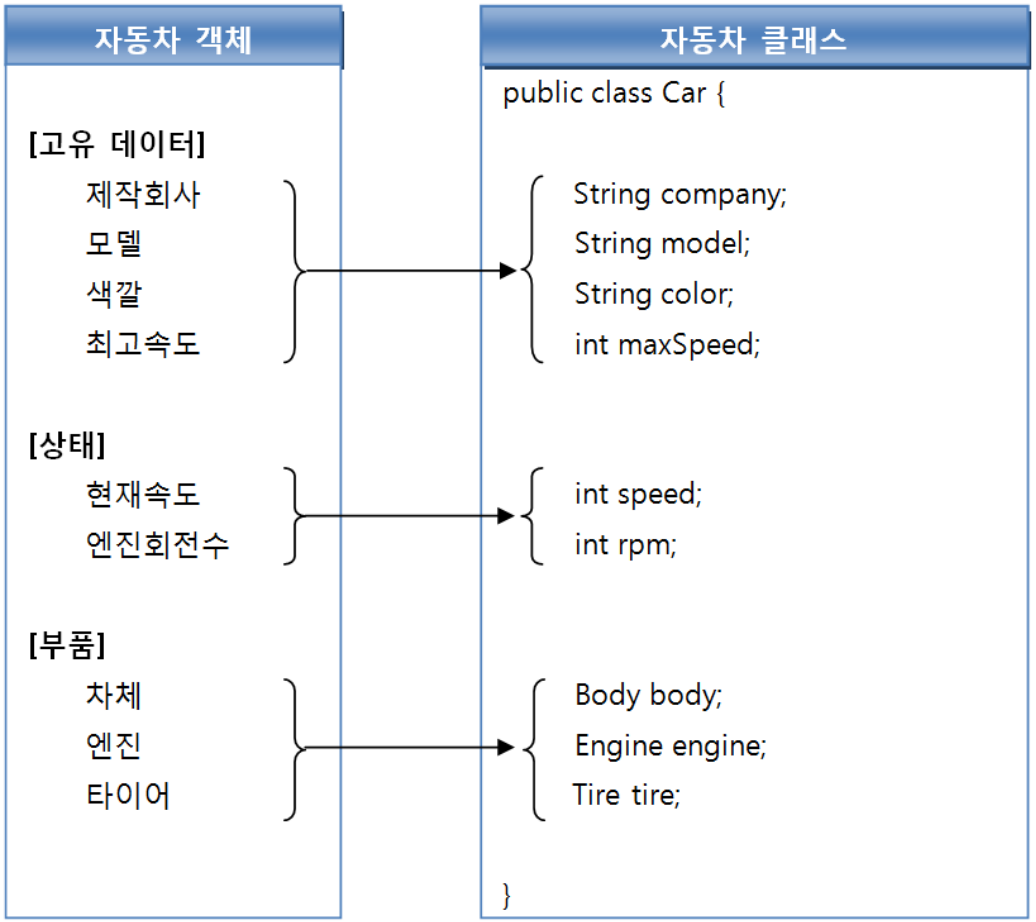
- 필드(Field)
- 생성자(Constructor)
- 메소드(Method)



# 6절. 필드(field)

## ❖ 필드의 내용

- 객체의 고유 데이터
- 객체가 가져야 할 부품 객체
- 객체의 현재 상태 데이터



# 6절. 필드(field)

## ❖ 필드 선언

```
타입 필드 [ = 초기값] ;
```

```
String company = "현대자동차";
String model = "그랜저";
int maxSpeed = 300;
int productionYear;
int currentSpeed;
boolean engineStart;
```



# 6절. 필드(field)

## ❖ 필드의 기본 초기값

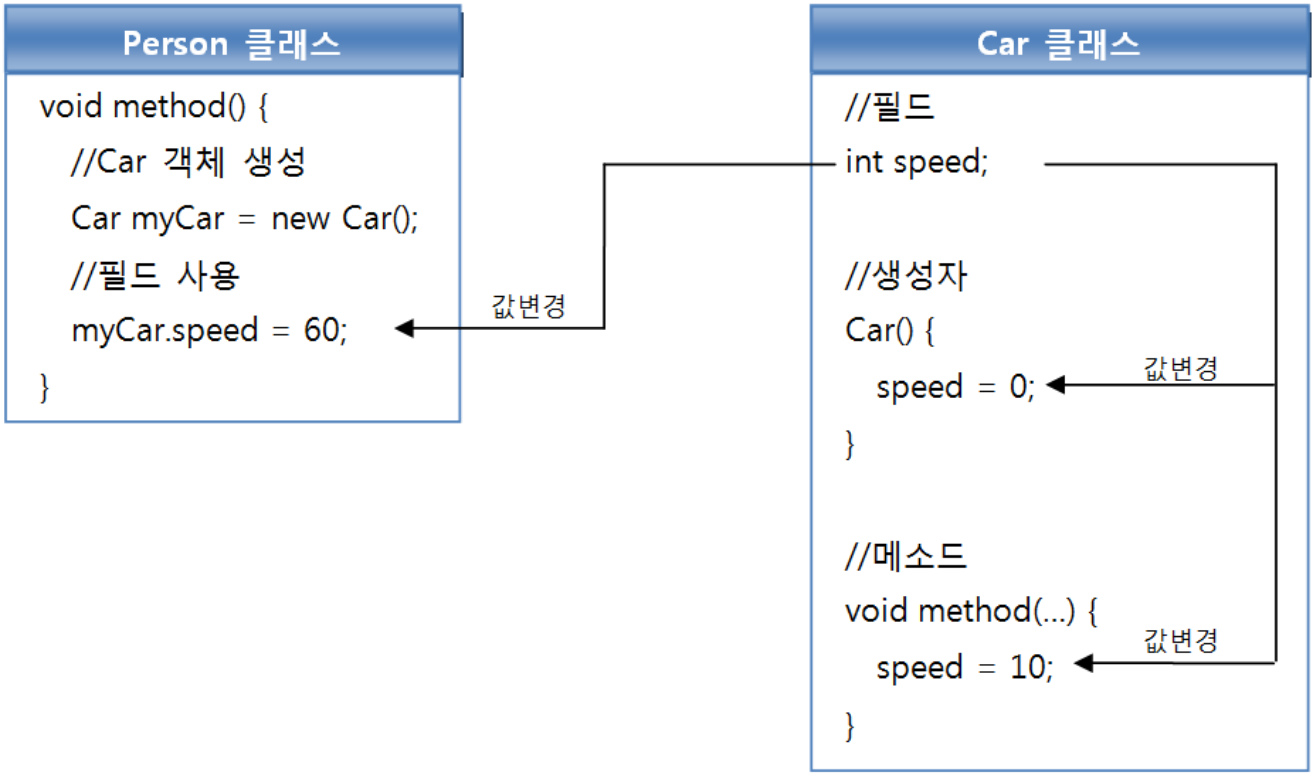
- 초기값 지정되지 않은 필드
  - 객체 생성시 자동으로 기본값으로 초기화

| 분류 | | 데이터 타입 | 초기값 |
|---|---|---|---|
| 기본 타입 | 정수 타입 | byte | 0 |
| | | char | ₩u0000 (빈 공백) |
| | | short | 0 |
| | | int | 0 |
| | | long | 0L |
| | 실수 타입 | float | 0.0F |
| | | double | 0.0 |
| | 논리 타입 | boolean | false |
| 참조 타입 | | 배열 열거 | null |
| | | 클래스(String 포함) | null |
| | | 인터페이스 | null |

# 6절. 필드(field)

## ❖ 필드 사용

- 필드 값을 읽고, 변경하는 작업을 말한다.
- 필드 사용 위치
  - 객체 내부: "필드이름" 으로 바로 접근
  - 객체 외부: "변수.필드이름" 으로 접근



# Car.java_Car 클래스 필드 선언

```java
public class Car {
    //필드
    String company = "현대자동차";
    String model = "그랜저";
    String color = "검정";
    int maxSpeed = 350;
    int speed;
}
```

```java
public class CarExample {
    public static void main(String[] args) {
        //객체 생성
        Car myCar = new Car();

        //필드 값 읽기
        System.out.println("제작회사: " + myCar.company);
        System.out.println("모델명: " + myCar.model);
        System.out.println("색깔: " + myCar.color);
        System.out.println("최고속도: " + myCar.maxSpeed);
        System.out.println("현재속도: " + myCar.speed);

        //필드 값 변경
        myCar.speed = 60;
        System.out.println("수정된 속도: " + myCar.speed);
    }
}
```

CarExample.java_외부 클래스에서 Car 필드값 읽기와 변경

# FieldInitValue.java_필드 자동 초기화

```java
public class FieldInitValue {
    //필드
    byte byteField;
    short shortField;
    int intField;
    long longField;

    boolean booleanField;
    char charField;

    float floatField;
    double doubleField;

    int[] arrField;
    String referenceField;
}
```

```java
public class FieldInitValueExample {
    public static void main(String[] args) {
        FieldInitValue fiv = new FieldInitValue();

        System.out.println("byteField: " + fiv.byteField);
        System.out.println("shortField: " + fiv.shortField);
        System.out.println("intField: " + fiv.intField);
        System.out.println("longField: " + fiv.longField);
        System.out.println("booleanField: " + fiv.booleanField);
        System.out.println("charField: " + fiv.charField);
        System.out.println("floatField: " + fiv.floatField);
        System.out.println("doubleField: " + fiv.doubleField);
        System.out.println("arrField: " + fiv.arrField);
        System.out.println("referenceField: " + fiv.referenceField);
    }
}
```

**FieldInitValueExample.java_필드값 출력**

# 7절. 생성자(Constructor)

## ❖ 생성자

- new 연산자에 의해 호출되어 객체의 초기화 담당

```
new 클래스();
```

  - 필드의 값 설정
  - 메소드 호출해  객체를 사용할 수 있도록 준비하는 역할 수행

## ❖ 기본 생성자(Default Constructor)

- 모든 클래스는 생성자가 반드시 존재하며 하나 이상 가질 수 있음
- 생성자 선언을 생략하면 컴파일러는 다음과 같은 기본 생성자 추가

```
[public] 클래스() { }
```

소스 파일(Car.java)

```
public class Car {


}
```

→

바이트 코드 파일(Car.class)

```
public class Car {
   public Car() { }   //자동 추가
}          기본 생성자
```

```
Car myCar = new Car();
                기본 생성자
```



# 7절. 생성자(Constructor)

## ❖ 생성자 선언

- 디폴트 생성자 대신 개발자가 직접 선언

```
클래스( 매개변수선언, ... ) {
    //객체의 초기화 코드
}
```

} 생성자 블록

- 개발자 선언한 생성자 존재 시 컴파일러는 기본 생성자 추가하지 않음
  - new 연산자로 객체 생성시 개발자가 선언한 생성자 반드시 사용

```
public class Car {
  //생성자
  Car(String model, String color, int maxSpeed) { ... }
}
```

```
Car myCar = new Car("그랜저", "검정", 300);
```



# Car.java_생성자 선언

```
1 public class Car {
2     //생성자
3     Car(String color, int cc) {
4     }
5 }
```

```
1 public class CarExample {
2     public static void main(String[] args) {
3         Car myCar = new Car("검정", 3000);
4         //Car myCar = new Car();  (x)
5
6         System.out.println(myCar);
7     }
8 }
9
```

```
C:\Windows\system32\cmd.exe
Car@15db9742
```

CarExample.java_생성자를 호출해서 객체 생성

# 7절. 생성자(Constructor)

## ❖ 필드 초기화

- 초기값 없이 선언된 필드는 객체가 생성될 때 기본값으로 자동 설정

- 다른 값으로 필드 초기화하는 방법
  - 필드 선언할 때 초기값 설정
  - 생성자의 매개값으로 초기값 설정

```
Korean k1 = new Korean("박자바", "011225-1234567");
Korean k2 = new Korean("김자바", "930525-0654321");
```

  - 매개 변수와 필드명 같은 경우 this 사용 (p.206~208)



# Korean.java_생성자에서 필드 초기화

Korean.java | KoreanExample.java

```
1  public class Korean {
2      //필드
3      String nation = "대한민국";
4      String name;
5      String ssn;
6
7      //생성자
8      /*public Korean(String n, String s) {
9          name = n;
10         ssn = s;
11     }*/
12
13     public Korean(String name, String ssn) {
14         this.name = name;
15         this.ssn = ssn;
16     }
17 }
```

```
C:₩Windows₩system32₩cmd.exe
k1.name : 박자바
k1.ssn : 011225-1234567
k2.name : 김자바
k2.ssn : 930525-0654321
계속하려면 아무 키나 누르십
```

Korean.java | KoreanExample.java

```
1  public class KoreanExample {
2      public static void main(String[] args) {
3          Korean k1 = new Korean("박자바", "011225-1234567");
4          System.out.println("k1.name : " + k1.name);
5          System.out.println("k1.ssn : " + k1.ssn);
6
7          Korean k2 = new Korean("김자바", "930525-0654321");
8          System.out.println("k2.name : " + k2.name);
9          System.out.println("k2.ssn : " + k2.ssn);
10     }
11 }
12
```

KoreanExample.java_객체 생성 후 필드값 출력

# 7절. 생성자(Constructor)

## ❖ 생성자 다양화해야 하는 이유

- 객체 생성할 때 외부 값으로 객체를 초기화할 필요
- 외부 값이 어떤 타입으로 몇 개가 제공될 지 모름 – 생성자도 다양화

## ❖ 생성자 오버로딩(Overloading) (p.208~211)

- 매개변수의 타입, 개수, 순서가 다른 생성자 여러 개 선언

```
public class 클래스 {
    클래스 ( [타입 매개변수, ...] ) {
        ...
    }
    클래스 ( [타입 매개변수, ...] ) {
        ...
    }
}
```

**[생성자의 오버로딩]**
매개변수의 타입, 개수, 순서가 다르게 선언

```
public class Car {
    Car() { ... }
    Car(String model) { ... }
    Car(String model, String color) { ... }
    Car(String model, String color, int maxSpeed) { ... }
}
```

```
Car(String model, String color) { ... }
Car(String color, String model) { ... } //오버로딩이 아님
```

```
Car car1 = new Car();
Car car2 = new Car("그랜저");
Car car3 = new Car("그랜저", "흰색");
Car car4 = new Car("그랜저", "흰색", 300);
```

# Car.java_생성자의 오버로딩

Car.java

```
1  public class Car {
2      //필드
3      String company = "현대자동차";
4      String model;
5      String color;
6      int maxSpeed;
7
8      //생성자
9      Car() {
10     }
11
12     Car(String model) {
13         this.model = model;
14     }
15
16     Car(String model, String color) {
17         this.model = model;
18         this.color = color;
19     }
20
21     Car(String model, String color, int maxSpeed) {
22         this.model = model;
23         this.color = color;
24         this.maxSpeed = maxSpeed;
25     }
26 }
```

```
car1.company : 현대자동차

car2.company : 현대자동차
car2.model : 자가용

car3.company : 현대자동차
car3.model : 자가용
car3.color : 빨강

car4.company : 현대자동차
car4.model : 택시
car4.color : 검정
car4.maxSpeed : 200
계속하려면 아무 키나 누르십시
```

Car.java | CarExample.java

```
1  public class CarExample {
2      public static void main(String[] args) {
3          Car car1 = new Car();
4          System.out.println("car1.company : " + car1.company);
5          System.out.println();
6
7          Car car2 = new Car("자가용");
8          System.out.println("car2.company : " + car2.company);
9          System.out.println("car2.model : " + car2.model);
10         System.out.println();
11
12         Car car3 = new Car("자가용", "빨강");
13         System.out.println("car3.company : " + car3.company);
14         System.out.println("car3.model : " + car3.model);
15         System.out.println("car3.color : " + car3.color);
16         System.out.println();
17
18         Car car4 = new Car("택시", "검정", 200);
19         System.out.println("car4.company : " + car4.company);
20         System.out.println("car4.model : " + car4.model);
21         System.out.println("car4.color : " + car4.color);
22         System.out.println("car4.maxSpeed : " + car4.maxSpeed);
23     }
24 }
```

## CarExample.java_객체 생성 시 생성자 선택

# 7절. 생성자(Constructor)

## ❖ 다른 생성자 호출( this() )

- 생성자 오버로딩되면 생성자 간의 중복된 코드 발생
- 초기화 내용이 비슷한 생성자들에서 이러한 현상을 많이 볼 수 있음
  - 초기화 내용을 한 생성자에 몰아 작성
  - 다른 생성자는 초기화 내용을 작성한 생성자를 this(…)로 호출

```
Car(String model) {
   this.model = model;
   this.color = "은색";
   this.maxSpeed = 250;
}

Car(String model, String color) {
   this.model = model;
   this.color = color;
   this.maxSpeed = 250;
}

Car(String model, String color, int maxSpeed) {
   this.model = model;
   this.color = color;
   this.maxSpeed = maxSpeed;
}
```

중복 코드

중복 코드

중복 코드



## Car.java_다른 생성자를 호출해서 중복 코드 줄이기

Car.java

```
public class Car {
    //필드
    String company = "현대자동차";
    String model;
    String color;
    int maxSpeed;

    //생성자
    Car() {
    }

    Car(String model) {
        this(model, null, 0);
    }

    Car(String model, String color) {
        this(model, color, 0);
    }

    Car(String model, String color, int maxSpeed) {
        this.model = model;
        this.color = color;
        this.maxSpeed = maxSpeed;
    }
}
```

CarExample.java

```
public class CarExample {
    public static void main(String[] args) {
        Car car1 = new Car();
        System.out.println("car1.company : " + car1.company);
        System.out.println();

        Car car2 = new Car("자가용");
        System.out.println("car2.company : " + car2.company);
        System.out.println("car2.model : " + car2.model);
        System.out.println();

        Car car3 = new Car("자가용", "빨강");
        System.out.println("car3.company : " + car3.company);
        System.out.println("car3.model : " + car3.model);
        System.out.println("car3.color : " + car3.color);
        System.out.println();

        Car car4 = new Car("택시", "검정", 200);
        System.out.println("car4.company : " + car4.company);
        System.out.println("car4.model : " + car4.model);
        System.out.println("car4.color : " + car4.color);
        System.out.println("car4.maxSpeed : " + car4.maxSpeed);
    }
}
```

C:₩Windows₩system32₩cmd.exe

```
car1.company : 현대자동차

car2.company : 현대자동차
car2.model : 자가용

car3.company : 현대자동차
car3.model : 자가용
car3.color : 빨강

car4.company : 현대자동차
car4.model : 택시
car4.color : 검정
car4.maxSpeed : 200
계속하려면 아무 키나 누르
```

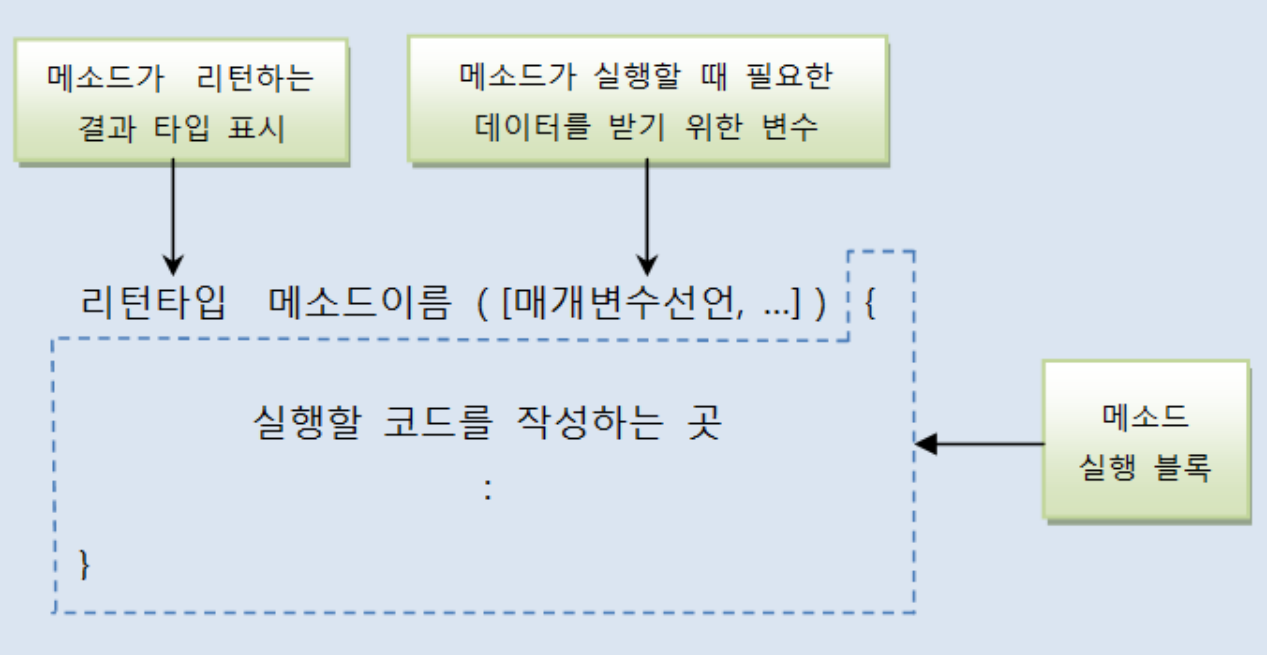
## CarExample.java_객체 생성시 생성자 선택

# 8절. 메소드(method)

## ❖ 메소드란?

- 객체의 동작(기능)
- 호출해서 실행할 수 있는 중괄호 { } 블록
- 메소드 호출하면 중괄호 { } 블록에 있는 모든 코드들이 일괄 실행

## ❖ 메소드 선언



# Calculator.java_메소드 선언

Calculator.java

```
public class Calculator {
    //메소드
    void powerOn() {
        System.out.println("전원을 켭니다.");
    }

    int plus(int x, int y) {
        int result = x + y;
        return result;
    }

    double divide(int x, int y) {
        double result = (double)x / (double)y;
        return result;
    }

    void powerOff() {
        System.out.println("전원을 끕니다");
    }
}
```

```
C:₩Windows₩system
전원을 켭니다.
result1: 11
result2: 2.5
전원을 끕니다
계속하려면 아무
```

CalculatorExample.java

```
public class CalculatorExample {
    public static void main(String[] args) {
        Calculator myCalc = new Calculator();
        myCalc.powerOn();

        int result1 = myCalc.plus(5, 6);
        System.out.println("result1: " + result1);

        byte x = 10;
        byte y = 4;
        double result2 = myCalc.divide(x, y);
        System.out.println("result2: " + result2);

        myCalc.powerOff();
    }
}
```

작은 타입에서 큰 타입으로는 자동형변환

# CalculatorExample.java_메소드 호출

# 8절. 메소드(method)

## ❖ 메소드 리턴 타입

- 메소드 실행된 후 리턴하는 값의 타입

- 메소드는 리턴값이 있을 수도 있고 없을 수도 있음

[메소드 선언]

```
void powerOn() { ... }
double divide(int x, int y) { ... }
```

[메소드 호출]

```
powerOn();
double result = divide( 10, 20 );
```

## ❖ 메소드 이름

- 자바 식별자 규칙에 맞게 작성



# 8절. 메소드(method)

## ❖ 메소드 매개변수 선언

- 매개변수는 메소드를 실행할 때 필요한 데이터를 외부에서 받기 위해 사용
- 매개변수도 필요 없을 수 있음

[메소드 선언]

```
void powerOn() { ... }
double divide(int x, int y) { ... }
```

[메소드 호출]

```
powerOn();
double result = divide( 10, 20 );
```

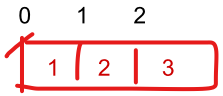
```
byte b1 = 10;
byte b2 = 20;
double result = divide(b1, b2);
```



# Computer.java_매개 변수의 수를 모를 경우

Computer.java

```
1  public class Computer {
2      int sum1(int[] values) {
3          int sum = 0;
4          for(int i=0; i<values.length; i++) {
5              sum += values[i];
6          }
7          return sum;
8      }
9
10     int sum2(int ... values) {
11         int sum = 0;
12         for(int i=0; i<values.length; i++) {
13             sum += values[i];
14         }
15         return sum;
16     }
17 }
18
```




C:₩Windows₩s

```
result1: 6
result2: 15
result3: 6
result4: 15
계속하려면 아
```

ComputerExample.java

```
1  public class ComputerExample {
2      public static void main(String[] args) {
3          Computer myCom = new Computer();
4
5          int[] values1 = {1, 2, 3};
6          int result1 = myCom.sum1(values1);
7          System.out.println("result1: " + result1);
8
9          int result2 = myCom.sum1(new int[] {1, 2, 3, 4, 5});
10         System.out.println("result2: " + result2);
11
12         int result3 = myCom.sum2(1, 2, 3);
           System.out.println("result3: " + result3);

           int result4 = myCom.sum2(1, 2, 3, 4, 5);
           System.out.println("result4: " + result4);
       }
19 ¤
```

ComputerExample.java_ 매개 변수의 수를 모를 경우

# 8절. 메소드(method)

## ❖ 리턴(return) 문 (p.221~224)

- 메소드 실행을 중지하고 리턴값 지정하는 역할

- 리턴값이 있는 메소드
  - 반드시 리턴(return)문 사용해 리턴값 지정해야

```
int plus(int x, int y) {
    int result = x + y;
    return result;
}
```

```
boolean isLeftGas() {
    if(gas==0) {
        System.out.println("gas 가  없습니다.");
        return false;
    }
    System.out.println("gas 가  있습니다.");
    return true;
}
```

  - return 문 뒤에 실행문 올 수 없음
- 리턴값이 없는 메소드
  - 메소드 실행을 강제 종료 시키는 역할

# Car.java_return문

Car.java | CarExample.java

```
public class Car {
    //필드
    int gas;

    //생성자

    //메소드
    void setGas(int gas) {
        this.gas = gas;
    }

    boolean isLeftGas() {
        if(gas==0) {
            System.out.println("gas가 없습니다.");
            return false;
        }
        System.out.println("gas가 있습니다.");
        return true;
    }

    void run() {
        while(true) {
            if(gas > 0) {
                System.out.println("달립니다.(gas잔량:" + gas + ")");
                gas -= 1;
            } else {
                System.out.println("멈춥니다.(gas잔량:" + gas + ")");
                return;
            }
        }
    }
}
```

set 개수는 필드개수의 범위와 같다

필드값을 변경. 기본 세팅값인듯?

Car.java | CarExample.java

```
public class CarExample {
    public static void main(String[] args) {
        Car myCar = new Car();

        myCar.setGas(5);  //Car의 setGas() 메소드 호출

        boolean gasState = myCar.isLeftGas();  //Car의 isLeftGas() 메소드 호출
        if(gasState) {
            System.out.println("출발합니다.");
            myCar.run();  //Car의 run() 메소드 호출
        }

        if(myCar.isLeftGas()) {  //Car의 isLeftGas() 메소드 호출
            System.out.println("gas를 주입할 필요가 없습니다.");
        } else {
            System.out.println("gas를 주입하세요.");
        }
    }
}
```

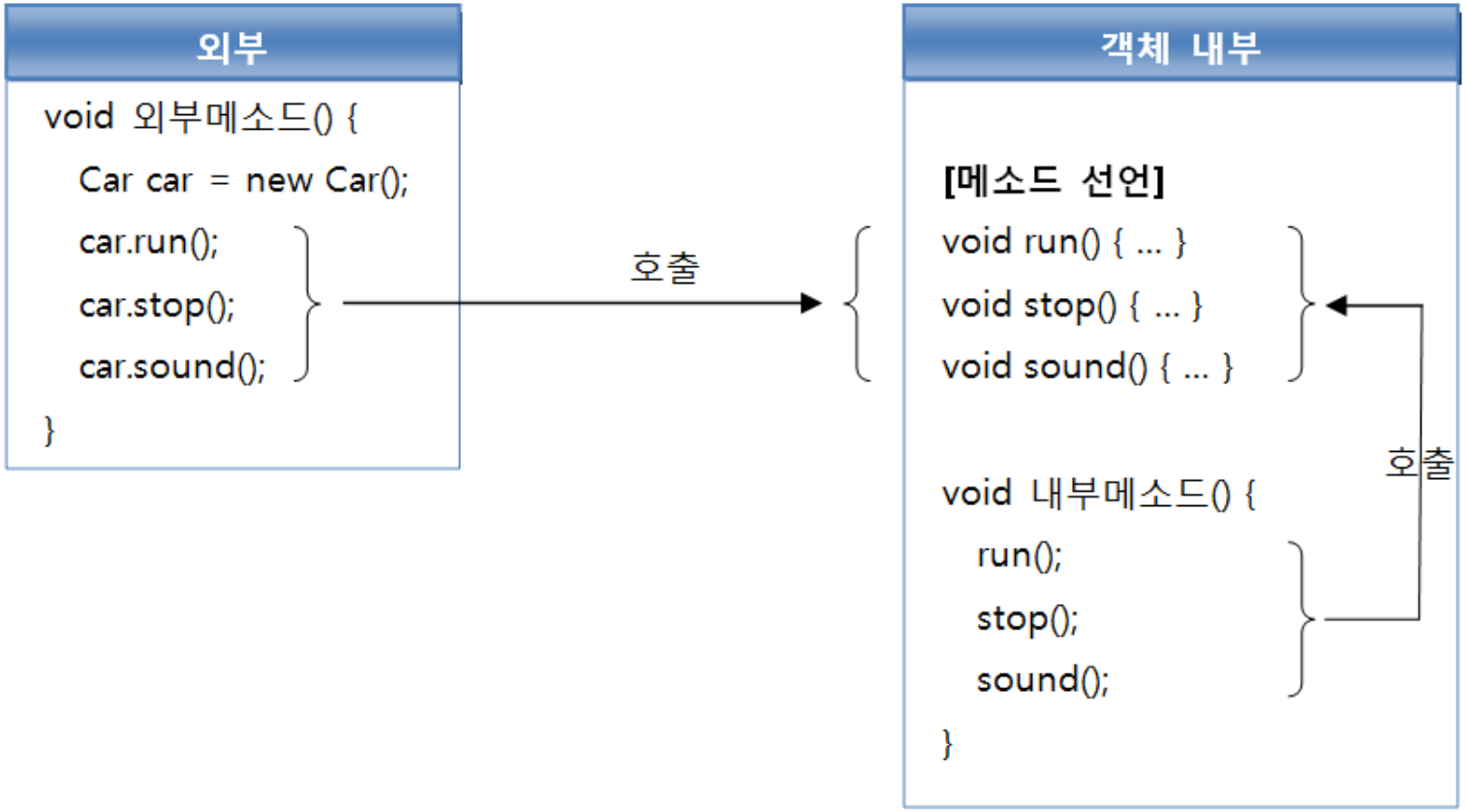
# CarExample.java_ return문

```
C:₩Windows₩system32₩cmd
gas가 있습니다.
출발합니다.
달립니다.(gas잔량:5)
달립니다.(gas잔량:4)
달립니다.(gas잔량:3)
달립니다.(gas잔량:2)
달립니다.(gas잔량:1)
멈춥니다.(gas잔량:0)
gas가 없습니다.
gas를 주입하세요.
```

# 8절. 메소드(method)

## ❖ 메소드 호출

- 메소드는 클래스 내·외부의 호출에 의해 실행
  - 클래스 내부: 메소드 이름으로 호출 (p.225~228)
  - 클래스 외부: 객체 생성 후, 참조 변수를 이용해 호출 (p.228~229)



# Calculator.java_클래스 내부에서 메소드 호출

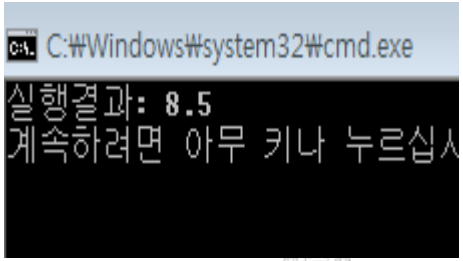
Calculator.java | CalculatorExample

```
1  public class Calculator {
2      //필드
3      //생성자
4      //메소드
5      int plus(int x, int y) {
6          int result = x + y;
7          return result;
8      }
9
10     double avg(int x, int y) {
11         double sum = plus(x, y);
12         double result = sum / 2;
13         return result;
14     }
15
16     void execute() {
17         double result = avg(7, 10);
18         println("실행결과: " + result);
19     }
20
21     void println(String message) {
22         System.out.println(message);
23     }
24 }
```

Calculator.java | CalculatorExample

```
1  public class CalculatorExample {
2      public static void main(String[] args) {
3          Calculator myCalc = new Calculator();
4          myCalc.execute();
5      }
6  }
7
8
```

**CalculatorExample.java_calculator의 execute()실행**



## Car.java_클래스 외부에서 메소드 호출

```java

public class Car {
    //필드
    int speed;

    //생성자

    //메소드
    int getSpeed() {
        return speed;
    }

    void keyTurnOn() {
        System.out.println("키를 돌립니다.");
    }

    void run() {
        for(int i=10; i<=50; i+=10) {
            speed = i;
            System.out.println("달립니다.(시속:" + speed + "km/h)");
        }
    }
}

```

내부

```java
public class CarExample {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.keyTurnOn();
        myCar.run();
        int speed = myCar.getSpeed();
        System.out.println("현재 속도: " + speed + "km/h");
    }
}

```

외부

CarExample.java_클래스 외부에서 메소드 호출

# 8절. 메소드(method)

## ❖ 메소드 오버로딩(Overloading)

- 클래스 내에 같은 이름의 메소드를 여러 개 선언하는 것
- 하나의 메소드 이름으로 다양한 매개값 받기 위해 메소드 오버로딩
- 오버로딩의 조건: 매개변수의 타입, 개수, 순서가 달라야

```
class 클래스 {
   리턴타입  메소드이름 ( 타입 변수, ... )  { ... }
}
```

무관 / 동일 / 매개변수의 타입, 개수, 순서가 달라야함

```
   리턴타입  메소드이름 ( 타입 변수, ... )  { ... }
```

a  b 임시변수명만 바꿔도 오버로딩x

```
int plus(int x, int y) {
   int result = x + y;
   return result;
}
```

plus(10, 20);

```
double plus(double x, double y) {
   double result = x + y;
   return result;
}
```

plus(10.5, 20.3);

```
void println() { .. }
void println(boolean x) { .. }
void println(char x) { .. }
void println(char[] x) { .. }
void println(double x) { .. }
void println(float x) { .. }
void println(int x) { .. }
void println(long x) { .. }
void println(Object x) { .. }
void println(String x) { .. }
```

```
int x = 10;
double y = 20.3;
plus(x, y);
```

?

```
int divide(int x, int y) { ... }
double divide(int boonja, int boonmo) { ... }
```



## Calculator.java_메소드 오버로딩

```java
public class Calculator {
    //필드
    //생성자
    //메소드
    int plus(int x, int y) {
        int result = x + y;
        return result;
    }

    double avg(int x, int y) {
        double sum = plus(x, y);
        double result = sum / 2;
        return result;
    }

    void execute() {
        double result = avg(7, 10);
        println("실행결과: " + result);
    }

    void println(String message) {
        System.out.println(message);
    }
}
```

```java
public class CalculatorExample {
    public static void main(String[] args) {
        Calculator myCalc = new Calculator();
        myCalc.execute();
    }
}
```

CalculatorExample.java_메소드 오버로딩

# 9절. 인스턴스 멤버와 this

## ❖ 인스턴스 멤버란?

- 객체(인스턴스) 마다 가지고 있는 필드와 메소드
  - 이들을 각각 인스턴스 필드, 인스턴스 메소드라고 부름

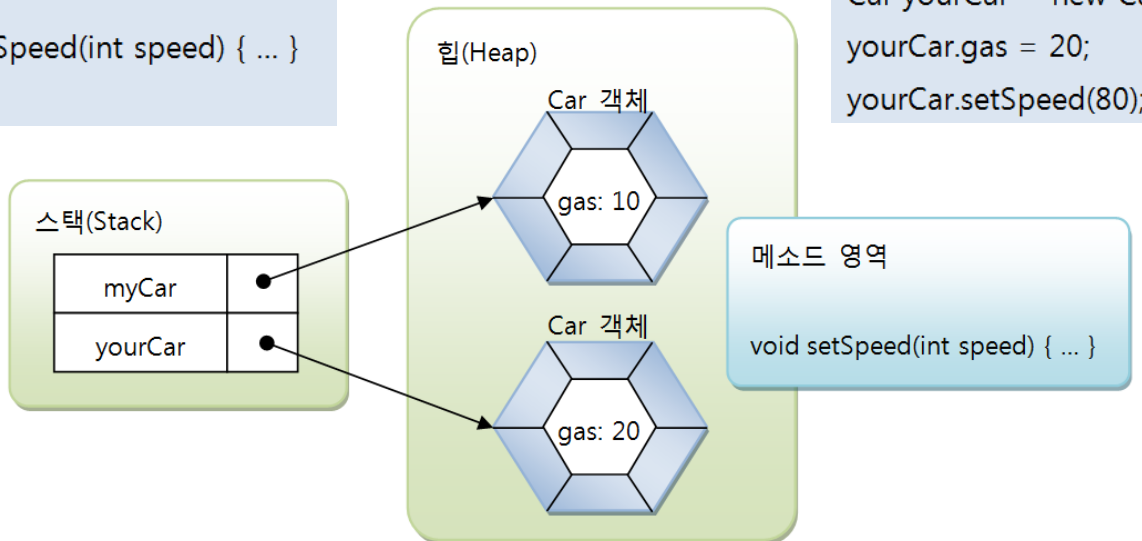
- 인스턴스 멤버는 객체 소속된 멤버이기 때문에 객체가 없이 사용불가

```
public class Car {
   //필드
   int gas;

   //메소드
   void setSpeed(int speed) { ... }
}
```

```
Car myCar = new Car();
myCar.gas = 10;
myCar.setSpeed(60);

Car yourCar = new Car();
yourCar.gas = 20;
yourCar.setSpeed(80);
```



# 9절. 인스턴스 멤버와 this

## ❖ this

- 객체(인스턴스) 자신의 참조(번지)를 가지고 있는 키워드
- 객체 내부에서 인스턴스 멤버임을 명확히 하기 위해 this. 사용
- 매개변수와 필드명이 동일할 때 인스턴스 필드임을 명확히 하기 위해 사용

```
Car(String model) {
   this.model = model;
}
void setModel(String model) {
   this.model = model;
}
```



## Car.java_인스턴스 멤버와 this

```java
public class Car {
    //필드
    String model;
    int speed;

    //생성자
    Car(String model) {
        this.model = model;
    }

    //메소드
    void setSpeed(int speed) {
        this.speed = speed;
    }

    void run() {
        for(int i=10; i<=50; i+=10) {
            this.setSpeed(i);
            System.out.println(this.model + "가 달립니다.(시속:" + this.speed + "km/h)");
        }
    }
}
```

```java
public class CarExample {
    public static void main(String[] args) {
        Car myCar = new Car("포르쉐");
        Car yourCar = new Car("벤츠");

        myCar.run();
        yourCar.run();
    }
}
```

인스턴스 멤버들은 생성자를 통해 객체를 생성해서 실행

인스턴스 멤버들만 this사용

CarExample.java_인스턴스 멤버와 this

# 10절. 정적 멤버와 static

## ❖ 정적(static) 멤버란?

- 클래스에 고정된 필드와 메소드 – 정적 필드, 정적 메소드

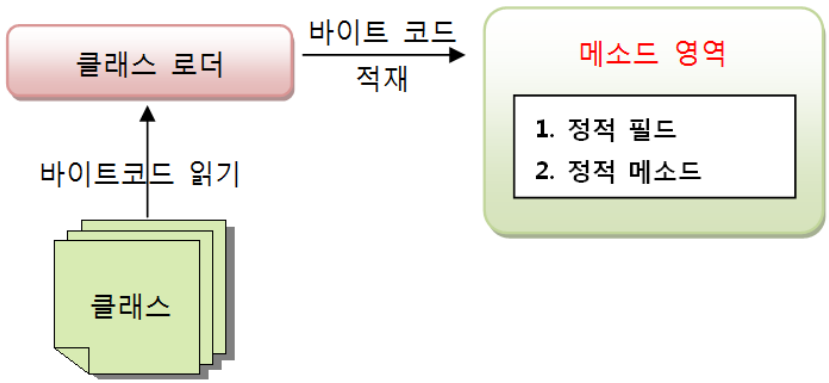
- 정적 멤버는 클래스에 소속된 멤버
  - 객체 내부에 존재하지 않고, 메소드 영역에 존재
  - 정적 멤버는 객체를 생성하지 않고 클래스로 바로 접근해 사용

## ❖ 정적 멤버 선언

- 필드 또는 메소드 선언할 때 **static** 키워드 붙임

```
public class 클래스 {
  //정적 필드
  static 타입 필드 [= 초기값];

  //정적 메소드
  static 리턴타입 메소드( 매개변수선언, ... ) { ... }
}
```



# 10절. 정적 멤버와 static

## ❖ 정적 멤버 사용

- 클래스 이름과 함께 도트(.) 연산자로 접근

```
클래스.필드;
클래스.메소드( 매개값, ... );
```

```
public class Calculator {
    static double pi = 3.14159;
    static int plus(int x, int y) { ... }
    static int minus(int x, int y) { ... }
}
```

[바람직한 사용]

```
double result1 = 10 * 10 * Calculator.pi;
int result2 = Calculator.plus(10, 5);
int result3 = Calculator.minus(10, 5);
```

**[바람직하지 못한 사용]**

```
Calculator myCalcu = new Calculator();
double result1 = 10 * 10 * myCalcu.pi;
int result2 = myCalcu.plus(10, 5);
int result3 = myCalcu.minus(10, 5);
```

# 10절. 정적 멤버와 static

## ❖ 인스턴스 멤버 선언 vs 정적 멤버 선언의 기준

- **필드**
  - **객체 마다 가지고 있어야 할 데이터 ➔ 인스턴스 필드**
  - **공용적인 데이터 ➔ 정적 필드**

```
public class Calculator {
   String color;                    //계산기 별로 색깔이 다를 수 있다.
   static double pi = 3.14159;   //계산기에서 사용하는 파이( π)값은 동일하다.
}
```

- **메소드**

스태택 클래스 자체에 소속된 변수

인스턴스 스태틱 필드에서 힙영역을 참조할 순 없다.

  - **인스턴스 필드로 작업해야 할 메소드 ➔ 인스턴스 메소드**
  - **인스턴스 필드로 작업하지 않는 메소드 ➔ 정적 메소드**

```
public Calculator {
   String color;
   void setColor(String color) { this.color = color; }
   static int plus(int x, int y) { return x + y; }
   static int minus(int x, int y) { return x – y; }
}
```

생성 시점: 클래스가 처음 사용될 때 (로딩 시) 한 번만 생성됩니다. new를 한 번도 안 해도 존재합니다.
소멸 시점: 클래스가 언로드되거나 JVM이 종료될 때 사라집니다.
개수: 객체를 몇 개 만들든 딱 하나이고, 모든 객체가 그걸 공유

## Calculator.java_정적 멤버 사용

```java
public class Calculator {
    static double pi = 3.14159;

    static int plus(int x, int y) {
        return x + y;
    }

    static int minus(int x, int y) {
        return x - y;
    }
}
```

new 생성자 형성 X

객체를 100개 만들어도 pi라는 저장 공간은 메모리에 1개만 있고, 100개 객체가 모두 그 하나를 같이 봅니다.

생성자면 필드데이터로 따로 자지고 볶나?

```java
s CalculatorExample {
static void main(String[] args) {
ble result1 = 10 * 10 * Calculator.pi;
int result2 = Calculator.plus(10, 5);
int result3 = Calculator.minus(10, 5);

System.out.println("result1 : " + result1);
System.out.println("result2 : " + result2);
System.out.println("result3 : " + result3);
    }
}
```

CalculatorExample.java_정적 멤버 사용

# 10절. 정적 멤버와 static

## ❖ 정적 초기화 블록

- 클래스가 메소드 영역으로 로딩될 때 자동으로 실행하는 블록

```
static {
    ...
}
```

- 정적 필드의 복잡한 초기화 작업과 정적 메소드 호출 가능
- 클래스 내부에 여러 개가 선언되면 선언된 순서대로 실행

```
public class Television {
    static String company = "Samsung";
    static String model = "LCD";
    static String info;

    static {
        info = company + "-" + model;
    }
}
```



## Television.java_정적 초기화 블록

```
public class Television {
   static String company = "Samsung";
   static String model = "LCD";
   static String info;

   static {
      info = company + "-" + model;
   }
}
```

```
1 public class TelevisionExample {
2     public static void main(String[] args) {
3         System.out.println(Television.info);
4     }
5 }
```

TelevisionExample.java_정적 초기화 블록

# 10절. 정적 멤버와 static

## ❖ 정적 메소드와 정적 블록 작성시 주의할 점

- 객체가 없어도 실행 가능
- 블록 내부에 인스턴스 필드나 인스턴스 메소드 사용 불가
- 객체 자신의 참조인 this 사용 불가
  - EX) main()

```
//인스턴스 필드와 메소드
int field1;
void method1() { ... }

//정적 필드와 메소드
static int field2;
static void method2() { ... }
```

```
//정적 블록
static {
    field1 = 10;        (x)
    method1();          (x)
    field2 = 10;        (o)
    method2();          (o)
}

//정적 메소드
static void Method3 {
    this.field1 = 10;       (x)
    this.method1();         (x)
    field2 = 10;            (o)
    method2();              (o)
}
```

컴파일 에러

컴파일 에러

```
static void Method3() {
    ClassName obj = new ClassName();
    obj.field1 = 10;
    obj.method1();
}
```



## 정적 메소드와 블록 선언 시 주의점

```
1  public class Car {
2      int speed;
3
4      void run() {
5          System.out.println(speed + "으로 달립니다.");
6      }
7
8      public static void main(String[] args) {
9          Car myCar = new Car();
10         myCar.speed = 60;
11         myCar.run();
12     }
13 }
```



# 10절. 정적 멤버와 static

## ❖ 싱글톤(Singleton)

- 하나의 애플리케이션 내에서 단 하나만 생성되는 객체

## ❖ 싱글톤을 만드는 방법

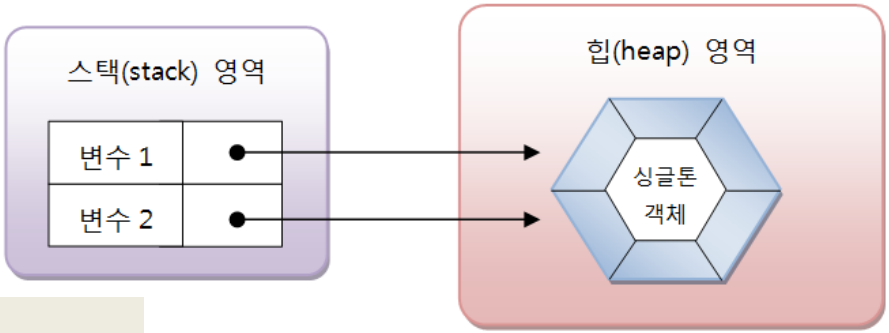
- 외부에서 new 연산자로 생성자를 호출할 수 없도록 막기
  - private 접근 제한자를 생성자 앞에 붙임
- 클래스 자신의 타입으로 정적 필드 선언
  - 자신의 객체를 생성해 초기화
  - private 접근 제한자 붙여 외부에서 필드 값 변경 불가하도록
- 외부에서 호출할 수 있는 정적 메소드인 getInstance() 선언
  - 정적 필드에서 참조하고 있는 자신의 객체 리턴



# 10절. 정적 멤버와 static

## ❖ 싱글톤 얻는 방법

```
클래스 변수 1 = 클래스.getInstance();
클래스 변수 2 = 클래스.getInstance();
```




```
/*
Singleton obj1 = new Singleton();  //컴파일 에러
Singleton obj2 = new Singleton();  //컴파일 에러
*/

Singleton obj1 = Singleton.getInstance();
Singleton obj2 = Singleton.getInstance();

if(obj1 == obj2) {
   System.out.println("같은 Singleton 객체 입니다.");
} else {
   System.out.println("다른 Singleton 객체 입니다.");
}
```

## Singleton.java 싱글톤

```java
public class Singleton {
    private static Singleton singleton = new Singleton();

    private Singleton() {}

    static Singleton getInstance() {
        return singleton;
    }
}
```

```java
public class SingletonExample {
    public static void main(String[] args) {
        /*
        Singleton obj1 = new Singleton();  //컴파일 에러
        Singleton obj2 = new Singleton();  //컴파일 에러
        */

        Singleton obj1 = Singleton.getInstance();
        Singleton obj2 = Singleton.getInstance();

        if(obj1 == obj2) {
            System.out.println("같은 Singleton 객체 입니다.");
        } else {
            System.out.println("다른 Singleton 객체 입니다.");
        }
    }
}
```

# 11절. final 필드와 상수(static final)

## ❖ final 필드

- 최종적인 값을 갖고 있는 필드 = 값을 변경할 수 없는 필드
- final 필드의 딱 한번의 초기값 지정 방법
  - 필드 선언 시
  - 생성자

```
public class Person {
    final String nation = "Korea";
    final String ssn;
    String name;

    public Person(String ssn, String name) {
        this.ssn = ssn;
        this.name = name;
    }
}
```



## Person.java final 필드 선언과 초기화

필드 변수형이 static /final 이런 느낌인가

```java
public class Person {
    final String nation = "Korea";
    final String ssn;
    String name;

    public Person(String ssn, String name) {
        this.ssn = ssn;
        this.name = name;
    }
}
```

개개인 사람별로 데이터를 수집할 때 유용함
사람 사물별 고유한 특징 정보

```java
public class PersonExample {
    public static void main(String[] args) {
        Person p1 = new Person("123456-1234567", "계백");

        System.out.println(p1.nation);
        System.out.println(p1.ssn);
        System.out.println(p1.name);

        //p1.nation = "usa";
        //p1.ssn = "654321-7654321";
        p1.name = "을지문덕";;
    }
}
```

PersonExample.java final 필드 테스트

# 11절. final 필드와 상수(static final)

## ❖ 상수(static final)

- 상수 = 정적 final 필드
  - final 필드:
    - 객체마다 가지는 불변의 인스턴스 필드
  - **상수(static final):** 이건 진짜 누구에게나 똑같이 적용되는 고유한 상수 불변?
    - 객체마다 가지고 있지 않음
    - 메소드 영역에 클래스 별 로 관리되는 불변의 정적 필드
    - 공용 데이터로서 사용
- 상수 이름은 전부 대문자로 작성
- 다른 단어가 결합되면 _ 로 연결



## Earth.java 상수 선언

```java
public class Earth {
    static final double EARTH_RADIUS = 6400;
    static final double EARTH_SURFACE_AREA;

    static {
        EARTH_SURFACE_AREA = 4 * Math.PI * EARTH_RADIUS * EARTH_RADIUS;
    }
}
```

대문자로

java.lang에 살아서 import안 해도 됨

```java
public class EarthExample {
    public static void main(String[] args) {
        System.out.println("지구의 반지름: " + Earth.EARTH_RADIUS + " km");
        System.out.println("지구의 표면적: " + Earth.EARTH_SURFACE_AREA + "  km^2");
    }
}
```

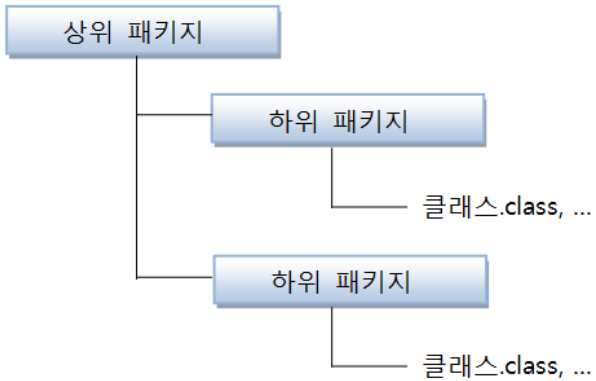
EarthExample.java 상수 사용

# 12절. 패키지(package)

## ❖ 패키지란?

- 클래스를 기능별로 묶어서 그룹 이름을 붙여 놓은 것
  - 파일들을 관리하기 위해 사용하는 폴더(디렉토리)와 비슷한 개념
  - 패키지의 물리적인 형태는 파일 시스템의 폴더

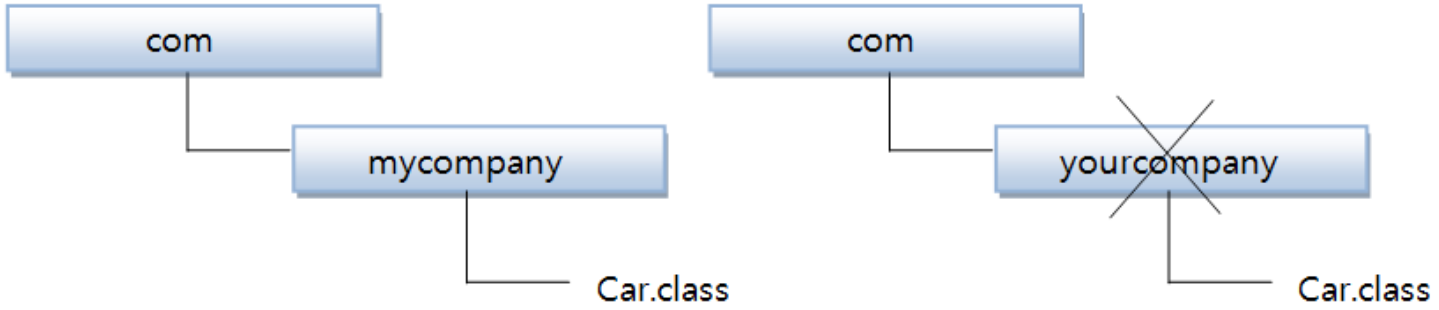
- 클래스 이름의 일부
  - 클래스를 유일하게 만들어주는 식별자
  - 전체 클래스 이름 = 상위패키지.하위패키지.클래스
  - 클래스명이 같아도 패키지명이 다르면 다른 클래스로 취급



# 12절. 패키지(package)

## ❖ 패키지란?

- 클래스 선언할 때 패키지 결정
  - 클래스 선언할 때 포함될 패키지 선언
  - 클래스 파일은(~.class) 선언된 패키지와 동일한 폴더 안에서만 동작
  - 클래스 파일은(~.class) 다른 폴더 안에 넣으면 동작하지 않음



# improt문

- 사용할 클래스가 속한 패키지를 지정하는데 사용.
- import문을 사용하면 클래스를 사용할 때 패키지명을 생략할 수 있다.

```
class ImportTest {
    java.util.Date today = new java.util.Date();
    // ...
}
```

→

```
import java.util.*;

class ImportTest {
    Date today = new Date();
}
```

date로 세부적으로 지정할 필요없음. 메모리부하 무관

- java.lang패키지의 클래스는 import하지 않고도 사용할 수 있다.

String, Object, System, Thread ...

```
import java.lang.*;

class ImportTest2
{
    public static void main(String[] args)
    {
        System.out.println("Hello World!");
    }
}
```

```
public static void main(java.lang.String[] args)
{
    java.lang.System.out.println("Hello World!");
}
```

# 12절. 패키지(package)

## ❖ import문의 선언

- import문은 패키지문과 클래스선언의 사이에 선언한다.

일반적인 소스파일(*.java)의 구성은 다음의 순서로 되어 있다.

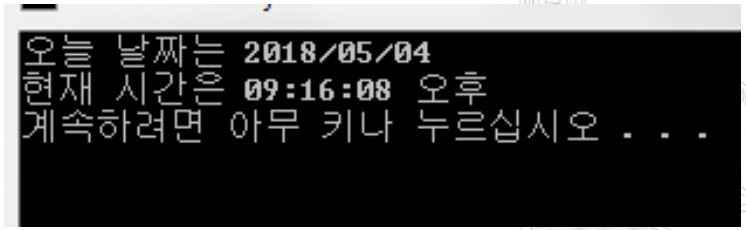
① package문
② import문
③ 클래스 선언

- import문을 선언하는 방법은 다음과 같다.

```
import 패키지명.클래스명;
   또는
import 패키지명.*;
```

# import문 예제

```
1  import java.text.SimpleDateFormat;
2  import java.util.Date;
3
4  class ImportTest
5  {
6      public static void main(String[] args)
7      {
8          Date today = new Date();
9
10         SimpleDateFormat date = new SimpleDateFormat("yyyy/MM/dd");
11         SimpleDateFormat time = new SimpleDateFormat("hh:mm:ss a");
12
13         System.out.println("오늘 날짜는 " + date.format(today));
14         System.out.println("현재 시간은 " + time.format(today));
15     }
16 }
17
```



# static import문

- static import문을 사용하면 static 멤버를 호출할 때 클래스 이름을 생략가능
  - system.out.println(Math.random());
    - out.println(random())

```
1  import static java.lang.System.out;
2  import static java.lang.Math.*;
3
4  class StaticImportEx1 {
5      public static void main(String[] args) {
6          // System.out.println(Math.random());
7          out.println(random());
8
9          // System.out.println("Math.PI :"+Math.PI);
10         out.println("Math.PI :" + PI);
11     }
12 }
```

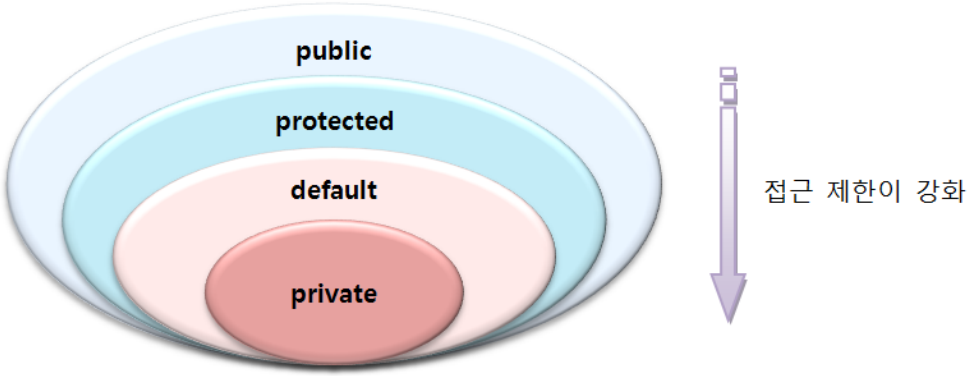
```
0.8167888631238076
Math.PI :3.141592653589793
계속하려면 아무 키나 누르십시오 . . .
```

# 13절. 접근 제한자

## ❖ 접근 제한자(Access Modifier)

- 클래스 및 클래스의 구성 멤버에 대한 접근을 제한하는 역할
  - 다른 패키지에서 클래스를 사용하지 못하도록 (클래스 제한)
  - 클래스로부터 객체를 생성하지 못하도록 (생성자 제한)
  - 특정 필드와 메소드를 숨김 처리 (필드와 메소드 제한)

- 접근 제한자의 종류

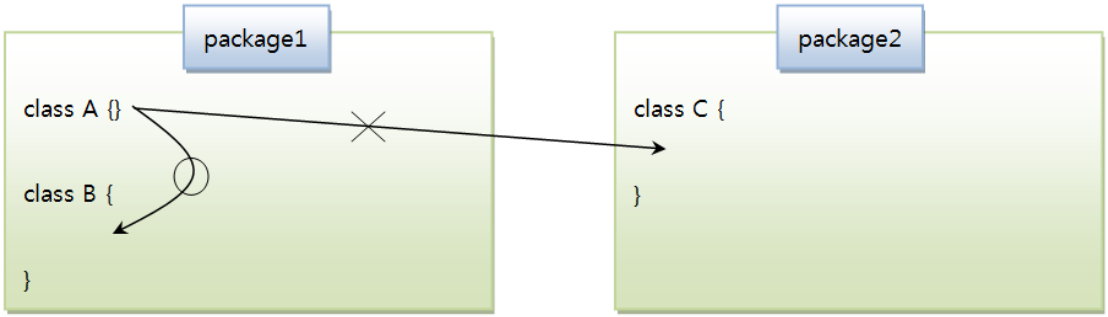



| 접근 제한 | 적용 대상 | 접근할 수 없는 클래스 |
|---|---|---|
| public | 클래스, 필드, 생성자, 메소드 | 없음 |
| protected | 필드, 생성자, 메소드 | 자식 클래스가 아닌 다른 패키지에 소속된 클래스 |
| default | 클래스, 필드, 생성자, 메소드 | 다른 패키지에 소속된 클래스 |
| private | 필드, 생성자, 메소드 | 모든 외부 클래스 |

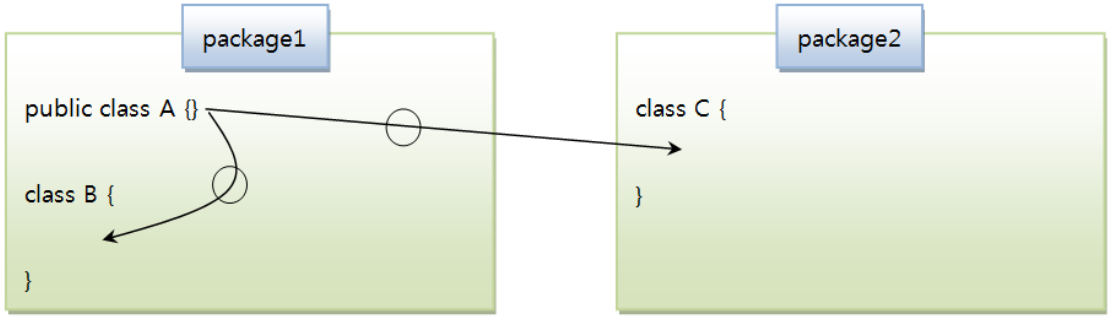
# 13절. 접근 제한자

## ❖ 클래스의 접근 제한

- default
  - 클래스 선언할 때 public 생략한 경우
  - 다른 패키지에서는 사용 불가

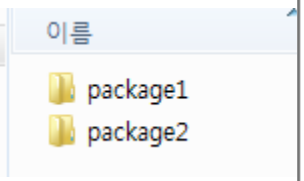



- public
  - 다른 개발자가 사용할 수 있도록 라이브러리 클래스로 만들 때 유용



## A.java 클래스의 접근 제한

```
1  package package1;
2
3  class A {
4
5  }
6
```




## B.java 클래스의 접근 제한

```
1  package sec13.exam01_class_access.package1;
2
3  public class B {
4      A a;
5  }
```

```
1  package sec13.exam01_class_access.package2;
2
3  import sec13.exam01_class_access.package1.*;
4
5  public class C {
6      //A a;
7      B b;
8  }
```

C.java 클래스의 접근 제한

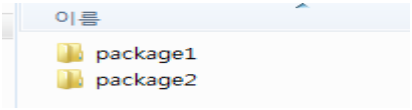
# 13절. 접근 제한자

## ❖ 생성자 접근 제한 (p.260~262)

- 생성자가 가지는 접근 제한에 따라 호출 여부 결정



## A.java 생성자의 접근 제한




## B.java 생성자의  접근 제한

```java
package sec13.exam02_constructor_access.package1;

public class A {
    //필드
    A a1 = new A(true);
    A a2 = new A(1);
    A a3 = new A("문자열");

    //생성자
    public A(boolean b) {}
    A(int b) {}
    private A(String s) {}
}
```

위에 패키지명을 보고 구분하기

```java
package sec13.exam02_constructor_access.package1;

public class B {
    //필드
    A a1 = new A(true);
    A a2 = new A(1);
    //A a3 = new A("문자열");
}
```

다른 폴더도 가능

같은 패키지(폴더) 내에서만

같은 클래스 내부에서만

```java
package sec13.exam02_constructor_access.package2;

import sec13.exam02_constructor_access.package1.A;

public class C {
  //필드
    A a1 = new A(true);
    //A a2 = new A(1);
    //A a3 = new A("문자열");
}
```

## C.java 생성자의 접근 제한

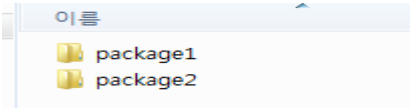
# 13절. 접근 제한자

## ❖ 필드와 메소드의 접근 제한 (p.262~264)

- 클래스 내부, 패키지 내, 패키지 상호간에 사용할 지 고려해 선언



## A.java 필드와 메소드의 접근 제한




## B.java 필드와 메소드의 접근 제한

```java
package sec13.exam03_field_method_access.package1;

public class A {
    //필드
    public int field1;
    int field2;
    private int field3; 클래스 단위

    //생성자
    public A() {
        field1 = 1;
        field2 = 1;
        field3 = 1;

        method1();
        method2();
        method3();
    }

    //메소드
    public void method1() {}
    void method2() {}
    private void method3() {}
}
```

```java
package sec13.exam03_field_method_access.package1;

public class B {
    public B() {
        A a = new A();
        a.field1 = 1;
        a.field2 = 1;
        //a.field3 = 1;

        a.method1();
        a.method2();
        //a.method3();
    }
}
```

연결 통로

```java
package sec13.exam03_field_method_access.package2;

import sec13.exam03_field_method_access.package1.A;

public class C {
    public C() {
        A a = new A();
        a.field1 = 1;
        //a.field2 = 1;
        //a.field3 = 1;

        a.method1();
        //a.method2();
        //a.method3();
    }
}
```

뭔가 import했다고 하더라도 본질적으로는 패키지가 다르기 때문에 X

## C.java 필드와 메소드의 접근 제한

# 14절. Getter와 Setter

- 클래스 선언할 때 필드는 일반적으로 private 접근 제한
  - 읽기 전용 필드가 있을 수 있음 (Getter의 필요성)
  - 외부에서 엉뚱한 값으로 변경할 수 없도록 (Setter의 필요성)

- Getter
  - private 필드의 값을 리턴 하는 역할 – 필요할 경우 필드 값 가공
  - getFieldName() 또는 isFieldName() 메소드
    - 필드 타입이 boolean 일 경우 isFieldName()

- Setter
  - 외부에서 주어진 값을 필드 값으로 수정
    - 필요할 경우 외부의 값을 유효성 검사
  - setFieldName(타입 변수) 메소드
    - 매개 변수 타입은 필드의 타입과 동일



## Car.java _Getter와 Setter 메소드 선언

```java
package sec14.exam01_getter_setter;

public class Car {
    //필드
    private int speed;
    private boolean stop;

    //생성자

    //메소드
    public int getSpeed() {
        return speed;
    }
    public void setSpeed(int speed) {
        if(speed < 0) {
            this.speed = 0;
            return;
        } else {
            this.speed = speed;
        }
    }

    public boolean isStop() {
        return stop;
    }
    public void setStop(boolean stop) {
        this.stop = stop;
        this.speed = 0;
    }
}
```

```java
package sec14.exam01_getter_setter;

public class CarExample {
    public static void main(String[] args) {
        Car myCar = new Car();

        //잘못된 속도 변경
        myCar.setSpeed(-50);

        System.out.println("현재 속도: " + myCar.getSpeed());

        //올바른 속도 변경
        myCar.setSpeed(60);

        //멈춤
        if(!myCar.isStop()) {
            myCar.setStop(true);
        }

        System.out.println("현재 속도: " + myCar.getSpeed());
    }
}
```

## CarExample.java _Getter와 Setter 메소드 사용

## Car.java _Getter와 Setter 메소드 선언

```java
public class Car {
    //필드
    private int speed;
    private boolean stop;

    //생성자

    //메소드
    public int getSpeed() {
        return speed;
    }
    public void setSpeed(int speed) {
        if(speed < 0) {
            this.speed = 0;
            return;
        } else {
            this.speed = speed;
        }
    }

    public boolean isStop() {
        return stop;
    }
    public void setStop(boolean stop) {
        this.stop = stop;
        this.speed = 0;
    }
}
```

```java
public class CarExample {
    public static void main(String[] args) {
        Car myCar = new Car();

        //잘못된 속도 변경
        myCar.setSpeed(-50);

        System.out.println("현재 속도: " + myCar.getSpeed());

        //올바른 속도 변경
        myCar.setSpeed(60);

        //멈춤
        if(!myCar.isStop()) {
            myCar.setStop(true);
        }

        System.out.println("현재 속도: " + myCar.getSpeed());
    }
}
```

CarExample.java _Getter와 Setter 메소드 사용

# 7장. 상속

이것이 자바다



# Contents


- 1절. 상속 개념
- 2절. 클래스 상속(extends)
- 3절. 부모 생성자 호출(super(…))
- 4절. 메소드 재정의(Override)
- 5절. final 클래스와 final 메소드
- 6절. protected 접근 제한자
- 7절. 타입변환과 다형성(polymorphism)
- 8절. 추상 클래스(Abstract Class)

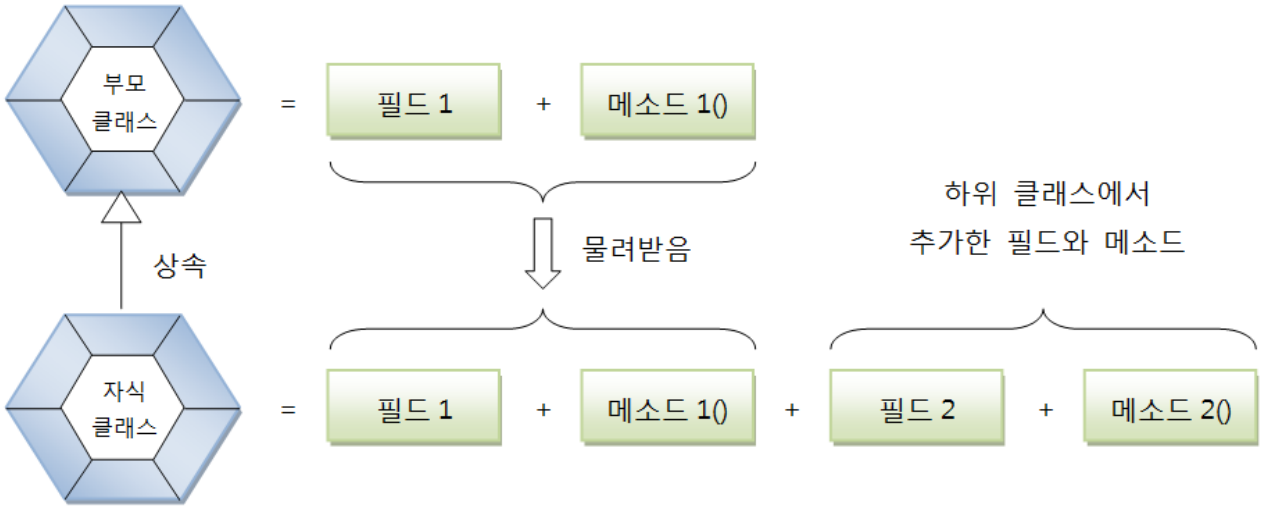



# 1절. 상속 개념

## ❖ 상속(Inheritance)이란?

- 현실 세계:
  - 부모가 자식에게 물려주는 행위
  - 부모가 자식을 선택해서 물려줌

- 객체 지향 프로그램:
  - 자식(하위, 파생) 클래스가 부모(상위) 클래스의 멤버를 물려받는 것
  - 자식이 부모를 선택해 물려받음
  - 상속 대상: 부모의 필드와 메소드



# 1절. 상속 개념

## ❖ 상속(Inheritance) 개념의 활용

- 상속의 효과
  - 부모 클래스 재사용해 자식 클래스 빨리 개발 가능
  - 반복된 코드 중복 줄임
  - 유지 보수 편리성 제공
  - 객체 다형성 구현 가능

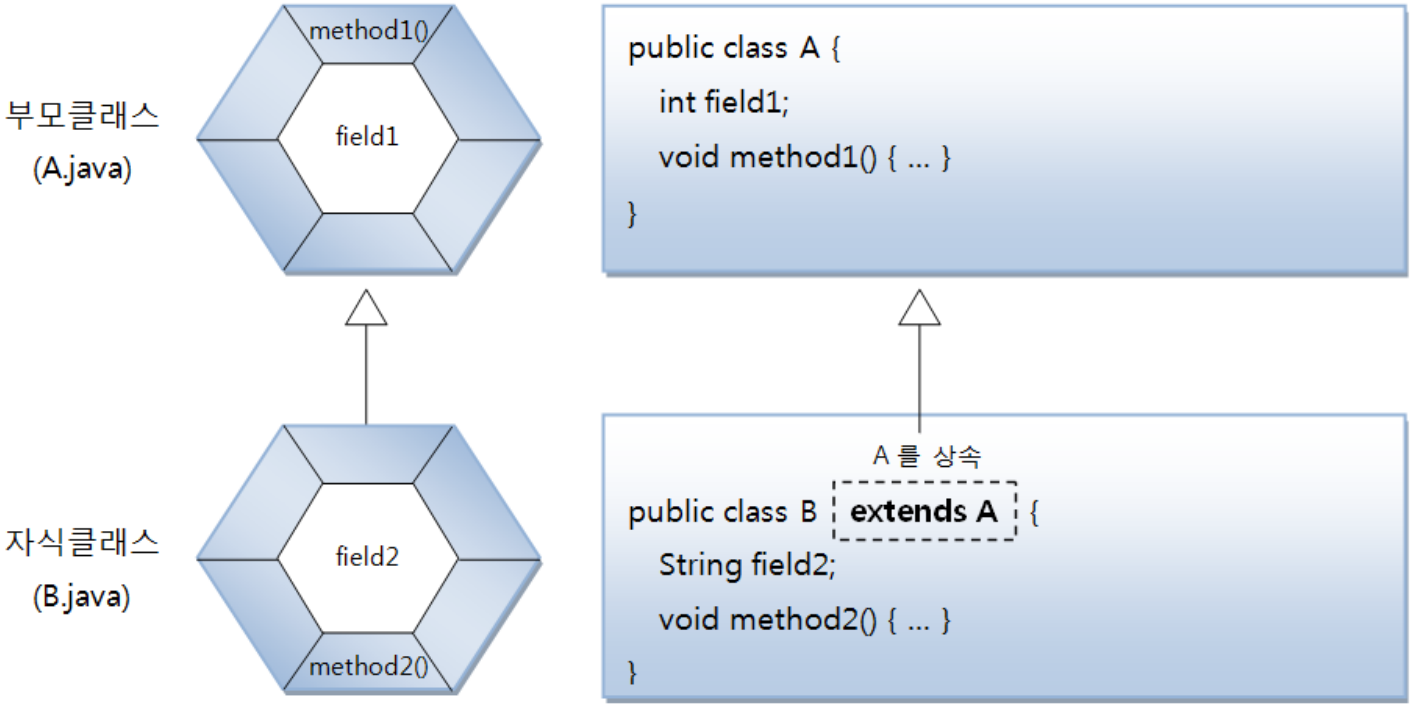
- 상속 대상 제한
  - 부모 클래스의 private 접근 갖는 필드와 메소드 제외
  - 부모 클래스가 다른 패키지에 있을 경우, default 접근 갖는 필드와 메소드도 제외



# 2절. 클래스 상속(extends)

## ❖ extends 키워드

- 자식 클래스가 상속할 부모 클래스를 지정하는 키워드




- 자바는 단일 상속 - 부모 클래스 나열 불가

```
class 자식클래스 extends 부모클래스1, 부모클래스2 {
}
```

## CellPhone.java _부모 클래스

```
1  public class CellPhone {
2      //필드
3      String model;
4      String color;
5
6      //생성자
7
8      //메소드
9      void powerOn() { System.out.println("전원을 켭니다."); }
10     void powerOff() { System.out.println("전원을 끕니다."); }
11     void bell() { System.out.println("벨이 울립니다."); }
12     void sendVoice(String message) { System.out.println("자기: " + message); }
13     void receiveVoice(String message) { System.out.println("상대방: " + message); }
14     void hangUp() { System.out.println("전화를 끊습니다."); }
15 }
16
```



## DmbCellPhone.java _자식 클래스

```
1  public class DmbCellPhone extends CellPhone {
2      //필드
3      int channel;
4
5      //생성자
6      DmbCellPhone(String model, String color, int channel) {
7          this.model = model;
8          this.color = color;
9          this.channel = channel;
10     }
11
12     //메소드
13     void turnOnDmb() {
14         System.out.println("채널 " + channel + "번 DMB 방송 수신을 시작합니다.");
15     }
16     void changeChannelDmb(int channel) {
17         this.channel = channel;
18         System.out.println("채널 " + channel + "번으로 바꿉니다.");
19     }
20     void turnOffDmb() {
21         System.out.println("DMB 방송 수신을 멈춥니다.");
22     }
23 }
24
```

this. 외래키랑 비슷?

자식 생성자 호출

부모 생성자 호출(자식생성자)

부모 객체 생성(0x10)

자식객체생성(0x100 {0x10}
-> this. 가능 (논리적으로)
부모객체의 주소를 받은 상태에서
객체가 생성되었기 때문에 가능

## DmbCellPhoneExample.java _자식 클래스 사용

```
    ├---+---1---+---2---+---3---+---4---+---5---+---6---+---7---+--
 1  public class DmbCellPhoneExample {
 2      public static void main(String[] args) {
 3          //DmbCellPhone 객체 생성
 4          DmbCellPhone dmbCellPhone = new DmbCellPhone("자바폰", "검정", 10);
 5
 6          //CellPhone으로부터 상속 받은 필드
 7          System.out.println("모델: " + dmbCellPhone.model);
 8          System.out.println("색상: " + dmbCellPhone.color);
 9
10          //DmbCellPhone의 필드
11          System.out.println("채널: " + dmbCellPhone.channel);
12
13          //CellPhone으로부터 상속 받은 메소드 호출
14          dmbCellPhone.powerOn();
15          dmbCellPhone.bell();
16          dmbCellPhone.sendVoice("여보세요");
17          dmbCellPhone.receiveVoice("안녕하세요! 저는 홍길동인데요");
18          dmbCellPhone.sendVoice("아~ 예 반갑습니다.");
19          dmbCellPhone.hangUp();
20
21          //DmbCellPhone의 메소드 호출
22          dmbCellPhone.turnOnDmb();
23          dmbCellPhone.changeChannelDmb(12);
24          dmbCellPhone.turnOffDmb();
25      }
26  }
```

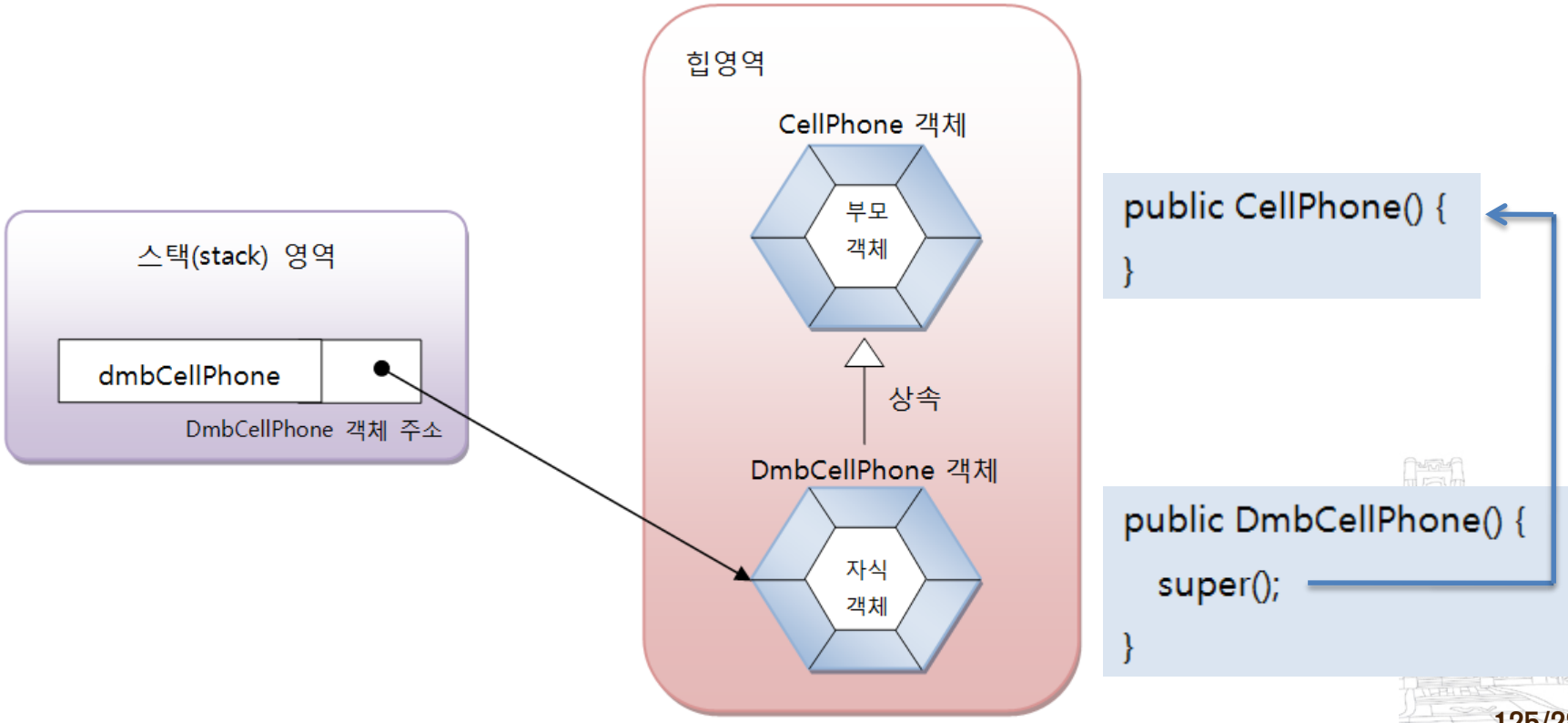


# 3절. 부모 생성자 호출(super(…))

## ❖ 자식 객체 생성하면 부모 객체도 생성되는가?

- 부모 없는 자식 없음
  - 자식 객체 생성할 때는 부모 객체부터 생성 후 자식 객체 생성
  - 부모 생성자 호출 완료 후 자식 생성자 호출 완료

```
DmbCellPhone dmbCellPhone = new DmbCellPhone();
```



# 3절. 부모 생성자 호출(super(…))

## ❖ 명시적인 부모 생성자 호출

- 부모 객체 생성할 때, 부모 생성자 선택해 호출

```
자식클래스( 매개변수선언, ... ) {
   super( 매개값, ... );
   ...
}
```

- super(매개값,…)
  - 매개값과 동일한 타입, 개수, 순서 맞는 부모 생성자 호출
- 부모 생성자 없다면 컴파일 오류 발생
- 반드시 자식 생성자의 첫 줄에 위치
- 부모 클래스에 기본(매개변수 없는) 생성자가 없다면 필수 작성



## People.java _부모 클래스

```java
public class People {
    public String name;
    public String ssn;

    public People(String name, String ssn) {
        this.name = name;
        this.ssn = ssn;
    }
}
```

```java
public class Student extends People{
    public int studentNo;

    public Student(String name, String ssn, int studentNo) {
        super(name, ssn);      반드시 첫 번째 행에 두기!
        this.studentNo = studentNo;
    }
}
```

People.java _자식 클래스



## People.java _자식 객체 이용

```
1 public class StudentExample {
2     public static void main(String[] args) {
3         Student student = new Student("홍길동", "123456-1234567", 1);
4         System.out.println("name : " + student.name);
5         System.out.println("ssn : " + student.ssn);
6         System.out.println("studentNo : " + student.studentNo);
7     }
8 }
```

자식이 먼저 호출되고 객체를 생성하기 전에 super로 부모객체가 생성됨
부모 없는 자식이 없듯이 부모객체가 생성된 이후에 자식객체가 생성되고
부모의 메모리주소를 자식객체가 메타데이터로 가지고 있음



# 오버라이딩(overriding)이란?

"조상클래스로부터 상속받은 메서드의 내용을 상속받는 클래스에 맞게 변경하는 것을 오버라이딩이라고 한다."

*

```
class Point {
      int x;
      int y;

      String getLocation() {
            return "x :" + x + ", y :"+ y;
      }
}

class Point3D extends Point {
      int z;
      String getLocation() {      // 오버라이딩
            return "x :" + x + ", y :"+ y + ", z :" + z;
      }
}
```

# 오버라이딩의 조건

1. 선언부가 같아야 한다.(이름, 매개변수, 리턴타입)

2. 접근제어자를 좁은 범위로 변경할 수 없다.
   - 조상의 메서드가 protected라면, 범위가 같거나 넓은 protected나 public으로만 변경할 수 있다.
     - public**을** default**나** private**으로 수정 불가**
     - **반대로** default**는** public **으로 수정 가능**

3. 조상클래스의 메서드보다 많은 수의 예외를 선언할 수 없다.

```
class Parent {
    void parentMethod() throws IOException, SQLException {
        // ...
    }
}

class Child extends Parent {
    void parentMethod() throws IOException {
        //..
    }
}

class Child2 extends Parent {
    void parentMethod() throws Exception {
        //..
    }
}
```



# 4절. 메소드 재정의(Override)

## ❖ @Override 어노테이션

- 컴파일러에게 부모 클래스의 메소드 선언부와 동일한지 검사 지시
- 정확한 메소드 재정의 위해 붙여주면 OK

## ❖ 메소드 재정의 효과

- 부모 메소드는 숨겨지는 효과 발생
  - 재정의된 자식 메소드 실행



## Calculator.java _부모 클래스

```java
public class Calculator {
    double areaCircle(double r) {
        System.out.println("Calculator 객체의 areaCircle() 실행");
        return 3.14159 * r * r;
    }
}
```

메서드에도 접근제어자가 붙는다
없으면 기본값 default다

```java
@Override
double areaCircle(double r) {
    System.out.println("Computer 객체의 areaCircle() 실행");
        return Math.PI * r * r;
    }
}
```

## Computer.java _자식 클래스

```
Calculator 객체의 areaCircle() 실행
원면적 : 314.159

Computer 객체의 areaCircle() 실행
원면적 : 314.1592653589793
계속하려면 아무 키나 누르십시오 . . .
```

```java
public class ComputerExample {
    public static void main(String[] args) {
        int r = 10;
        Calculator calculator = new Calculator();
        System.out.println("원면적 : " + calculator.areaCircle(r));
        System.out.println();
        Computer computer = new Computer();
        System.out.println("원면적 : " + computer.areaCircle(r));
    }
}
```

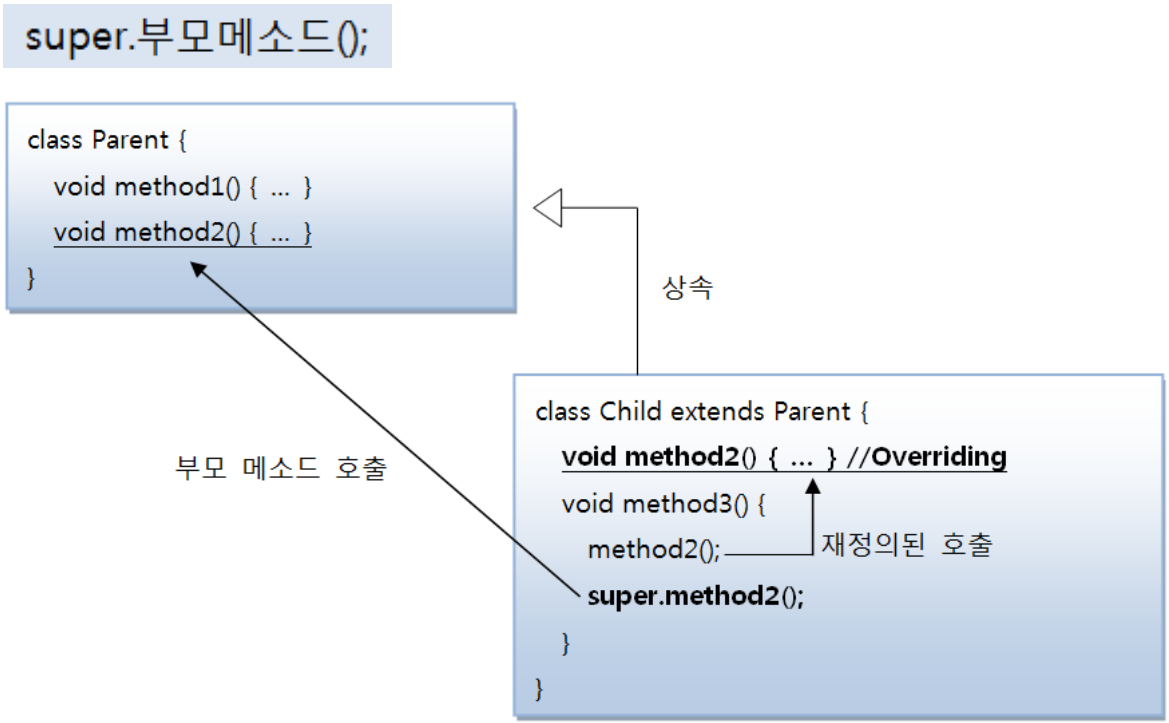
## ComputerExample.java _메소드 오버라이딩 테스트

# 4절. 메소드 재정의(Override)

## ❖ 부모 메소드 사용(super)

- 메소드 재정의는 부모 메소드 숨기는 효과 !!
  - 자식 클래스에서는 재정의된 메소드만 호출

- 자식 클래스에서 수정되기 전 부모 메소드 호출 – super 사용
  - super는 부모 객체 참조(참고: this는 자신 객체 참조)



## Airplane.java _super 변수

```java
public class Airplane {
    public void land() {
        System.out.println("착륙합니다.");
    }
    public void fly() {
        System.out.println("일반비행합니다.");
    }
    public void takeOff() {
        System.out.println("이륙합니다.");
    }
}
```

static 클래스명.변수명으로 호출가능

## SupersonicAirplane.java _super 변수

```java
public class SupersonicAirplane extends Airplane {
    public static final int NORMAL = 1;
    public static final int SUPERSONIC = 2;

    public int flyMode = NORMAL;

    @Override
    public void fly() {
        if(flyMode == SUPERSONIC) {
            System.out.println("초음속비행합니다.");
        } else {
            //Airplane 객체의 fly() 메소드 호출
            super.fly();
        }
    }
```

new예약어를 사용해서 호출 및 응용

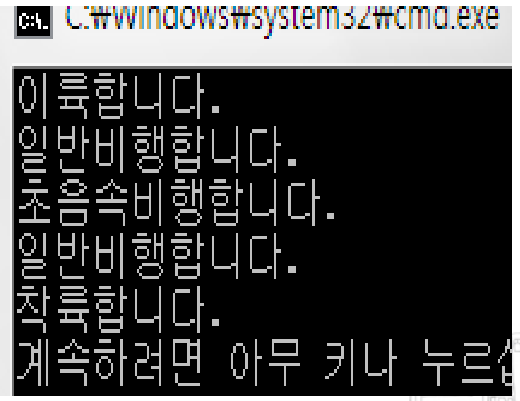
줄간격X

메서드 접근제어자 주의

## SupersoniAirplaneExample.java _super 변수

```java
public class SupersonicAirplaneExample {
    public static void main(String[] args) {
        SupersonicAirplane sa = new SupersonicAirplane();
        sa.takeOff();
        sa.fly();
        sa.flyMode = SupersonicAirplane.SUPERSONIC;
        sa.fly();
        sa.flyMode = SupersonicAirplane.NORMAL;
        sa.fly();
        sa.land();
    }
}
```

부모의 이름을 감히 부르지 않고 엄마 아빠같은 느낌



# 5절. final 클래스와 final 메소드

## ❖ final 키워드의 용도

- final 필드: 수정 불가 필드
- final 클래스: 부모로 사용 불가한 클래스 상속 X
- final 메소드: 자식이 재정의할 수 없는 메소드

## ❖ 상속할 수 없는 final 클래스

- 자식 클래스 만들지 못하도록 final 클래스로 생성

```
public final class 클래스 { ... }
```

```
public final class String { .. }
```

```
public class NewString extends String { ... }
```

## ❖ 오버라이딩 불가한 final 메소드

- 자식 클래스가 재정의 못하도록 부모 클래스의 메소드를 final로 생성



## Car.java _재정의 할 수 없는 final 메소드

```java
public class Car {
    //필드
    public int speed;

    //메소드
    public void speedUp() {
        speed += 1;
    }

    //final 메소드
    public final void stop() {
        System.out.println("차를 멈춤");
        speed = 0;
    }
}
```

```
    ├---+----1----+----2----+----3----+----4----+---
```

```java
public class SportsCar extends Car {
    @Override
    public void speedUp() {
        speed += 10;
    }

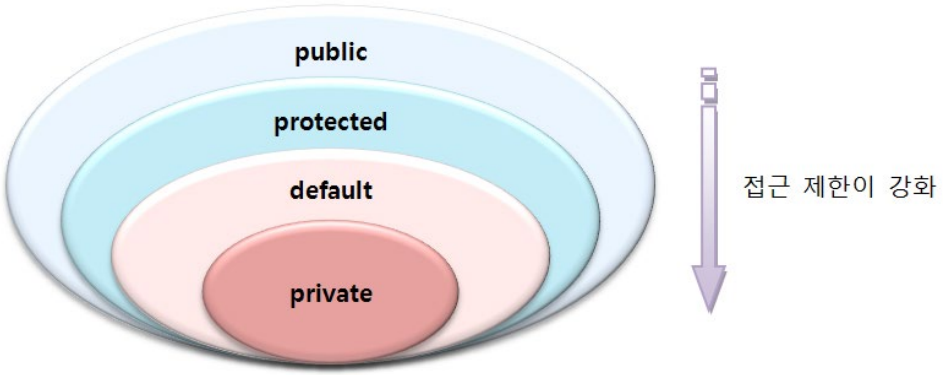
    //오버라이딩을 할 수 없음
    /*
    @Override
    public void stop() {
        System.out.println("스포츠카를 멈춤");
        speed = 0;
    }
    */
}
```

SportsCar.java _재정의 할 수 없는 final 메소드

# 6절. protected 접근 제한자

## ❖ protected 접근 제한자

- 상속과 관련된 접근 제한자
  - 같은 패키지: default와 동일
  - 다른 패키지: 자식 클래스만 접근 허용 (p.303~305)




| 접근 제한 | 적용할 내용 | 접근할 수 없는 클래스 |
|---|---|---|
| public | 클래스, 필드, 생성자, 메소드 | 없음 |
| protected | 필드, 생성자, 메소드 | 자식 클래스가 아닌 다른 패키지에 소속된 클래스 |
| default | 클래스, 필드, 생성자, 메소드 | 다른 패키지에 소속된 클래스 |
| private | 필드, 생성자, 메소드 | 모든 외부 클래스 |

package1

```
public class A{
protected  멤버
}

class B {


}
```

package 2

```
class C{
protected 멤버
}

class D extends A {


}
```

B.java _protected 접근 제한자

# 접근 제한자를 이용한 캡슐화 1/2 보충교재

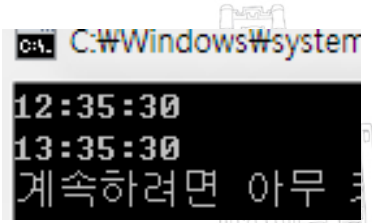
public 접근제어자로 클래스명은 1개만 ->

```
1  public class TimeTest {
2          public static void main(String[] args)
3          {
4                  Time t = new Time(12, 35, 30);
5                  System.out.println(t);
6  //              t.hour = 13;
7                  t.setHour(t.getHour()+1);    // 현재시간보다 1시간 후로 변경한다.
8                  System.out.println(t);       // System.out.println(t.toString());과 같다.
9          }
10 }
11
12 class Time {
13         private int hour;
14         private int minute;
15         private int second;
16
17         Time(int hour, int minute, int second) {
18                 setHour(hour);
19                 setMinute(minute);
20                 setSecond(second);
21         }
```

유효성 검사를 거치기 위해서 중간단계를 거침

# 접근 제한자를 이용한 캡슐화 2/2 보충교재

```
22
23        public int getHour() { return hour; }
24        public void setHour(int hour) {
25              if (hour < 0 || hour > 23) return;
26              this.hour = hour;
27        }
28        public int getMinute() { return minute; }
29        public void setMinute(int minute) {
30              if (minute < 0 || minute > 59) return;
31              this.minute = minute;
32        }
33        public int getSecond() { return second; }
34        public void setSecond(int second) {
35              if (second < 0 || second > 59) return;
36              this.second = second;
37        }
38        public String toString() {
39              return hour + ":" + minute + ":" + second;
40        }
41  }
42
```



# 7절. 타입변환과 다형성(polymorphism)

## ❖ 다형성 (多形性, Polymorphism)

- 같은 타입이지만 실행 결과가 다양한 객체 대입(이용) 가능한 성질
  - 부모 타입에는 모든 자식 객체가 대입 가능
    - 자식 타입은 부모 타입으로 자동 타입 변환
  - 효과: 객체 부품화 가능



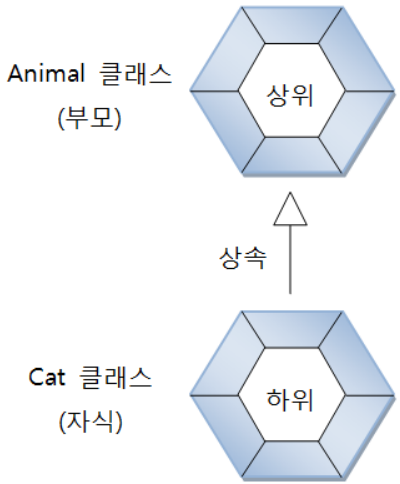
# 7절. 타입변환과 다형성(polymorphism)

## ❖ 자동 타입 변환(Promotion)

- 프로그램 실행 도중에 자동 타입 변환이 일어나는 것

자동 타입 변환

부모클래스 변수 = 자식클래스타입;




```
class Animal {
   ...
}
```

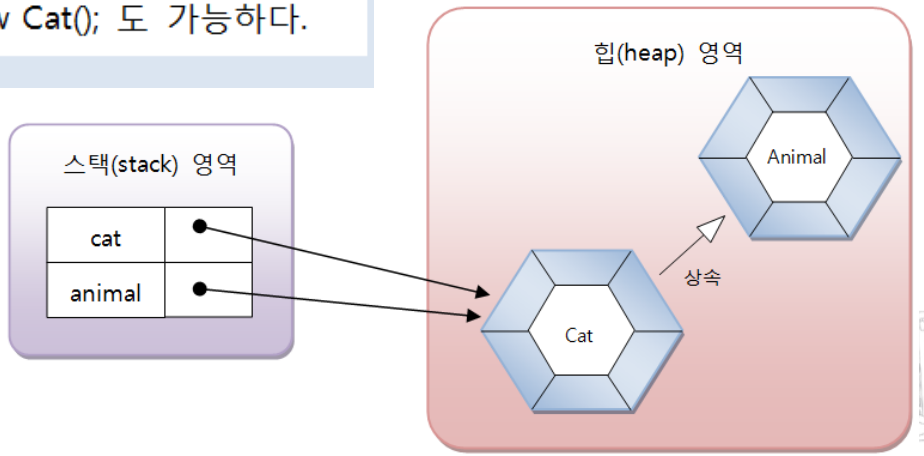
상속 UML 일반화

```
class Cat extends Animal {
   ...
}
```

```
Cat cat = new Cat();
Animal animal = cat;
```

Animal animal = new Cat(); 도 가능하다.

```
cat == animal    //true
```



# PromotionExample.java_자동 타입 변환

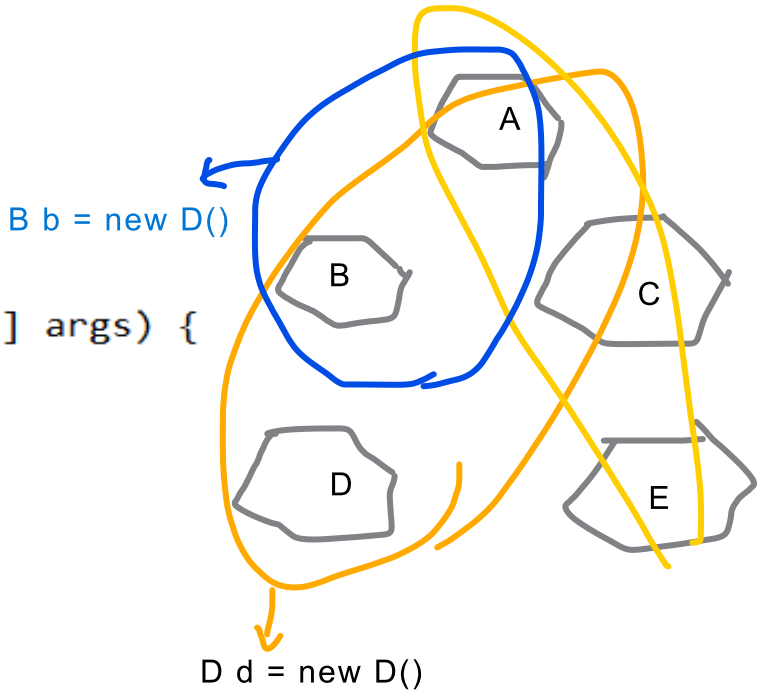
```java
class A {}
class B extends A {}
class C extends A {}
class D extends B {}
class E extends C {}


public class PromotionExample {
    public static void main(String[] args) {
        B b = new B();
        C c = new C();
        D d = new D();
        E e = new E();

        A a1 = b;
        A a2 = c;
        A a3 = d;
        A a4 = e;

        B b1 = d;
        C c1 = e;

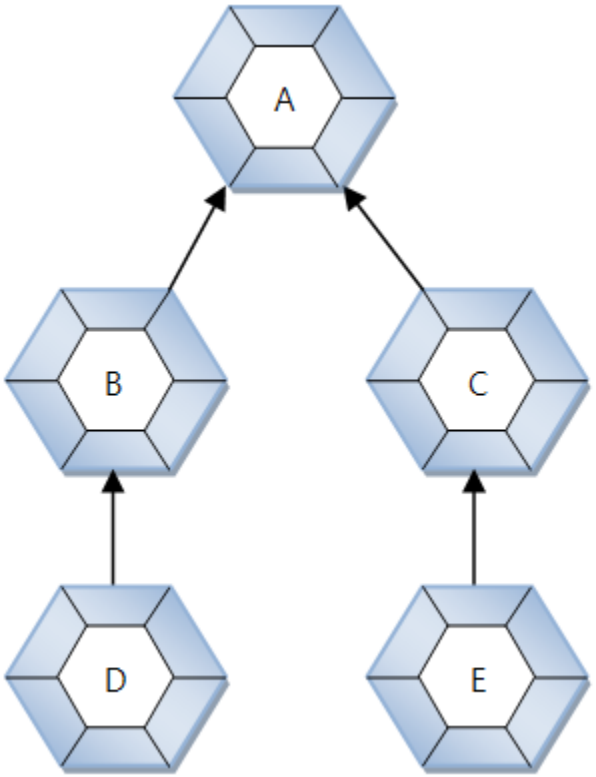
        //B b3 = e;
        //C c2 = d;
    }
}
```



# 7절. 타입변환과 다형성(polymorphism)

## ❖ 자동 타입 변환(Promotion)

- 바로 위의 부모가 아니더라도 상속 계층의 상위면 자동 타입 변환 가능
  - 변환 후에는 부모 클래스 멤버만 접근 가능 (p.308~310)




```
B b = new B();
C c = new C();
D d = new D();
E e = new E();
```

```
A a1 = b;   (가능)
A a2 = c;   (가능)
A a3 = d;   (가능)
A a4 = e;   (가능)

B b1 = d;   (가능)
C c1 = e;   (가능)

B b3 = e;   (불가능)
C c2 = d;   (불가능)
```

# Parent.java_ 자동 타입 변환 후의 멤버 접근

```
public class Parent {
    public void method1() {
        System.out.println("Parent-method1()");
    }

    public void method2() {
        System.out.println("Parent-method2()");
    }
}
```

```
public class Child extends Parent {
    @Override
    public void method2() {
        System.out.println("Child-method2()");
    }

    public void method3() {
        System.out.println("Child-method3()");
    }
}
```

child.java_ 자동 타입 변환 후의 멤버 접근

# ChildExample.java_ 자동 타입 변환 후의 멤버 접근

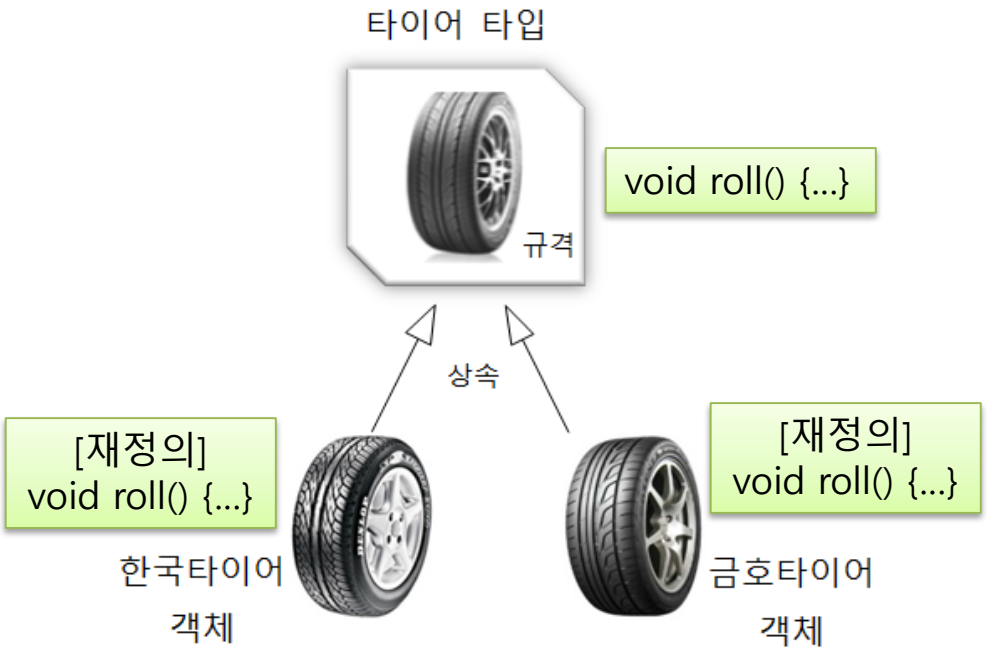
```
1  public class ChildExample {
2      public static void main(String[] args) {
3          Child child = new Child();
4
5          Parent parent = child;
6
7          parent.method1();
8
9          parent.method2();
10
11         //parent.method3();  (호출 불가능)
12     }
13 }
```

```
Parent-method1()
Child-method2()
```

# 7절. 타입변환과 다형성(polymorphism)

## ❖ 필드의 다형성 (p.311~317)

- 다형성을 구현하는 기술적 방법
  - 부모 타입으로 자동 변환
  - 재정의된 메소드(오버라이딩)



# Tire.java_ 타이어 클래스(필드의 다형성)

```
|----+----1----+----2----+----3----+----4----+----5----+----6----+----7----+----8----+----9----+----0---
1  public class Tire {
2      //필드
3      public int maxRotation;                 //최대 회전수(최대 수명)
4      public int accumulatedRotation;     //누적 회전수
5      public String location;                 //타이어의 위치
6
7      //생성자
8      public Tire(String location, int maxRotation) {
9          this.location = location;
10         this.maxRotation = maxRotation;
11     }
12
13     //메소드
14     public boolean roll() {
15         ++accumulatedRotation;
16         if(accumulatedRotation<maxRotation) {
17             System.out.println(location + " Tire 수명: " + (maxRotation-accumulatedRotation) + "회");
18             return true;
19         } else {
20             System.out.println("*** " + location + " Tire 펑크 ***");
21             return false;
22         }
23     }
24 }
25
```

# car.java_ Tire를 부품으로 가지는 클래스(필드의 다형성)

```java
public class Car {
    //필드
    Tire frontLeftTire = new Tire("앞왼쪽", 6);
    Tire frontRightTire = new Tire("앞오른쪽", 2);
    Tire backLeftTire = new Tire("뒤왼쪽", 3);
    Tire backRightTire = new Tire("뒤오른쪽", 4);

    //생성자

    //메소드
    int run() {
        System.out.println("[자동차가 달립니다.]");
        if(frontLeftTire.roll()==false) { stop(); return 1; };
        if(frontRightTire.roll()==false) { stop(); return 2; };
        if(backLeftTire.roll()==false) { stop(); return 3; };
        if(backRightTire.roll()==false) { stop(); return 4; };
        return 0;
    }

    void stop() {
        System.out.println("[자동차가 멈춥니다.]");
    }
}
```

0이 되면 false니까

하드코딩하면 실수할 가능성이 높음.



# HankookTire.java_ Tire의 자식 클래스(필드의 다형성)

```java
public class HankookTire extends Tire {
    //필드
    //생성자
    public HankookTire(String location, int maxRotation) {
        super(location, maxRotation);
    }
    //메소드
    @Override
    public boolean roll() {
        ++accumulatedRotation;
        if(accumulatedRotation<maxRotation) {
            System.out.println(location + " HankookTire 수명: " + (maxRotation-accumulatedRotation) + "회");
            return true;
        } else {
            System.out.println("*** " + location + " HankookTire 펑크 ***");
            return false;
        }
    }
}
```

# KumhoTire.java_ Tire의 자식 클래스(필드의 다형성)

```
1  public class KumhoTire extends Tire {
2      //필드
3      //생성자
4      public KumhoTire(String location, int maxRotation) {
5          super(location, maxRotation);
6      }
7      //메소드
8      @Override
9      public boolean roll() {
10         ++accumulatedRotation;
11         if(accumulatedRotation<maxRotation) {
12             System.out.println(location + " KumhoTire 수명: " + (maxRotation-accumulatedRotation) + "회");
13             return true;
14         } else {
15             System.out.println("*** " + location + " KumhoTire 펑크 ***");
16             return false;
17         }
18     }
19 }
20
```

# carExample.java_ 실행 클래스(필드의 다형성)

```
1  public class CarExample {
2      public static void main(String[] args) {
3          Car car = new Car();
4
5          for(int i=1; i<=5; i++) {
6              int problemLocation = car.run();
7              switch(problemLocation) {
8                  case 1:
9                      System.out.println("앞왼쪽 HankookTire로 교체");
10                     car.frontLeftTire = new HankookTire("앞왼쪽", 15);
11                     break;
12                 case 2:
13                     System.out.println("앞오른쪽 KumhoTire로 교체");
14                     car.frontRightTire = new KumhoTire("앞오른쪽", 13);
15                     break;
16                 case 3:
17                     System.out.println("뒤왼쪽 HankookTire로 교체");
18                     car.backLeftTire = new HankookTire("뒤왼쪽", 14);
19                     break;
20                 case 4:
21                     System.out.println("뒤오른쪽 KumhoTire로 교체");
22                     car.backRightTire = new KumhoTire("뒤오른쪽", 17);
23                     break;
24             }
25             System.out.println("-----------------------------------------");
26         }
27     }
28 }
```

# 7절. 타입변환과 다형성(polymorphism)

## ❖ 하나의 배열로 객체 관리 (p.318~320)

```
class Car {
    Tire frontLeftTire = new Tire("앞왼쪽", 6);
    Tire frontRightTire = new Tire("앞오른쪽", 2);
    Tire backLeftTire = new Tire("뒤왼쪽", 3);
    Tire backRightTire = new Tire("뒤오른쪽", 4);
}
```

```
class Car {
    Tire[] tires = {
        new Tire("앞왼쪽", 6),
        new Tire("앞오른쪽", 2),
        new Tire("뒤왼쪽", 3),
        new Tire("뒤오른쪽", 4)
    };
}
```

각각의 객체를 배열로 관리

```
tires[1] = new KumhoTire("앞오른쪽", 13);
```

```
int run() {
        System.out.println("[자동차가 달립니다.]");
        for(int i=0; i<tires.length; i++) {
            if(tires[i].roll()==false) {
                stop();
                return (i+1);
            }
        }
        return 0;
}
```

# Car.java_ Tire를 부품으로 가지는 클래스(다형성)

```java
public class Car {
    //필드
    Tire[] tires = {
            new Tire("앞왼쪽", 6),
            new Tire("앞오른쪽", 2),
            new Tire("뒤왼쪽", 3),
            new Tire("뒤오른쪽", 4)
    };

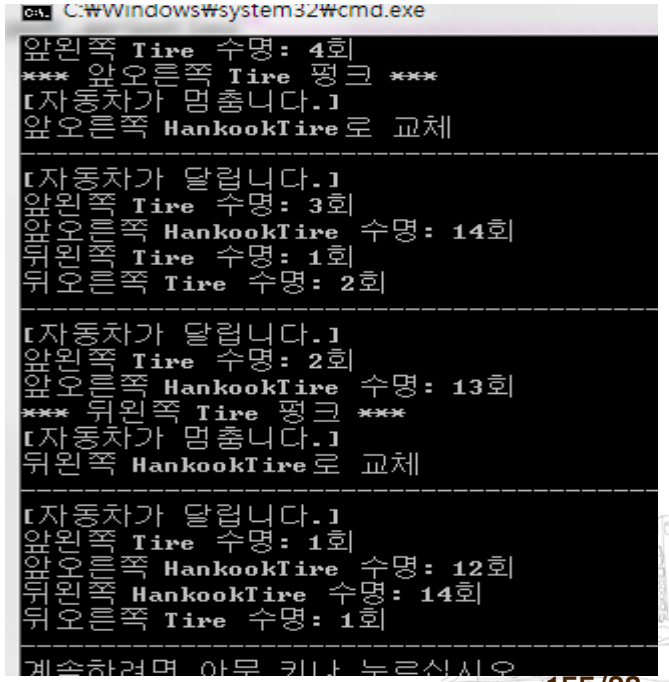
    //메소드
    int run() {
        System.out.println("[자동차가 달립니다.]");
        for(int i=0; i<tires.length; i++) {
            if(tires[i].roll()==false) {
                stop();
                return (i+1);
            }
        }
        return 0;
    }

    void stop() {
        System.out.println("[자동차가 멈춥니다.]");
    }
}
```



# CarExample.java_ 실행 클래스(다형성)

```
3 public class CarExample {
4     public static void main(String[] args) {
5         Car car = new Car();
6 
7         for(int i=1; i<=5; i++) {
8             int problemLocation = car.run();
9             if(problemLocation != 0) {
10                System.out.println(car.tires[problemLocation-1].location + " HankookTire로 교체");
11                car.tires[problemLocation-1] = new HankookTire(car.tires[problemLocation-1].location, 15);
12            }
13            System.out.println("--------------------------------------");
14        }
15    }
16 }
```



# 7절. 타입변환과 다형성(polymorphism)

## ❖ 매개변수의 다형성 (p.321~323)

- 매개변수가 클래스 타입일 경우
  - 해당 클래스의 객체 대입이 원칙이나 자식 객체 대입하는 것도 허용
    - 자동 타입 변환
    - 매개변수의 다형성

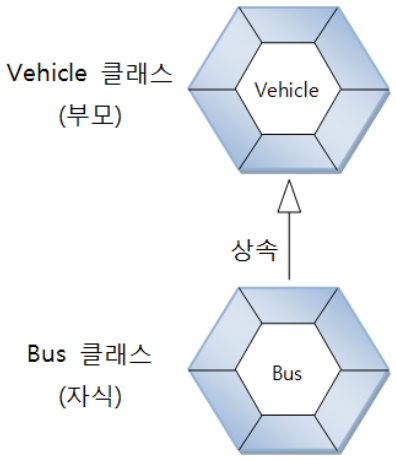
```
class Driver {
   void drive(Vehicle vehicle) {
      vehicle.run();
   }
}
```

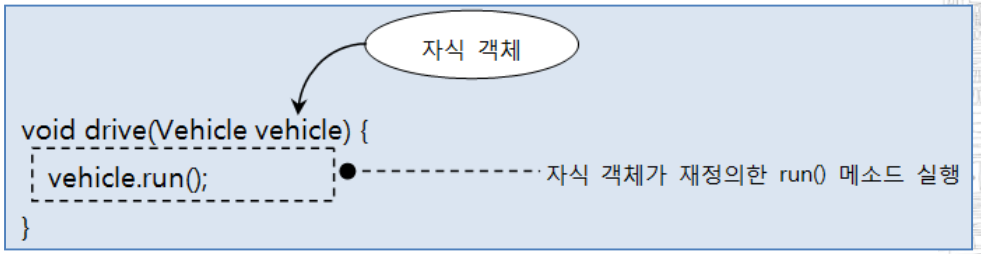
```
Driver driver = new Dirver();

Bus bus = new Bus();

driver.drive( bus );
```

자동 타입 변환 발생
Vehicle vehicle = bus;



## vehicle.java_ 부모 클래스(매개 변수의 다형성)

```java
public class Vehicle {
    public void run() {
        System.out.println("차량이 달립니다.");
    }
}
```

```java
public class Driver {
    public void drive(Vehicle vehicle) {
        vehicle.run();
    }
}
```

driver.java_ Vehicl을 이용하는 클래스(매개 변수의 다형성)

# Bus.java_ 자식 클래스(매개 변수의 다형성)

```java
public class Bus extends Vehicle {
    @Override
    public void run() {
        System.out.println("버스가 달립니다.");
    }
}
```

```java
public class Taxi extends Vehicle {
    @Override
    public void run() {
        System.out.println("택시가 달립니다.");
    }
}
```

Taxi.java_ 자식 클래스(매개 변수의 다형성)

```java
public class DriverExample {
    public static void main(String[] args) {
        Driver driver = new Driver();

        Bus bus = new Bus();
        Taxi taxi = new Taxi();

        driver.drive(bus);
        driver.drive(taxi);
    }
}
```

0x70

0x150

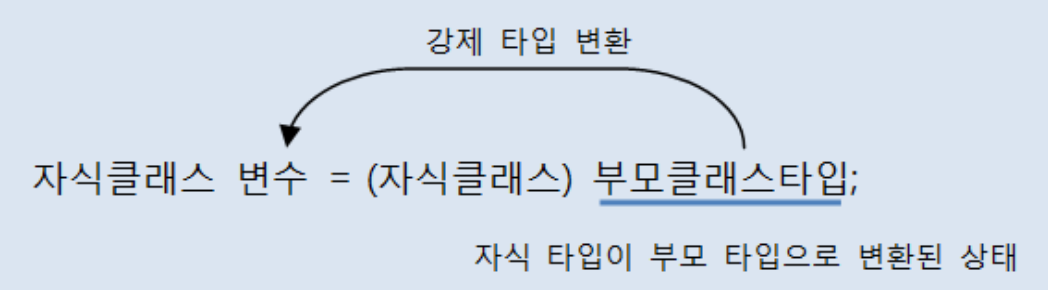
여기에는 스택 메서드영역에서 힙영역 객체의 메모리 주소를 가지고 있다.

DriverExample.java_ 실행 클래스(매개 변수의 다형성)

# 7절. 타입변환과 다형성(polymorphism)

## ❖ 강제 타입 변환(Casting) (p.324~325)

- 부모 타입을 자식 타입으로 변환하는 것




- 조건
  - 자식 타입을 부모 타입으로 자동 변환 후, 다시 자식 타입으로 변환할 때

- 강제 타입 변환 이 필요한 경우
  - 자식 타입이 부모 타입으로 자동 변환
    - 부모 타입에 선언된 필드와 메소드만 사용 가능
  - 자식 타입에 선언된 필드와 메소드를 다시 사용해야 할 경우

## Parent.java_ 부모 클래스(Casting)

```java
public class Parent {
    public String field1;

    public void method1() {
        System.out.println("Parent-method1()");
    }

    public void method2() {
        System.out.println("Parent-method2()");
    }
}
```

```java
public class Child extends Parent {
    public String field2;
    @Override 어노테이션 생략가능하기 때문에 동일형식의 메서드 유무 확인하기
    public void method3() {
        System.out.println("Child-method3()");
    }
}

```

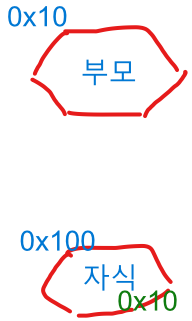
## Child.java_ 자식 클래스(Casting)

# ChildExample.java_ 강제 타입 변환(Casting)

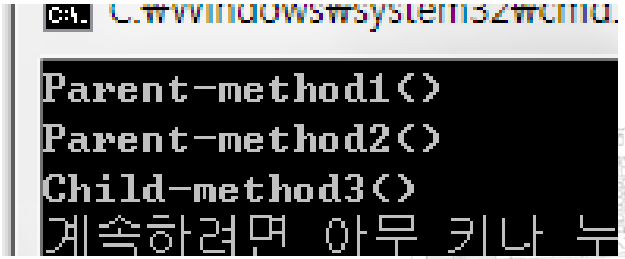
```java
public class ChildExample {
    public static void main(String[] args) {
        Parent parent = new Child();
        parent.field1 = "data1";
        parent.method1();
        parent.method2();
        /*
        parent.field2 = "data2";  //(불가능)
        parent.method3();         //(불가능)
        */

        Child child = (Child) parent;
        child.field2 = "yyy";  //(가능)
        child.method3();     //(가능)
    }
}

```




자식객체를 부모형으로 생성하면 부모의 메서드 필드를 공유할 수 있지만 자식이 재정의한 메서드가 아닌 자식만의 메서드는 활용할 수가 없다.
그걸 자식으로 캐스팅하면 자식의 메서드도 모두 활용이 가능하다.



# 7절. 타입변환과 다형성(polymorphism)

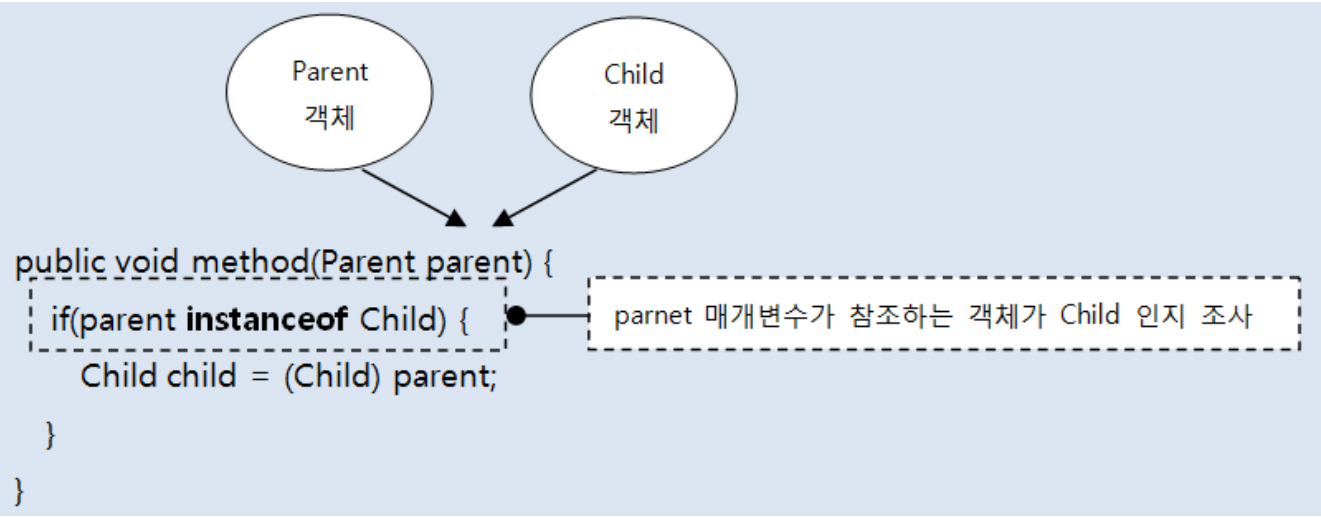
## ❖ 객체 타입 확인(instanceof) (p.326~329)

- 부모 타입이면 모두 자식 타입으로 강제 타입 변환할 수 있는 것 아님
  - ClassCastException 예외 발생 가능

```
Parent parent = new Parent();
Child child = (Child) parent;      //강제 타입 변환을 할 수 없다.
```

- 먼저 자식 타입인지 확인 후 강제 타입 실행해야 함

```
boolean result = 좌항(객체) instanceof 우항(타입)
```



## Parent.java_ 부모 클래스

```java
public class Parent { }
```

## Child.java_ 자식 클래스

```java
public class Child extends Parent { }
```

```java
public class InstanceofExample {
    public static void method1(Parent parent) {
        if(parent instanceof Child) {
            Child child = (Child) parent;
            System.out.println("method1 - Child로 변환 성공");
        } else {
            System.out.println("method1 - Child로 변환되지 않음");
        }
    }

    public static void method2(Parent parent) {
        Child child = (Child) parent;
        System.out.println("method2 - Child로 변환 성공");
    }

    public static void main(String[] args) {
        Parent parentA = new Child();
        method1(parentA);
        method2(parentA);

        Parent parentB = new Parent();
        method1(parentB);
        method2(parentB); //예외 발생
    }
}
```

검증? 검문소느낌이네

유효성 검사 필요!

도구로는 되지만 실질적으로는 에러

```
method1 - Child로 변환 성공
method2 - Child로 변환 성공
method1 - Child로 변환되지 않음
Exception in thread "main" java.lang.ClassCastException: Parent cannot be cast t
o Child
        at InstanceofExample.method2(InstanceofExample.java:12)
        at InstanceofExample.main(InstanceofExample.java:23)
```

## InstanceofExample.java_ 객체 타입 확인

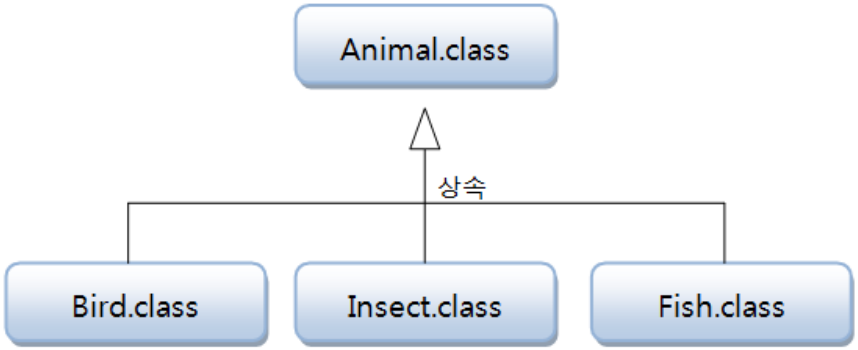
# 8절. 추상 클래스(Abstract Class)

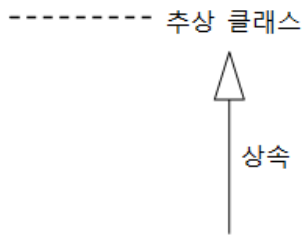
## ❖ 추상 클래스 개념

- 추상(abstract)
  - 실체들 간에 공통되는 특성을 추출한 것
    - 예1: 새, 곤충, 물고기→ 동물 (추상)
    - 예2: 삼성, 현대, LG → 회사 (추상)

- 추상 클래스(abstract class)
  - 실체 클래스들의 공통되는 필드와 메소드 정의한 클래스
  - 추상 클래스는 실체 클래스의 부모 클래스 역할 (단독 객체 X)

*실체 클래스: 객체를 만들어 사용할 수 있는 클래스



# 8절. 추상 클래스(Abstract Class)

## ❖ 추상 클래스의 용도

- 실체 클래스의 공통된 필드와 메소드의 이름 통일할 목적
  - 실체 클래스를 설계자가 여러 사람일 경우,
  - 실체 클래스마다 필드와 메소드가 제각기 다른 이름을 가질 수 있음
- 실체 클래스를 작성할 때 시간 절약
  - 실체 클래스는 추가적인 필드와 메소드만 선언
- 실체 클래스 설계 규격을 만들고자 할 때
  - 실체 클래스가 가져야 할 필드와 메소드를 추상 클래스에 미리 정의
  - 실체 클래스는 추상 클래스를 무조건 상속 받아 작성



# 8절. 추상 클래스(Abstract Class)

## ❖ 추상 클래스 선언

- 클래스 선언에 abstract 키워드 사용
  - New 연산자로 객체 생성하지 못하고 상속 통해 자식 클래스만 생성 가능

```
public abstract class 클래스 {
   //필드
   //생성자
   //메소드
}
```



# Phone.java_추상 클래스

```java
public abstract class Phone {
    //필드
    public String owner;

    //생성자
    public Phone(String owner) {
        this.owner = owner;
    }

    //메소드
    public void turnOn() {
        System.out.println("폰 전원을 켭니다.");
    }
    public void turnOff() {
        System.out.println("폰 전원을 끕니다.");
    }
}
```

추상도 하나만 가능?

생성자가 코드한줄이라도 있으면 개발자가 전적으로 전부

```java
public class SmartPhone extends Phone {
    //생성자
    public SmartPhone(String owner) {
        super(owner);
    }
    //메소드
    public void internetSearch() {
        System.out.println("인터넷 검색을 합니다.");
    }
}
```

여기로 오버라이드해야 더 추상클래스의 본질에 맞다

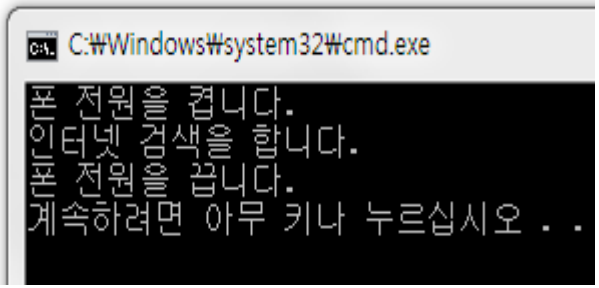
# SmartPhone.java_실체 클래스

# PhoneExample.java_추상 클래스

```java
public class PhoneExample {
    public static void main(String[] args) {
        //Phone phone = new Phone(); (x)

        SmartPhone smartPhone = new SmartPhone("홍길동");

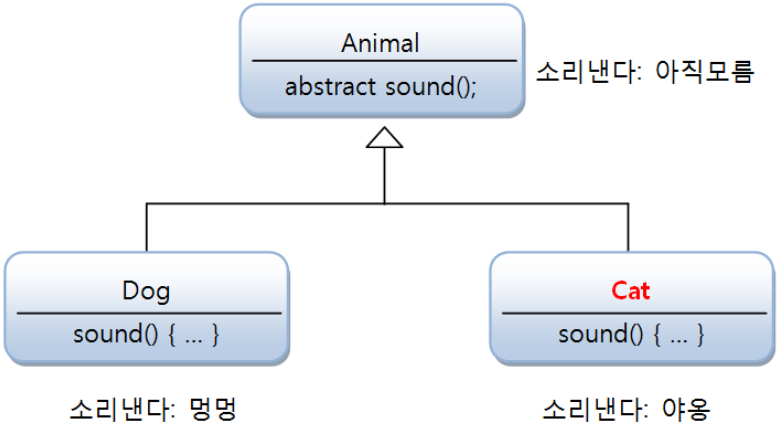
        smartPhone.turnOn();
        smartPhone.internetSearch();
        smartPhone.turnOff();
    }
}
```



# 8절. 추상 클래스(Abstract Class)

## ❖ 추상 메소드와 오버라이딩(재정의)

- 메소드 이름 동일하지만, 실행 내용이 실체 클래스마다 다른 메소드
- 예: 동물은 소리를 낸다. 하지만 실체 동물들의 소리는 제각기 다르다.
- 구현 방법
  - 추상 클래스에는 메소드의 선언부만 작성 (추상 메소드)
  - 실체 클래스에서 메소드의 실행 내용 작성(오버라이딩(Overriding))




```
public abstract class Animal {
    public abstract void sound();
}
```

```
public class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("멍멍");
    }
}
```

```
public class Cat extends Animal {
    @Override
    public void sound() {
        System.out.println("야옹");
    }
}
```

## Dog.java_추상 메소드 오버라이딩

```java
public class Dog extends Animal {
    public Dog() {
        this.kind = "포유류";
    }

    @Override
    public void sound() {
        System.out.println("멍멍");
    }
}
```

0x90

## Cat.java_추상 메소드 오버라이딩

여기 abstract 넣어야함

```java
public class Cat extends Animal {
    public Cat() {
        this.kind = "포유류";
    }

    @Override
    public void sound() {
        System.out.println("야옹");
    }
}

```

재정의 안 하면 추상클래스로 돼버림

0x40

```java
public abstract class Animal {
    public String kind;

    public void breathe() {
        System.out.println("숨을 쉽니다.");
    }

    public abstract void sound();
}
```

처리부가 없음

## Animal.java_추상 메소드 선언



# AnimalExample.java_실행 클래스

```
1  public class AnimalExample {
2      public static void main(String[] args) {
3          Dog dog = new Dog();   0x90
4          Cat cat = new Cat();
5          dog.sound();
6          cat.sound();
7          System.out.println("-----");
8
9          //변수의 자동 타입 변환
10         Animal animal = null;
11         animal = new Dog();
12         animal.sound();
13         animal = new Cat();
14         animal.sound();
15         System.out.println("-----");
16
17         //매개변수의 자동 타입 변환
18         animalSound(new Dog());
19         animalSound(new Cat());
20     }
21
22     public static void animalSound(Animal animal) {
23         animal.sound();
24     }
25 }
```

